The Scaly Programming Language

The Scaly Programming Language

Table of Contents

I First Steps **11**

1 Introduction 12

1.1 Overview 12

1.2 A Language Still Changing 12

2 Tutorial 12

2.1 Getting Started 12

2.1.1 Installing Scaly 13

2.1.2 Hello, World! 13

2.1.3 Trying Things Out 14

2.2 Using Scaly with Visual Studio Code on macOS and Linux 14

2.2.1 Installing the Scaly Extension 15

2.2.2 What the Language Server Gives You 15

2.2.3 Writing Scaly with GitHub Copilot 16

2.2.4 Run, Build and Test 16

2.2.5 Starting the Debugger 17

2.2.6 Breakpoints in Scaly Files 17

2.2.7 Setting a Breakpoint and Stepping 17

2.2.8 Stepping into Functions and the Call Stack 18

2.2.9 Arguments and the Environment 19

2.2.10 What the Debugger Shows 20

2.3 Scaly on Windows 20

2.3.1 Using Scaly with Visual Studio Code on Windows 21

2.3.1.1 Installing Scaly, VS Code and the Extensions 21

2.3.1.2 Checking the Language Server 21

2.3.1.3 Run, Build and Test 22

2.3.1.4 Setting a Breakpoint and Stepping 23

2.3.1.5 Stepping into Functions and the Call Stack 23

2.3.1.6 Arguments and the Environment 24

2.3.1.7 What the Debugger Shows 25

2.3.2 Using Scaly with Visual Studio 26

2.3.2.1 Installing Visual Studio, Scaly and the Extension 26

2.3.2.2 Opening the Folder and Checking the Language Server 27

2.3.2.3 Building and Running 27

2.3.2.4 Setting a Breakpoint and Stepping 28

2.3.2.5 Stepping into Functions and the Call Stack 28

2.3.2.6 Arguments and the Environment 29

2.3.2.7 Tests in the Test Explorer 29

2.3.2.8 What Visual Studio Shows 30

2.4 A Tour of Scaly 30

2.4.1 Values and Bindings 31

2.4.2 Operators and Operations 31
2.4.3 Control Flow 32
2.4.4 Functions 32
2.4.5 Procedures and Mutation 33
2.4.6 Structures and Methods 33
2.4.7 Unions, Options, and choose 34
2.4.8 Modules and Imports 34
2.5 Putting It Together 35
II Scaling 37
3 Why Scaly Can Scale 38
3.1 Two Rounds at Once 38
3.2 Functions Read, Procedures Write 38
3.3 A Procedure Says What It Writes 39
3.4 Memory That Is Not Shared 40
3.5 The Same Program on One Core 40
4 Loops on All Cores 41
4.1 A Plain for 41
4.2 What a Round May Do 42
4.3 What Keeps a Loop in Order 43
4.4 Sums, Products and the Largest 44
4.5 Values a Round Keeps 45
4.6 When the Cores Are Worth It 46
5 Recursion That Forks 46
5.1 Two Calls at Once 46
5.2 What Forks 47
5.3 Forks Nobody Waits For 49
5.4 An Uneven Tree 50
6 Lanes 51
6.1 Four Numbers in One 51
6.2 Masks 52
6.3 Lanes from Memory 53
6.4 Mandelbrot, Eight Points at a Time 54
6.5 Lanes and Cores 56
7 Many Connections 57
7.1 Waiting Is Not Working 57
7.2 A Server 57
7.3 The Same Calls Everywhere 60
7.4 One Scheduler per Core 60
7.5 A Region per Connection 60
8 The Graphics Card 61
8.1 Thousands of Small Cores 61
8.2 What a Card Can Run 62

8.3 The Same Numbers 64
8.4 Where a Loop Runs 65
8.5 The Numbers a Card Computes 66
9 More Than One Machine 67
9.1 Where Memory Ends 67
9.2 Nodes and Channels 68
9.3 A Message Is a Region 70
9.4 One Program, One Build 71
9.5 When a Node Fails 71
III Language Reference 72
10 Lexical Structure 73
10.1 Overview 73
10.2 Whitespace and Line Breaks 73
10.3 Comments 73
10.4 Literals 73
10.4.1 Integer Literals 73
10.4.2 Floating-Point Literals 74
10.4.3 String Literals 74
10.4.4 Character Literals 75
10.4.5 Boolean Literals 75
10.5 Operators and Special Symbols 75
10.5.1 The Equals Sign 75
10.6 Identifiers 76
11 Type System 77
11.1 Overview 77
11.2 Intrinsic Types (Future Direction) 77
11.3 Type Aliases (Future Direction) 77
11.4 Backend Preludes (Future Direction) 78
11.5 Literal Type Inference 79
11.6 Tuples and the Empty Tuple 79
11.7 References 80
11.7.1 Low-Level Pointers 80
11.7.2 Nullable Values 80
11.8 Parameter Passing 81
11.8.1 Implementation 81
11.8.2 Mutation via Procedures 82
11.9 Constructors 82
11.9.1 Implicit Constructors 82
11.9.2 Explicit Constructors 82
11.9.3 The `this` Prefix 83
11.9.4 Complete Initialization 83
11.9.5 Constructor Return 83

11.9.6 Types That Allocate 84
11.10 Lambdas and Closures 84
11.10.1 Capturing Variables 84
11.10.2 Function Types: Delegates 84
11.10.3 Closure Lifetime 85
11.10.4 Implementation 86
11.11 Error Handling 86
11.11.1 The `throws` Clause 86
11.11.2 Single Error Type 87
11.11.3 The `try/when` Pattern 87
11.11.4 Error Propagation 87
11.11.5 Error Lifetime 88
11.12 Union Types 88
11.12.1 Memory Layout 88
11.12.2 Pattern Matching with `choose` 89
11.12.3 Option Type 89
11.12.4 Option Optimization 89
11.13 Generics 90
11.13.1 Monomorphization 91
11.13.2 Name Mangling 91
11.14 Lifetimes and Memory Regions 91
11.14.1 Allocation Is Inferred 91
11.14.2 Escape Checking 92
11.14.3 Explicit Lifetime Suffixes 92
11.14.4 By-Value Returns 93
12 Grammar 93
13 Definitions 98
13.1 Overview 98
13.2 Functions, Procedures, and Operators 98
13.3 Definitions 100
13.3.1 Structure Definitions 100
13.3.2 Union Definitions 100
13.3.3 Function Names 100
13.3.4 Operator Definitions 101
13.3.5 Lazy Parameters 101
14 Expressions 102
14.1 Overview 102
14.2 Literals 103
14.2.1 Boolean Literals 103
14.2.2 Integer Literals 103
14.2.3 Hexadecimal Literals 104
14.2.4 Floating-Point Literals 104

14.2.5 String Literals 104
14.2.6 Character Literals 104
14.3 Operations 105
14.3.1 Prefix Operators 105
14.3.2 Operator Chains 105
14.3.3 Application Before Operators 106
14.3.4 Parentheses and Grouping 106
14.3.5 Casts Take the Whole Operation 107
14.3.6 Identifier Shapes 107
14.3.7 Let Bindings 107
14.4 Tuples and Function Values 108
14.4.1 Tuples 108
14.4.2 Applying a Bound Tuple 108
14.4.3 Functions as Values 109
14.5 Comparison Operators 109
14.5.1 Equality 110
14.5.2 Relational Operators 110
14.6 Boolean Operators 110
14.6.1 Boolean Negation 110
14.6.2 Boolean And/Or 111
14.7 Bitwise Operators 111
14.7.1 Bit Shifts 112
14.8 Lambdas 112
14.8.1 Lambda Application 112
14.8.2 Captures 112
14.8.3 Captures in Braced Bodies 113
14.8.4 Embedded Calls 113
14.8.5 Delegate Values 114
14.8.6 Closures 114
14.9 Choose Expressions 114
14.9.1 Choose on Variant Construction 114
14.9.2 Choose with Binding 115
14.9.3 Multiple When Clauses 115
14.10 Optional Types 115
14.10.1 Is Null Check 115
14.10.2 Optional Type Sugar 116
14.10.3 Generic Option Types 116
14.11 Variant Construction 116
14.11.1 Construct Variants 116
14.11.2 Variant Is Expression 116
15 Control Flow 117
15.1 Overview 117

15.2 If Expressions 117
15.2.1 Basic If 117
15.2.2 If with Bindings 117
15.2.3 Else-If Chains without a Final Else 118
15.3 While Loops 118
15.3.1 Basic While 118
15.3.2 While with Accumulation 119
15.4 For Loops 119
15.4.1 Basic For 119
15.4.2 For with Accumulation 119
15.5 Break and Continue 120
15.5.1 Break Statements 120
15.5.2 Continue Statements 120
15.5.3 Nested Loops 120
15.5.4 Labeled Break and Continue 121
15.6 Repeat Loops 121
15.6.1 Repeat and Break Value 121
15.7 Mutable Variables 122
15.7.1 Var and Assignment 122
15.8 Match 122
16 Modules and Packages 123
16.1 Overview 123
16.2 Modules 123
16.3 Packages 124
16.3.1 Packages from a git repository 124
16.3.2 The cache 126
16.3.3 Versions, and what publishing means 126
16.3.4 Starting a package 128
16.3.5 The programs of a package 129
16.4 Imports and Visibility 130
16.4.1 What Is Visible Without an Import 130
16.4.2 Importing a Single Type 130
16.4.3 Importing Everything from a Module 131
16.4.4 When a Type Is Not Visible 131
A. The Tools 131
A.1 Installation 131
A.1.1 Ready to run, or built on your machine 132
A.1.2 Versions, updates, removal 132
A.2 scaly 133
A.2.1 Building for Production 134
A.2.2 Installing a Program 134
A.3 scalyc 135

A.4 scalyls 136
B. Standard Library and Packages 136
B.1 The Standard Library 136
B.2 Packages Beside It 137
C. Bootstrap History and the Seed 138
D. On the Shoulders of Giants 139
D.1 The Text of a Program 139
D.2 Names and Values 141
D.3 Control Flow 142
D.4 Types and Definitions 142
D.5 Memory 144
D.6 Self-Scaling 145
D.7 Modules and Packages 145
D.8 An Unexpected Ancestor 146
D.9 And What Is Scaly's Own? 146
D.10 The Implementation 147

List of Examples

11-1. LLVM Prelude (excerpt) 78
11-2. JavaScript Prelude (excerpt) 78

I First Steps

1 Introduction

1.1 Overview

Scaly is a compiled programming language. Three ideas shape it:

- *Self-scaling* — A plain `for` loop is spread across the cores when the compiler can prove that its iterations are independent and the work pays for it; there is no syntax to ask for it. The proof rests on the difference between a `function`, which only reads its arguments and returns a value, and a `procedure`, which may change things.
- *Memory without ceremony* — Memory is managed in regions the compiler infers: no garbage collector, no reference counting, no manual freeing, and in ordinary code no annotations. An escape checker proves at compile time that no reference outlives its memory.
- *A uniform grammar* — Operators are ordinary identifiers rather than keywords baked into the grammar, so the language and its standard library share one small, regular syntax. An operation is read from left to right, the type of the value in hand saying what comes next; how strongly operators bind is stated by the standard library, not by the compiler.

Simple programs stay simple — a program is a file of statements, with nothing to declare and nothing to import — and the rest of the language is there when a program needs it.

1.2 A Language Still Changing

Scaly is below version 1.0, and it is still changing. Its syntax, its rules and its standard library move where something better is found — recently the assignment became `:=`, and the precedence table of the compiler gave way to the binding strengths the standard library declares. A version below 1.0 promises nothing about the next one.

What stays is a version that is out. A published toolchain and a published package do not change afterwards, so a program can go on using the ones it was written with: the toolchain by its number (Section A.1.2), a package by the version it declares (Section 16.3.3).

2 Tutorial

This chapter is a hands-on introduction to Scaly. It gets you from an empty file to a running program, then takes you on a quick tour of the language. Later chapters cover everything here in depth; the goal now is to write and run real code as fast as possible.

2.1 Getting Started

To try Scaly before installing anything, a browser is enough. The tour at

`scaly.io/play/tour.html` runs the programs of Section 2.4 step by step, and you can change them; the REPL at `scaly.io/play` is the interactive session of Section 2.1.3. The compiler runs in the browser tab itself: nothing is sent anywhere.

2.1.1 Installing Scaly

Scaly is installed by one command. On macOS and Linux, in a terminal:

```
curl -fsSL https://scaly.io/install.sh | sh
```

On macOS 26.4 and newer, Terminal stops a pasted command that downloads something with a warning, “Possible Malware, Paste Blocked” — this one too. Choose Paste Anyway; the script it runs can be read first at `scaly.io/install.sh`.

On Windows, in PowerShell (Section 2.3 goes through the installation there step by step, and through working with VS Code and Visual Studio):

```
irm https://scaly.io/install.ps1 | iex
```

Open a new terminal afterwards. The installer has put three commands on your PATH: **scaly**, the tool you will use most, **scalyc**, the compiler by itself, and **scalyls**, the language server your editor talks to. Everything else it installed lives in `~/.scaly`. Section A.1 has the details: which systems get Scaly ready to run, what the others need, how to ask for a particular version, and how to remove it again.

2.1.2 Hello, World!

A Scaly program is just a file of statements — there is no `main` function to declare and nothing to import. Put this single line in a file called `hello.scaly`:

```
print "Hello, World!"
```

Run it:

```
scaly run hello.scaly
```

The program prints:

```
Hello, World!
```

`scaly run` compiles the file in memory and runs it at once; nothing is left on disk. To get a program you can keep and hand to someone else, build it:

```
scaly build hello.scaly -o hello
./hello
```

Add `--release` for the optimised build: the whole program — your code, the standard library and every package it uses — is optimised as one piece.

`scaly new hello` writes such a start for you: a directory `hello` with `hello.scaly` in it — a function, a test for it (**`scaly test hello.scaly`** runs every function whose name begins with `test`) and the line that prints.

That is the whole edit-compile-run loop. Everything that follows is just more interesting things to put between the braces.

2.1.3 Trying Things Out

scaly without arguments starts an interactive session. An expression shows its value and its type; a `let` or `var` keeps a value for the entries that follow:

```
$ scaly
scaly> 6 * 7
it: int = 42
scaly> let name "Scaly"
name: String = "Scaly"
scaly> :where name
  name: String
    the value   8 bytes, on the session page of its entry
    its text    5 bytes, on a page of its own
scaly> :quit
```

`:where` shows where a value lives — Scaly manages memory in regions, and here you can watch it do so. **`:help`** lists the other commands.

2.2 Using Scaly with Visual Studio Code on macOS and Linux

On Windows, follow Section 2.3 instead: the installation is different there.

You can drive the whole edit-compile-run-debug loop from Visual Studio Code. The *Scaly* extension provides syntax highlighting and connects the editor to the **scalyls** language server you installed in Section 2.1.1 — giving live error squiggles, hover, go-to-definition, completion, rename, and more.

If you do not have VS Code yet, download it from `code.visualstudio.com` (installers for macOS and Linux; on Ubuntu, **`sudo snap install code --classic`** also works).

2.2.1 Installing the Scaly Extension

The Scaly extension is distributed as a `.vsix` package on scaly.io. Download `scaly.io/downloads/scaly-vscode.vsix`, then install it from within VS Code: open the Extensions view, click the ⋯ menu in its title bar, choose Install from VSIX…, and pick the downloaded file.

The extension launches **scalyls** from your PATH, so install the toolchain first (Section 2.1.1). If VS Code was already running when you installed the toolchain, restart it (not just the window) so its process picks up the updated PATH. Should the server binary live somewhere unusual, point the `scaly.server.path` setting at it.

2.2.2 What the Language Server Gives You

Open any folder, create a file `double.scaly`, and type a small program:

```
function double(n: int) returns int
    n * 2

let d double(21)
if d = 42
    print "doubled!"
```

As you type, the language server is at work. Things to try:

- *Diagnostics* — misspell a call (say, change `double(21)` to `doubel(21)`): a red squiggle appears and the Problems panel shows `function not found: doubel`. Fix it and the squiggle clears.
- *Hover* — rest the mouse on `double` or a binding to see its signature or type.
- *Go to Definition* (**F12**) and *Find All References* (**Shift**+**F12**) — work across files and into the standard library.
- *Completion* — names in scope and members appear as you type.
- *Rename* (**F2**) — renames a symbol everywhere it is used, shadow-aware.
- *Outline and symbols* — the Outline view and **Cmd**+**Shift**+**O** list the file's functions and types; **Cmd**+**T** searches symbols across the workspace. (Use **Ctrl** instead of **Cmd** on Linux.)
- *Signature help* — parameter hints while you type a call; the hover over a construction also tells where it is allocated, on the stack or in which region; *semantic highlighting* and *folding* come for free.

If the server ever needs a kick — say, after you rebuilt it — run Scaly: Restart Language Server from the Command Palette.

2.2.3 Writing Scaly with GitHub Copilot

An AI assistant has seen little Scaly. Left to itself it writes what looks like Rust or Swift, and Scaly differs exactly there: = compares and `:=` assigns, `not` negates, a function cannot print. Two things make it write Scaly: a file that teaches it the language, and the compiler, whose messages it reads.

Put the language reference for assistants into your folder, in a terminal at the folder's root:

```
mkdir -p .github
curl -fsSL https://scaly.io/AGENTS.md -o .github/copilot-instructions.md
```

GitHub Copilot reads `.github/copilot-instructions.md` with every chat request in this folder. Other assistants read the same text under the name `AGENTS.md` at the folder's root.

Open the Chat view, set its mode to Agent, and say what you want — say, “Write a Scaly program that counts the words of the files named on its command line, with a test for the counting function.” In agent mode Copilot writes the files, runs **scaly run** and **scaly test** in the terminal, and reads the errors the language server reports in the Problems panel. Scaly's messages name the line and usually the fix, and Copilot corrects itself from them. Let it run the program and the tests before you accept what it wrote.

2.2.4 Run, Build and Test

The extension offers three tasks for the file in the active editor, with no `tasks.json` to write. Terminal > Run Task… lists them under scaly:

scaly: run

> **`scaly run`**: the program is compiled in memory and runs in the terminal panel.

scaly: build

> **`scaly build -g`**: the program, with debug information, beside the file. ⇧⌘**B** (**Ctrl**+**Shift**+**B**) offers it.

scaly: test

> **`scaly test`**: the file's tests.

A compiler error lands in the Problems panel, a click away from its line. To change a task — another output, an argument —, choose Terminal > Configure Tasks… and pick it: VS Code writes it into `.vscode/tasks.json`, where it can be edited.

2.2.5 Starting the Debugger

Debugging needs nothing more. The extension drives **lldb-dap**, the debugger that comes with Apple's Command Line Tools (the installer fetches them where they are missing), with Xcode, and with LLVM — on Ubuntu, **`sudo apt install lldb-21`**. Open a `.scaly` file and press **F5**: the extension builds it with debug information and starts it under the debugger. Should **lldb-dap** live somewhere unusual, point the `scaly.debugAdapter.path` setting at it.

For a configuration of your own — arguments, an environment, another program — you write a `.vscode/launch.json`, and that needs a folder open (File > Open Folder…), not just a file. Open the Run and Debug view: the icon with the play triangle and the bug in the Activity Bar on the left, or ⇧⌘**D** (**Ctrl**+**Shift**+**D**). As long as the folder has no `launch.json`, the view shows a Run and Debug button — the same as **F5** — and beneath it the link create a launch.json file. Click the link with a `.scaly` file in the editor; should VS Code ask which debugger, choose Scaly. It writes the file, opens it, and what it writes debugs the open file:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "type": "scaly",
            "request": "launch",
            "name": "Debug the open Scaly file",
            "source": "${file}"
        }
    ]
}
```

`source` is built before the launch; `program` names an executable to debug as it is, and `args`, `env` and `cwd` go to the program.

Section 2.2.9 shows a configuration with arguments and an environment.

2.2.6 Breakpoints in Scaly Files

VS Code only lets you set breakpoints in files whose language an installed extension has declared debuggable. The Scaly extension from Section 2.2.1 does exactly that for `.scaly` files, so with it installed, clicking in the editor gutter places a breakpoint (a red dot appears) — nothing to configure.

2.2.7 Setting a Breakpoint and Stepping

Put this program in `area.scaly` in your workspace folder. It computes a rectangle's area from two mutable bindings — small, but enough to stop and inspect some values:

```
var width 8
var height 5
var area width * height
if area = 40
    print "the area is 40"
else
    print "unexpected area"
```

Click in the gutter to the left of the `var area width * height` line to set a breakpoint (a red dot appears), then press **F5**. The program builds and runs, and execution stops at that line before it executes — at which point `width` and `height` already have values but `area` does not yet.

With execution paused you can:

- Read `width` and `height` in the Variables pane, or hover them in the editor to see their values.
- Step one line at a time with **F10** (Step Over) — after stepping past the breakpoint line, `area` appears with the value `40`. Step into a called function with **F11**.
- Press **F5** again to continue to the program's end.

Breakpoints, stepping, and variable inspection all rely on the `-g` debug information; without it the program still runs, but the debugger has no source lines to stop on.

2.2.8 Stepping into Functions and the Call Stack

Stepping becomes more interesting once a function is involved. **F10** (Step Over) runs a call as a single step; **F11** (Step Into) descends into the called function so you can watch it run. Put this program in `rectangle.scaly` in your workspace folder — it factors the area calculation out into a function:

```
function area(w: int, h: int) returns int
{
    var result w * h
    result
}

var width 8
var height 5
var a area(width, height)
if a = 40
    print "the area is 40"
else
    print "unexpected area"
```

Set a breakpoint on the `var a area(width, height)` line and press **F5**. When execution stops there, `width` and `height` show in the Variables pane. Now press **F11** instead of **F10**: the editor jumps into the body of `area` and stops on `var result w * h`.

The Call Stack pane (bottom-left of the Run and Debug view) now lists two frames — `area` on top and `main` below it — showing exactly how you got here. Click a frame to switch the Variables pane and the editor to that frame's context: selecting `main` shows `width` and `height` again, while `area` shows its own local `result` (still unassigned at this point, just as `area` was before its line ran in the previous example).

From inside the function you can keep stepping, press **Shift**+**F11** (Step Out) to return to the caller, or **F5** to continue to the end. The function-and-procedure model is covered in Section 2.4.4 and Section 2.4.5.

2.2.9 Arguments and the Environment

Put this program in `greet.scaly` in your workspace folder. It greets everyone named on its command line, with the greeting the environment variable GREETING holds, and “Hello” where there is none:

```
var greeting os.environment_variable("GREETING")
if greeting.length = 0
    greeting := "Hello"
for name in os.arguments()
    print "`greeting`, `name`!"
```

`os.arguments()` answers the arguments the program was started with, its own name left out; `os.environment_variable` reads a variable of the environment, and is empty where it is not set. In a terminal:

```
$ GREETING=Hi scaly run greet.scaly Ada Grace
Hi, Ada!
Hi, Grace!
```

Under the debugger, the arguments and the environment come from `.vscode/launch.json` (Section 2.2.5 says how to create one). A `launch.json` may hold several configurations; the drop-down at the top of the Run and Debug view chooses the one **F5** starts. Here the first builds `greet.scaly` and runs it with two arguments, a variable in its environment and the workspace folder as its working directory; the second debugs a program that is already built — with **`scaly build -g`**, or there are no line tables for a breakpoint to bind to:

```
{
    "version": "0.2.0",
```

```
    "configurations": [
        {
            "type": "scaly",
            "request": "launch",
            "name": "Debug greet with arguments",
            "source": "${workspaceFolder}/greet.scaly",
            "args": ["Ada", "Grace"],
            "env": { "GREETING": "Hello" },
            "cwd": "${workspaceFolder}"
        },
        {
            "type": "scaly",
            "request": "launch",
            "name": "Debug the built server",
            "program": "${workspaceFolder}/build/server",
            "args": ["--port", "8080"]
        }
    ]
}
```

Choose Debug greet with arguments in the drop-down, set a breakpoint on the `print` line and press **F5**. The program stops there with `name "Ada"` in the Variables pane; **F5** runs the next round, with `"Grace"`. What the program prints appears in the Debug Console.

2.2.10 What the Debugger Shows

For whichever stack frame is selected:

- Breakpoints land on top-level statements, bindings and function bodies, and line by line inside loop and conditional bodies — on a single statement within a `while`, `for`, `if` or `choose`, not just on the statement around it.
- The Variables pane shows a function's arguments and its `let` and `var` bindings: numbers and booleans; structures, unions and pointers with their fields to expand, a union's variant by its name; a `String` as its text; a `Vector`, `Array` or `List` with its elements.
- The Watch view and Debug Console evaluate expressions as C/C++ and do not understand Scaly names. To read a value, use the Variables pane or hover over the name in the editor.

2.3 Scaly on Windows

Everything in this chapter works on Windows 10 and 11, on x64 and on Arm. Choose how you want to work: in Visual Studio Code (Section 2.3.1) or in Visual Studio (Section 2.3.2).

2.3.1 Using Scaly with Visual Studio Code on Windows

The following sections describe how to use Visual Studio Code to develop Scaly programs.

2.3.1.1 Installing Scaly, VS Code and the Extensions

1. Install Scaly. Click Start, type **`PowerShell`** and press **Enter**. In the window that opens, paste this line (a right-click pastes) and press **Enter**:

   ```
   irm https://scaly.io/install.ps1 | iex
   ```

   Along with Scaly, the installer sets up Microsoft's C++ build tools, which Scaly needs to build programs. This download is several gigabytes and takes a few minutes. When Windows asks whether to allow the installation, answer Yes. When the Scaly installer ends with `scaly build: ok`, everything is in place. Close PowerShell.

2. Install VS Code: download the installer from `code.visualstudio.com` and run it. If VS Code was installed and running already, close every one of its windows now, so that it starts again and finds Scaly.

3. Download the Scaly extension, `scaly.io/downloads/scaly-vscode.vsix`.

4. Start VS Code and open the Extensions view: the icon with the four squares in the Activity Bar on the left, or **Ctrl**+**Shift**+**X**. Click the ⋯ menu in the view's title bar, choose Install from VSIX…, and pick `scaly-vscode.vsix` from your `Downloads` folder.

5. In the same view, type **`C/C++`** into the search box at the top. Choose the extension named C/C++ whose publisher is Microsoft (its identifier is `ms-vscode.cpptools`) and click Install; it brings the debugger. If you skip this step, the first **F5** asks for it with a button Install C/C++.

The Scaly extension launches **scalyls** and **scaly** from your PATH; should they live somewhere unusual, point the `scaly.server.path` and `scaly.tool.path` settings at them.

2.3.1.2 Checking the Language Server

1. Make a folder for this tutorial: choose File > Open Folder…, go to `Documents`, click New folder, name it **`scaly-tutorial`**, double-click it so that the dialog shows its (still empty) inside, and click Select Folder.

2. Trust the folder. VS Code opens a new folder in Restricted Mode, where it starts no language server and no debugger; a bar at the top of the window says so. Click Manage in that bar, then Trust. (Should VS Code ask in a dialog instead, answer Yes, I trust the authors.)

3. Choose File > New File…, save it right away (**Ctrl**+**S**) as `double.scaly` in that folder, and type:

   ```
   function double(n: int) returns int
   ```

```
    n * 2

let d double(21)
if d = 42
    print "doubled!"
```

The words `function`, `returns` and `let` are coloured: the extension knows the file.

4. Change `double(21)` to `doubel(21)`. A red squiggle appears under it, and the Problems panel (**Ctrl**+**Shift**+**M**) shows `function not found: doubel`: the language server is running. Change it back; the squiggle clears.

5. Rest the mouse on `double` to see its signature, press **F12** on it to go to its definition and **Shift**+**F12** to list its uses, **F2** to rename it everywhere. **Ctrl**+**Shift**+**O** lists the file's functions and types, **Ctrl**+**T** searches symbols across the folder.

Section 2.2.2 lists what else the language server does. If the squiggle never appears, VS Code did not find the language server: close all of its windows and start it again.

To write Scaly with GitHub Copilot, follow Section 2.2.3; in PowerShell the reference for assistants is fetched with:

```
mkdir -Force .github
irm https://scaly.io/AGENTS.md -OutFile .github\copilot-instructions.md
```

2.3.1.3 Run, Build and Test

With `double.scaly` in the editor, choose Terminal > Run Task…, then scaly. Three tasks are offered:

scaly: run

> **`scaly run`**: the program runs in the terminal panel at the bottom and prints `doubled!`.

scaly: build

> **`scaly build -g`**: writes `double.exe`, the program, and `double.pdb`, its debug information, beside the file. **Ctrl**+**Shift**+**B** offers it too.

scaly: test

> **`scaly test`**: the file's tests.

A compiler error lands in the Problems panel, a click away from its line. Should scaly: build say that it has nothing to link with, the C++ build tools are missing: run the Scaly installer again (Section 2.3.1.1) and answer Yes when Windows asks. To change a task, choose Terminal >

Configure Tasks… and pick it: VS Code writes it into `.vscode\tasks.json`, where it can be edited.

2.3.1.4 Setting a Breakpoint and Stepping

1. Create `area.scaly` in the folder, as `double.scaly` above, with this program:

```
var width 8
var height 5
var area width * height
if area = 40
    print "the area is 40"
else
    print "unexpected area"
```

2. Click in the gutter to the left of the line number of `var area width * height`. A red dot appears: a breakpoint. (**F9** sets one on the line the cursor is in.)

3. Press **F5**. Should VS Code ask which debugger to use, choose Scaly. The extension builds `area.exe` and `area.pdb`, then starts the program under the Visual Studio debugger. Should a message say that the C/C++ extension is missing, click Install C/C++, wait until it is installed, and press **F5** again.

4. The program stops at the breakpoint, before the line runs; the line is highlighted in yellow. The Run and Debug view on the left shows the Variables pane: `width` is `8`, `height` is `5`, `area` has no value yet. Resting the mouse on a name in the editor shows its value too.

5. Press **F10** (Step Over): the line runs, and `area` shows `40`.

6. Press **F5** again: the program runs to its end. What it prints, `the area is 40`, appears in the Debug Console (**Ctrl**+**Shift**+**Y**).

Every **F5** builds the file again, so a change to the source is always in the program you debug.

2.3.1.5 Stepping into Functions and the Call Stack

1. Create `rectangle.scaly` in the folder, with the area calculation in a function:

```
function area(w: int, h: int) returns int
{
    var result w * h
    result
}

var width 8
var height 5
var a area(width, height)
if a = 40
```

```
    print "the area is 40"
else
    print "unexpected area"
```

2. Set a breakpoint on `var a area(width, height)` and press **F5**. The program stops there.

3. Press **F11** (Step Into): the editor jumps into the body of `area` and stops on `var result w * h`.

4. The Call Stack pane, in the Run and Debug view below Variables, lists two frames: `area` on top, `main` below it, each with the program's name in front (`rectangle.exe!`). Click `main`: the Variables pane shows `width` and `height`. Click `area`: it shows `w`, `h` and `result`, still without a value.

5. Press **Shift**+**F11** (Step Out) to return to the caller, then **F5** to run to the end.

2.3.1.6 Arguments and the Environment

A program that is started with arguments and an environment gets them under the debugger from a `.vscode\launch.json`.

1. Create `greet.scaly` in the folder, as `double.scaly` above, with this program. It greets everyone named on its command line, with the greeting the environment variable GREETING holds, and "Hello" where there is none:

   ```
   var greeting os.environment_variable("GREETING")
   if greeting.length = 0
       greeting := "Hello"
   for name in os.arguments()
       print "`greeting`, `name`!"
   ```

   `os.arguments()` answers the arguments the program was started with, its own name left out; `os.environment_variable` reads a variable of the environment, and is empty where it is not set.

2. Open the Run and Debug view: the icon with the play triangle and the bug in the Activity Bar, or **Ctrl**+**Shift**+**D**.

3. Click the link create a launch.json file below the Run and Debug button. Should VS Code ask which debugger, choose Scaly. It writes `.vscode\launch.json` and opens it.

4. Replace its content with this and save it (**Ctrl**+**S**):

   ```
   {
       "version": "0.2.0",
       "configurations": [
           {
   ```

```
            "type": "scaly",
            "request": "launch",
            "name": "Debug greet with arguments",
            "source": "${workspaceFolder}/greet.scaly",
            "args": ["Ada", "Grace"],
            "env": { "GREETING": "Hello" },
            "cwd": "${workspaceFolder}"
        },
        {
            "type": "scaly",
            "request": "launch",
            "name": "Debug the open Scaly file",
            "source": "${file}"
        }
    ]
}
```

The first configuration builds `greet.scaly` and runs it with two arguments, a variable in its environment and the folder as its working directory. The second is what **F5** did so far: it builds and debugs the file in the editor.

5. Choose Debug greet with arguments in the drop-down at the top of the Run and Debug view.

6. Open `greet.scaly`, set a breakpoint on the `print` line and press **F5**. The program stops there; the Variables pane shows `name` as `"Ada"`.

7. Press **F5**: the next round stops with `"Grace"`. Press **F5** once more to run to the end. The Debug Console shows `Hello, Ada!` and `Hello, Grace!`.

A configuration may also debug a program built beforehand: `"program": "${workspaceFolder}/greet.exe"` in place of `source` runs it as it is (built with `-g`, or there is nothing for a breakpoint to bind to).

2.3.1.7 What the Debugger Shows

- Breakpoints land on top-level statements, bindings and function bodies, and line by line inside loop and conditional bodies.
- The Variables pane shows a function's arguments and its `let` and `var` bindings: numbers and booleans; structures, unions and pointers with their fields to expand; a `String` as its text; a `Vector`, `Array` or `List` with its elements. A generic type is written as in C++, `Vector<int>`.
- The Watch view and the Debug Console evaluate expressions as C++ and do not understand Scaly. To read a value, use the Variables pane or hover over the name in the editor.

2.3.2 Using Scaly with Visual Studio

Visual Studio works on a folder of `.scaly` files, without a project file. The Scaly extension colours the source, runs the language server, builds and debugs a file with **F5**, and lists its tests in the Test Explorer. VS Code (Section 2.3.1) can have the same folder open at the same time.

2.3.2.1 Installing Visual Studio, Scaly and the Extension

Install Visual Studio first. Scaly builds programs with its C++ tools, and when it finds them, it installs nothing of its own.

1. Download Visual Studio 2026 from `visualstudio.microsoft.com`; the free Community edition will do. Run the installer. If Visual Studio 2026 is installed already, start the Visual Studio Installer from the Start menu instead and click Modify.

2. On the Workloads tab, tick Desktop development with C++. It brings the C++ debugger and the C++ tools Scaly builds with.

3. On a PC with an Arm processor (a Snapdragon, for instance; Settings > System > About says "ARM-based processor"), also tick the tools for Arm: in the Installation details on the right, under Desktop development with C++, tick C++ ARM64/ARM64EC build tools. On any other PC, skip this step.

4. Click Install (or Modify) and wait until the installer is done. If it asks you to restart your PC, do so.

5. Install Scaly. Click Start, type **`PowerShell`** and press **Enter**. In the window that opens, paste this line (a right-click pastes) and press **Enter**:

   ```
   irm https://scaly.io/install.ps1 | iex
   ```

   The installer finds the C++ tools of Visual Studio and fetches nothing more. When it ends with `scaly build: ok`, everything is in place. Close PowerShell.

6. Download the Scaly extension, `scaly.io/downloads/scaly-visualstudio.vsix`. If the browser warns that the file is not commonly downloaded, choose to keep it.

7. Close every Visual Studio window. Then open your `Downloads` folder and double-click `scaly-visualstudio.vsix`. The VSIX Installer opens and names your Visual Studio. Click Install. It mentions that the extension carries no digital signature; that is expected. When it says the installation is complete, click Close.

The extension finds **scaly** and **scalyls** on your PATH, else where the installer put them. If they are somewhere else, set their paths under Tools > Options > Scaly.

2.3.2.2 Opening the Folder and Checking the Language Server

1. Start Visual Studio. In its start window, choose Open a local folder. If Visual Studio is already open, choose File > Open > Folder… instead. Go to `Documents`. If there is no `scaly-tutorial` yet from Section 2.3.1, click New folder and name it **`scaly-tutorial`**. Double-click `scaly-tutorial`, so that the dialog shows its inside, and click Select Folder.

2. The Solution Explorer lists the folder's files. If it is not there, choose View > Solution Explorer, or press **Ctrl**+**Alt**+**L**.

3. If `double.scaly` is there already, from Section 2.3.1.2, double-click it. Otherwise, right-click the folder at the top of the Solution Explorer, choose Add > New File…, name it **`double.scaly`**, type this program, and save it with **Ctrl**+**S**:

   ```
   function double(n: int) returns int
       n * 2

   let d double(21)
   if d = 42
       print "doubled!"
   ```

   The words `function`, `returns` and `let` are coloured. The Scaly file has the Scaly lizard as its icon in the Solution Explorer.

4. Change `double(21)` to `doubel(21)`. A red squiggle appears under it, and the Error List (View > Error List) shows `function not found: doubel`: the language server is running. Change it back, and the squiggle clears.

5. Rest the mouse on `double` in its definition to see its signature. Press **F12** on a use to go to the definition, and **Shift**+**F12** to list every use. To rename it everywhere, press **Ctrl**+**R**, then **Ctrl**+**R** again. **Ctrl**+**T** searches symbols across the folder, and while you type, Visual Studio offers completions.

If the squiggle never appears, Visual Studio did not find the language server: check that Scaly is installed (Section 2.3.2.1), then close Visual Studio and start it again.

2.3.2.3 Building and Running

1. Right-click `double.scaly` in the Solution Explorer and choose Build. The Output window (View > Output) shows the command line and `Build succeeded.`, and `double.exe`, the program, and `double.pdb`, its debug information, appear beside the source.

2. With `double.scaly` in the editor, press **Ctrl**+**F5**. Visual Studio builds the file and runs the program without the debugger, in a console window of its own, which prints `doubled!`.

A compiler error goes to the Error List; double-click it to go to its line. If the build says that it has

nothing to link with, the C++ tools of Visual Studio are missing: go back to Section 2.3.2.1.

2.3.2.4 Setting a Breakpoint and Stepping

1. If `area.scaly` is there already, from Section 2.3.1.4, double-click it. Otherwise, add it to the folder as you added `double.scaly`, with this program:

```
var width 8
var height 5
var area width * height
if area = 40
    print "the area is 40"
else
    print "unexpected area"
```

2. Click in the grey margin left of `var area width * height`, or put the cursor on that line and press **F9**. A red dot appears: a breakpoint.

3. With `area.scaly` in the editor, press **F5**. Visual Studio builds the file, starts the program in a console window, and stops at the breakpoint before the line runs; a yellow arrow marks the line.

4. The Locals window at the bottom shows the bindings: `width` is `8`, `height` is `5`. If it is not there, choose Debug > Windows > Locals. Resting the mouse on a name in the editor shows its value too.

5. Press **F10** (Step Over): the line runs, and `area` shows `40`.

6. Press **F5** again: the program runs to its end and prints `the area is 40` into its console window.

F5 builds the file every time, so a change to the source is in the program you debug. The console window closes when the program ends. To read its output afterwards, untick Tools > Options > Debugging > Automatically close the console when debugging stops.

2.3.2.5 Stepping into Functions and the Call Stack

1. Add `rectangle.scaly` to the folder, with the area calculation in a function:

```
function area(w: int, h: int) returns int
{
    var result w * h
    result
}

var width 8
var height 5
var a area(width, height)
```

```
if a = 40
    print "the area is 40"
else
    print "unexpected area"
```

2. Set a breakpoint on `var a area(width, height)` and press **F5**.

3. Press **F11** (Step Into): the editor jumps into the body of `area` and stops on `var result w * h`.

4. The Call Stack window (Debug > Windows > Call Stack, or **Ctrl**+**Alt**+**C**) lists `rectangle.exe!area` on top and `rectangle.exe!main` below it. Double-click `main`: Locals shows `width` and `height`. Double-click `area`: it shows `w`, `h` and `result`.

5. Press **Shift**+**F11** (Step Out) to return to the caller, then **F5** to run to the end.

2.3.2.6 Arguments and the Environment

1. Add `greet.scaly` to the folder. It greets everyone named on its command line, with the greeting that the environment variable GREETING holds, or "Hello" if it is not set:

   ```
   var greeting os.environment_variable("GREETING")
   if greeting.length = 0
       greeting := "Hello"
   for name in os.arguments()
       print "`greeting`, `name`!"
   ```

   `os.arguments()` answers the arguments the program was started with, its own name left out. `os.environment_variable` reads a variable of the environment and is empty if the variable is not set.

2. Right-click `greet.scaly`, in the Solution Explorer or in the editor, and choose Scaly Launch Settings…. In the dialog, type **`Ada Grace`** under Arguments and **`GREETING=Hi`** under Environment, and click OK. Working directory can stay empty: the program then starts in the folder of its file.

3. Set a breakpoint on the `print` line and press **F5**. The program stops there, and Locals shows `greeting` as `"Hi"` and `name` as `"Ada"`.

4. Press **F5**: the next round stops with `"Grace"`. Press **F5** once more to run to the end; the console window shows `Hi, Ada!` and `Hi, Grace!`.

The settings belong to the file and apply to **Ctrl**+**F5** too. Visual Studio keeps them in the folder's hidden `.vs` folder.

2.3.2.7 Tests in the Test Explorer

1. Add `sums.scaly` to the folder. A test is a function whose name is `test` or begins with

`test_`, that takes nothing and answers an `int`: 0 when it passes, anything else when it fails. The third test fails on purpose:

```
function add(a: int, b: int) returns int
    a + b

function test_add() returns int
    if add(1, 2) = 3: 0 else 1

function test_zero() returns int
    if add(0, 0) = 0: 0 else 1

function test_wrong() returns int
    if add(1, 1) = 3: 0 else 1
```

2. Choose Test > Test Explorer. It lists the three tests under `sums.scaly`.
3. Click Run All Tests, the double triangle in the Test Explorer's toolbar. `test_add` and `test_zero` pass and turn green; `test_wrong` fails and turns red. Click it: the pane below says `(answered 1)`.
4. Double-click `test_wrong` to go to it in the editor. Change the `3` to `2`, save, and run the test again: it passes.

A test that stops the program, for instance with an index out of bounds, fails with the message the program gave. The tests of its file that would have run after it are shown as not run.

2.3.2.8 What Visual Studio Shows

- The Locals and Autos windows show a function's arguments and its bindings: numbers and booleans; structures, unions and pointers with their fields to expand; a `String` as its text; a `Vector`, `Array` or `List` with its elements. A generic type is written as in C++: `Vector<int>`.
- The Watch and Immediate windows evaluate C++, not Scaly.
- Visual Studio does not show code lenses or the call hierarchy for Scaly, though the language server offers both; VS Code shows them.
- On a PC with an Arm processor, the Target setting under Tools > Options > Scaly builds for x64 instead: **`x86_64-pc-windows-msvc`**. Windows runs such a program, and Visual Studio debugs it, under emulation.

2.4 A Tour of Scaly

This section sweeps quickly across the language using small, complete programs. If something is not fully explained here, do not worry — every feature is covered in detail in the rest of this

specification, and each part of the tour points you to the relevant chapter. Each program prints what it computes; `print` writes a line:

```
print "Hello from Scaly"
```

The same tour runs in the browser, step by step: `scaly.io/play/tour.html`. There every program of this section stands in an editor with a Run button — change it, run it again, try what happens when you break it; Reset brings the original back. Nothing to install: the compiler runs in the browser tab.

2.4.1 Values and Bindings

Bind a value to a name with `let`. There is no equals sign — the name is followed directly by its value. Use `var` for a binding you intend to change, and `:=` to change it. A semicolon starts a comment, and backticks inside a string put a value into the text.

```
let answer 42        ; immutable binding
var count 0          ; mutable binding
count := count + 1   ; assignment is :=
print "answer is `answer`, count is `count`"
```

The equals sign = is reserved for comparison, so it never gets confused with assignment. See Chapter 10 for the full lexical rules.

2.4.2 Operators and Operations

Operators are ordinary identifiers, not keywords baked into the grammar. An operation is read from left to right; when operators are chained, they combine by the binding strength the standard library gives them — multiplication binds tighter than addition, comparison weaker than both:

```
; 3 + 4 * 5 is 3 + 20, which is 23 — not 35
let result 3 + 4 * 5
if result = 23
    print "got 23"
else
    print "unexpected"
```

Parentheses group explicitly, and an operator written where a value is expected acts as a prefix (`-x`, `not done`, `a - -2`). A call takes its arguments before the operators around it combine, and a call without parentheses takes exactly one argument: `print "hi"` is `print("hi")`, but an argument that is itself a call takes its parentheses, `print(describe(x))`. See Chapter 14 for the complete model, tuples and functions as values included.

2.4.3 Control Flow

`if` chooses between branches. Single-statement branches need no braces; the condition needs no parentheses:

```
let n 7
if n < 0
    print "negative"
else if n = 0
    print "zero"
else
    print "positive"
```

`while` repeats as long as its condition holds. Use braces for a multi-statement body:

```
var sum 0
var i 1
while i <= 10
{
    sum := sum + i
    i := i + 1
}
print "the sum is `sum`"
```

`for` iterates. `for i in 4` runs `i` from 0 up to — but not including — 4:

```
var total 0
for i in 4
    total := total + i
print "0 + 1 + 2 + 3 is `total`"
```

2.4.4 Functions

A function declares its parameter types and return type. The value of the last expression is returned automatically — no `return` keyword is needed for the final result. A single-expression body needs no braces:

```
function add(a: int, b: int) returns int
    a + b

print "3 + 4 is `add(3, 4)`"
```

Functions can call themselves. Here is the classic factorial:

```
function factorial(n: int) returns int
    if n <= 1
        1
    else
        n * factorial(n - 1)

print "5! is `factorial(5)`"
```

Functions are pure: they read their arguments and return a value, with no side effects. To mutate, use a procedure.

2.4.5 Procedures and Mutation

A `procedure` is like a function but may change its inputs and perform side effects. It says which inputs it changes: a parameter it writes is marked `mutable`, the receiver `this` included, and a field that may change after the object is built is declared `var`. A reader always knows which calls can mutate state, and the compiler checks it. See Section 11.8.2 for the details.

```
define Counter(var value: int)
{
    procedure increment(mutable this)
        value := value + 1

    function get(this) returns int
        value
}

var c Counter(0)
c.increment()
c.increment()
print "the counter is at `c.get()`"
```

2.4.6 Structures and Methods

Define a structure by listing its fields. A structure with public fields gets a constructor that takes them in order. Methods are functions or procedures whose first parameter is `this`:

```
define Point(x: int, y: int)
{
    function distance_squared(this) returns int
        this.x * this.x + this.y * this.y
}

let p Point(3, 4)
print "the distance squared is `p.distance_squared()`"
```

Section 11.9 covers constructors, including explicit `init` forms with their own signatures.

2.4.7 Unions, Options, and choose

A `union` holds one of several variants. Each variant may carry a payload. The most common union is `Option`, which represents a value that may be absent:

```
define Option union(Some: int, None)

let present Option.Some(42)
let absent  Option.None
```

Match on a union with `choose`. Each `when` clause names a binding, then the variant, then the result; the binding is bound to the variant's payload. An `else` clause handles the remaining variants:

```
define Option union(Some: int, None)

function or_zero(o: Option) returns int
    choose o
        when v: Some
            v
        else
            0

print "`or_zero(Option.Some(7))` and `or_zero(Option.None)`"
```

See Section 11.12 for union layout, the `T?` shorthand, and pattern matching in full.

2.4.8 Modules and Imports

Split a program across files with `module`: the statement `module shapes` loads the sibling file `shapes.scaly`. A type defined there stays hidden until the using file imports it with `use`:

```
; shapes.scaly
define Box(width: int, height: int)
{
    function area(this) returns int
        width * height
}

; myprog.scaly
module shapes
use myprog.shapes.Box        ; or: use myprog.shapes.*
```

```
let b Box(3, 4)
if b.area() = 12
    print "area is 12"
```

Forgetting the import is caught at compile time, and the error states the exact `use` line to add. The standard library needs no imports — it is always visible.

Code that several programs share is a *package*, and a package somebody else wrote is one line away:

```
package json 0.1.3 "github.com/someone/json"
```

scaly fetches it from that git repository the first time the program is run, tested or built; and **`scaly install`** builds the programs such a package offers and puts them on your PATH. See Chapter 16 for packages and their sources, visibility rules, and glob imports.

2.5 Putting It Together

Let us combine the pieces into one small, complete program: a safe division that returns an `Option` instead of crashing on a zero divisor, with the caller deciding what to print.

Start with the function. It returns `None` when the divisor is zero, and `Some` with the quotient otherwise:

```
define Option union(Some: int, None)

function safe_divide(a: int, b: int) returns Option
    if b = 0
        Option.None
    else
        Option.Some(a / b)
```

Now call it and react to the result with `choose`. The `Some` arm receives the quotient; the `else` arm handles division by zero:

```
let result safe_divide(20, 4)
choose result
    when q: Some
        if q = 5
            print "20 / 4 = 5"
        else
            print "unexpected quotient"
    else
```

```
        print "cannot divide by zero"
```

Put both pieces in `divide.scaly`, then build and run it:

```
scaly build divide.scaly -o divide
./divide
```

It prints:

```
20 / 4 = 5
```

Change the call to `safe_divide(7, 0)` and the same program prints `cannot divide by zero` — the union makes the failure a value you must handle, not a crash you can forget.

From here, the rest of this specification fills in the details: the Chapter 10 rules, the full Chapter 14 model, and the Chapter 11 system including generics and the inferred region-based memory management.

II Scaling

3 Why Scaly Can Scale

3.1 Two Rounds at Once

Two rounds of a loop may run at the same time when neither can see what the other does: neither writes anything the other reads, and neither does something whose order the world outside the program can tell — printing a line, writing a file, answering a request. A compiler that can prove this for a loop can hand its rounds to every core of the machine, and the program cannot tell the difference except by the clock.

Most languages give the compiler no way to prove it. Any call may write through any pointer it was handed, change a global, or print; so parallel code is written by hand, with threads, locks and a library, and getting it right is the programmer's business. Scaly gives the compiler three facts, and checks every one of them in every compile: what a routine may write is part of its declaration, a `function` writes nothing and does no input or output, and memory belongs to regions rather than to one heap that every core shares. A plain `for` loop is all a program writes; whether its rounds run in parallel is the compiler's decision.

This part of the book shows what that gives a program, a chapter for each way to scale: loops over every core, recursion that forks, the lanes of one core, many connections at once, and more than one machine. Each chapter is built around a program you can run. This chapter is about the three facts.

3.2 Functions Read, Procedures Write

A `function` reads its arguments and returns a value. That is not a convention but a rule the compiler enforces: a function writes nothing its parameters reach, reads no global that can change, and performs no input or output — not even through the routines it calls. A function that prints is refused:

```
function shout(word: String) returns String
{
    print word          ; error: function shout performs I/O
    word
}
```

So a call of a function is a question with an answer and nothing else, and two such calls can run side by side. Here the rounds of the first loop each compute one square and put it into a slot of their own; the second loop adds them up:

```
function square(x: int) returns int
    x * x
```

```
let n 1000
var squares Vector[int](n as size_t)
for i in n
    squares.put(i as size_t, square(i))

var total 0
for s in squares
    total := total + s
print "sum of squares below `n`: `total`"
```

```
$ scaly run squares.scaly
sum of squares below 1000: 332833500
$ scalyc --task-plan squares.scaly
task-plan: 2 for-loops, 2 parallelizable
task-plan: squares.scaly:6:1: parallel
task-plan: squares.scaly:10:1: parallel
$ SCALY_WORKERS=1 scaly run squares.scaly
sum of squares below 1000: 332833500
```

`--task-plan` shows the compiler's decision for each `for` of the file: both loops run in parallel. The first writes to `squares`, which the loop did not make — but each round only to the slot its own `i` names, so no two rounds write the same place. The second is a sum, which the compiler knows how to split and put together again; Chapter 4 says which loops it takes and which it leaves alone. The last command runs the program on one core; Section 3.5 says why it prints the same.

3.3 A Procedure Says What It Writes

A `procedure` may change things, and its declaration says which: each parameter it writes is marked `mutable`, the receiver `this` included, and each global it writes is named after its signature. The compiler checks the marks against the body, so they cannot lie (Section 11.8.2). A loop that hands its own state to a procedure that writes it keeps its order — here every round adds to the one `log`:

```
procedure record(mutable log: Array[int], value: int)
    log.add(value)

var log Array[int]()
for i in 5
    record(log, i * i)
print "`log.length` entries, the last one `log[4]`"
```

```
$ scaly run record.scaly
5 entries, the last one 16
```

```
$ scalyc --task-plan record.scaly
task-plan: 1 for-loops, 0 parallelizable
task-plan: record.scaly:5:1: blocked: transitively impure call record
```

A loop that prints keeps its order for the same reason: `print` is a procedure that reaches the world outside the program, and the lines come out as the rounds were written.

```
for i in 3
    print "round `i`"
```

```
round 0
round 1
round 2
```

None of this is an error. A loop the compiler cannot prove independent runs one round after the other, as written, and the program means the same either way; `--task-plan` is the only place where the difference shows.

3.4 Memory That Is Not Shared

The third fact is about memory. Scaly has no heap that every part of a program allocates from and a collector cleans up; memory belongs to regions — pages that belong to a call, or to an owner that lives longer — and a region is released as a whole. The compiler works out which region each value belongs to, and an escape checker refuses a program in which a value would outlive its region (Section 11.14; Section D.5 tells where the idea comes from).

For a loop on many cores that means there is nothing to take turns at. A worker allocates from pages of its own, so the rounds do not queue at one allocator, and there is no collector that must stop every core to look at all of memory. What a round builds and drops is gone with its page; what a round builds and the loop keeps (Section 4.5) is handed to the region of the code around the loop.

3.5 The Same Program on One Core

Because a loop runs in parallel only where its rounds cannot see each other, a program prints the same whether it runs on one core or on all of them. The environment variable SCALY_WORKERS caps how many cores a program uses; with **`SCALY_WORKERS=1`** it runs every loop on one, as the last command of `squares.scaly` did — which is also how you measure what the other cores bring.

Every program of this part is checked that way: what it prints with every core and with one is the output printed beside it.

4 Loops on All Cores

4.1 A Plain for

Scaly has no parallel loop. It has `for`, and the compiler decides for each `for` of a program whether its rounds may run at the same time. This chapter is built around one program: how many points of a grid lie in the Mandelbrot set. Each row of the grid is counted by a function, and a `for` over the rows puts each count into a slot of its own:

```
function row_count(y: int, n: int) returns int
{
    let ci (2.0 * (y as double) / (n as double)) - 1.0
    var count 0
    var x 0
    while x < n
    {
        let cr (2.0 * (x as double) / (n as double)) - 1.5
        var zr: double 0.0
        var zi: double 0.0
        var i 0
        var inside true
        while i < 50 and inside
        {
            let tr zr * zr - zi * zi + cr
            zi := 2.0 * zr * zi + ci
            zr := tr
            if zr * zr + zi * zi > 4.0
                inside := false
            i := i + 1
        }
        if inside
            count := count + 1
        x := x + 1
    }
    count
}

let n 2000
var counts Vector[int](n as size_t)
for y in n
    counts.put(y as size_t, row_count(y, n))

var total 0
for c in counts
    total := total + c
```

```
print "mandelbrot `n`: `total` points inside"
```

```
$ scaly run mandelbrot.scaly
mandelbrot 2000: 1588053 points inside
$ scalyc --task-plan mandelbrot.scaly
task-plan: 2 for-loops, 2 parallelizable
task-plan: mandelbrot.scaly:31:1: parallel
task-plan: mandelbrot.scaly:35:1: parallel
$ SCALY_WORKERS=1 scaly run mandelbrot.scaly
mandelbrot 2000: 1588053 points inside
```

Nothing in the program says that the rows may be counted side by side; the compiler sees it. `row_count` is a `function`, so it reads its arguments and nothing else, and the round for row `y` writes only slot `y` of `counts`. The second loop adds the counts up, which the compiler knows how to split as well. The `while` loops inside `row_count` run as written: only a `for` is ever handed to the cores.

Built with **`scaly build --release`** and `n` at 8000, the program took 4.1 seconds with **`SCALY_WORKERS=1`** and 0.54 seconds without, on a laptop with eight performance cores and two efficiency cores: 7.5 times as fast, from the same source.

4.2 What a Round May Do

A round runs in parallel with the others when nothing it does can be seen by another round. The compiler checks each `for` for that, and it accepts a body whose rounds do only these things:

- read anything that does not change while the loop runs: the loop variable, bindings made before the loop, the elements of a vector;
- call functions, which read and return and do nothing else (Section 3.2), and procedures that write only what the round made itself;
- make values of their own — bindings inside the body, strings, records — which are gone when the round ends unless the round keeps them;
- write a slot of a vector made before the loop with `put`, at an index of which the compiler can see that no two rounds compute the same one: the loop's counter, the counter plus or minus a value that stays the same, or a row of a table, `y * width + x` with `x` counted below `width` inside the round;
- add to a sum, a product or a maximum that the loop accumulates (Section 4.4).

The first loop of `mandelbrot.scaly` does the first, second and fourth of them, the second loop the fifth.

4.3 What Keeps a Loop in Order

Everything else keeps a loop sequential: its rounds run one after the other, in the order they are written, exactly as they would in a language that never runs anything in parallel. Three loops that each look harmless:

```
function cube(x: int) returns int
    x * x * x

var cubes Array[int]()
for i in 6
    cubes.add(cube(i))

var last 0
for i in 6
    last := cube(i)

var first 0
for i in 6
{
    if cube(i) > 50
    {
        first := i
        break
    }
}
print "`cubes.length` cubes, the last `last`, the first above 50: `first`"

$ scaly run order.scaly
6 cubes, the last 125, the first above 50: 4
$ scalyc --task-plan order.scaly
task-plan: 3 for-loops, 0 parallelizable
task-plan: order.scaly:5:1: blocked: transitively impure call add
task-plan: order.scaly:9:1: blocked: writes loop-external last
task-plan: order.scaly:13:1: blocked: writes loop-external first
```

The first loop appends to an array it did not make: `add` is a procedure that writes `cubes`, and in which order the rounds append decides the order of the elements. The second leaves in `last` what the last round wrote there — run side by side, whichever round finished last would win. The third stops at the first round that finds what it looks for, and which round is the first is again a question of order. Each verdict names the first reason the compiler found; a loop may have more than one.

The other reasons are of the same kind. A loop keeps its order when a round reads a global that can change (`mutable` or `shared`), writes one, reaches the world outside the program, writes a

slot of a vector at an index two rounds could share (`i / 2`, an element of another vector), reads the vector it writes at any other slot, calls a routine the compiler cannot see into (a delegate, a C function it does not know), or leaves the loop early with `break`, `return` or a throw. A `while` or a `repeat` is never split at all; a loop over a range is a `for`.

A loop that keeps its order is not an error and gets no warning: it was written to run in order, and it does. When a loop should run on all cores, `--task-plan` names what stands in the way, and the fix is usually to give each round its own slot: the cubes into a `Vector` of six with `put`, as the counts of the rows above.

4.4 Sums, Products and the Largest

A loop that folds its rounds into one variable writes something it did not make, and still runs in parallel when the fold is one the compiler knows: `v := v + e` and the same with `-` and `*`, the bitwise `|`, `&` and `xor` on integers, and `if e > v` followed by `v := e` for a maximum (`<` for a minimum) of integers. Each part of the range folds its own rounds, and the parts are folded together at the end. The rectangles of perimeter 40, their areas summed and the largest one found:

```
function area(width: int) returns int
    width * (20 - width)

var sum 0
var largest 0
for w in 20
{
    let a area(w)
    sum := sum + a
    if a > largest
        largest := a
}
print "the areas add up to `sum`, the largest is `largest`"

$ scaly run areas.scaly
the areas add up to 1330, the largest is 100
$ scalyc --task-plan areas.scaly
task-plan: 1 for-loops, 1 parallelizable
task-plan: areas.scaly:6:1: parallel
```

The variable must be a local of the routine the loop is in, and the body may use it only in its update: a round that also reads the running sum — to store it somewhere, say — sees a value that depends on which rounds ran before it, and the loop stays in order (`reduction variable used outside its update`). In a maximum, the value compared and the value taken must be the same expression, written alike on both sides — a name such as `a`, an element `v[i]`, a field, a call of a function; a round that compares one value and takes another computes no

maximum, and its loop stays in order.

Floating-point addition is not associative: added in another order, the same numbers give a slightly different sum. A parallel sum of floating-point numbers that depended on how the range was split would print different digits on another machine, or on the same one with another load. Scaly's does not: each round's contribution is kept, and they are added up in the order of the rounds, so the sum has every bit of the one a sequential loop computes.

```
function term(k: int) returns double
{
    let t 4.0 / ((2 * k + 1) as double)
    if k % 2 = 0: t else 0.0 - t
}

let n 1000000
var pi: double 0.0
for k in n
    pi := pi + term(k)
print "`n` terms of Leibniz's series: pi is about `pi`"
```

```
$ scaly run leibniz.scaly
1000000 terms of Leibniz's series: pi is about 3.1415916535897743
$ SCALY_WORKERS=1 scaly run leibniz.scaly
1000000 terms of Leibniz's series: pi is about 3.1415916535897743
```

4.5 Values a Round Keeps

A round may build a value and keep it: put a string or a record it made into its slot of a vector that outlives the loop.

```
function caption(i: int) returns String
    "row `i`: `i * i`"

let n 5
var captions Vector[String](n as size_t)
for i in n
    captions.put(i as size_t, caption(i))
print "`captions[0]` ... `captions[4]`"
```

```
$ scaly run captions.scaly
row 0: 0 ... row 4: 16
$ scalyc --task-plan captions.scaly
task-plan: 1 for-loops, 1 parallelizable
task-plan: captions.scaly:6:1: parallel
```

Run in order, each caption would be built on the page of the code around the loop and live as long as it. Run in parallel, the rounds of one part of the range build on a page of their own; when the loop ends, those pages are handed to the region of the code around the loop, and the captions live exactly as long as they would have. Nothing is copied, and the parts never share an allocator (Section 3.4). What a round keeps must be a value it built — a reference to something else is refused, and so is a body that takes an address with `&`, which could point into the round's own stack frame.

4.6 When the Cores Are Worth It

That a loop *may* run in parallel does not mean that it should. Waking other cores costs time, and a loop of a few dozen cheap rounds is done before they would be awake. Whether the cores are worth it depends on how many rounds there are and how much each one does, and the compiler knows neither in general. So it does not decide; the running program does.

A loop the compiler accepted starts in order, on the core that reaches it. Now and then — at checkpoints that lie twice as far apart each time, the first one placed by what the compiler estimated for the body — it reads the clock and works out what a round costs. As soon as the rounds still to run promise more work than splitting costs, about a tenth of a millisecond of it, they are cut into parts and handed to a pool of workers, one per core. A short loop ends before its first checkpoint and pays nothing for having been eligible. A loop that has run often remembers what its rounds cost, and the next time it decides at its start, without measuring again.

That is why the squares of the previous chapter and the cubes above print instantly either way, and why the rows of the Mandelbrot set run on every core. The pool has as many workers as the machine has cores, or as many as SCALY_WORKERS says; with one, every loop runs in order, and the program prints the same.

5 Recursion That Forks

5.1 Two Calls at Once

A recursion that splits its problem in two and calls itself on each half has the same independence a loop has: the two calls read their arguments and nothing the other one writes. When the function is declared a `function`, the compiler knows it, and the two calls may run on two cores. The classic example does nothing but call and add:

```
function fib(n: int) returns int
{
    if n < 2
        return n
    fib(n - 1) + fib(n - 2)
}
```

```
print "fib(35) is `fib(35)`"

$ scaly run fib.scaly
fib(35) is 9227465
$ scalyc --task-plan fib.scaly
task-plan: 0 for-loops, 0 parallelizable
fork-plan: 1 fork pairs
fork-plan: fib.scaly:5:5: fib + fib
$ SCALY_WORKERS=1 scaly run fib.scaly
fib(35) is 9227465
```

Again there is no annotation. `--task-plan`, which reports the loops of a file, reports its recursion as well: below the loops, one line for each pair of calls that may run on two cores. Built with **`scaly build --release`** and asked for `fib(42)`, the program took 0.83 seconds on one core and 0.14 seconds on the ten of the laptop of the previous chapter. That is six times as fast for a function that is the worst case for forking, because each call does almost nothing besides calling again.

5.2 What Forks

The compiler looks for two shapes in a function that calls itself: the two calls on both sides of one operator, as in `fib`, and two bindings in a row, each a call, the second one not using the first:

```
function count_multiples(lo: int, hi: int) returns int
{
    if hi - lo < 4096
    {
        var count 0
        var i lo
        while i < hi
        {
            if i % 7 = 0 or i % 11 = 0
                count := count + 1
            i := i + 1
        }
        return count
    }
    let mid (lo + hi) / 2
    let left count_multiples(lo, mid)
    let right count_multiples(mid, hi)
    left + right
}

let n 200000000
```

```
print "below `n`, `count_multiples(0, n)` numbers are multiples of 7 or 11"
```

```
$ scaly run multiples.scaly
below 200000000, 44155845 numbers are multiples of 7 or 11
$ scalyc --task-plan multiples.scaly
task-plan: 0 for-loops, 0 parallelizable
fork-plan: 1 fork pairs
fork-plan: multiples.scaly:16:5: let left count_multiples, let right count_multiples
$ SCALY_WORKERS=1 scaly run multiples.scaly
below 200000000, 44155845 numbers are multiples of 7 or 11
```

With **`scaly run`** on that laptop, one core took 0.33 seconds and all of them 0.09. The rules are narrow, and each of them has a reason:

- The function is a `function`, so that the two calls cannot see each other.
- It calls *itself*, twice, in one expression or in two `let` bindings that follow each other directly. Two different functions side by side do not fork, nor does a function that reaches itself through another one; of three calls, `f(a) + f(b) + f(c)`, only the first two form a pair.
- The operator between the calls is not `and` or `or`: their right side runs only when the left side does not decide, so running it early would run it where the program does not.
- Its parameters are numbers or pointers, its result a number. A function that takes or returns a string or a record runs as written: the value a call on another core builds would have to be handed over, which loops do (Section 4.5) and recursion does not yet.
- It is a free function, not a method, and it cannot throw.

A function outside these rules runs as written, and nothing is wrong with it; `--task-plan` simply lists no pair for it:

```
function both(n: int) returns bool
{
    if n < 2
        return n = 1
    both(n - 1) and both(n - 2)
}

function three(n: int) returns int
{
    if n < 2
        return n
    three(n - 1) + three(n - 2) + three(n - 3)
}

function even(n: int) returns int
```

```
{
    if n < 2
        return n
    odd(n - 1) + odd(n - 2)
}

function odd(n: int) returns int
{
    if n < 2
        return n
    even(n - 1) + 1
}

function name(n: int) returns String
{
    if n < 1
        return String("x")
    let a name(n - 1)
    let b name(n - 2)
    a
}

print "`both(10)` `three(10)` `even(10)` `name(3)`"

$ scaly run others.scaly
false 68 31 x
$ scalyc --task-plan others.scaly
task-plan: 0 for-loops, 0 parallelizable
fork-plan: 1 fork pairs
fork-plan: others.scaly:12:5: three + three
```

`both` joins its calls with `and`, `even` and `odd` call each other, and `name` returns a string; of the three calls in `three`, the first two form a pair.

5.3 Forks Nobody Waits For

A recursion has many more pairs of calls than a machine has cores: `fib(35)` makes some thirty million calls. Handing each pair to the pool would cost far more than the additions it spares, and deciding at each pair whether it is worth it costs too much as well — the frames entered in the first microseconds would decide before any clock could tell. Scaly does neither.

At each pair near the top of the recursion, the running call notes its first call as one another core *could* take — a few writes into a list of its own thread, no lock —, runs the second call itself, and then takes the first back and runs it, unless someone took it meanwhile. While such a recursion runs, a clock beats every tenth of a millisecond. On a beat, if a worker of the pool has nothing to

do, the oldest noted call of a thread is offered, and the idle worker takes it: the oldest is the one nearest the top of the recursion, the biggest piece of work still waiting. A call whose first half was taken does not wait idly for it; it helps the core that took it.

So a fork happens only where a core is free to take it, and the noting costs a few writes per call. Below the twelfth level the function runs as a plain copy without even those. What is left of the cost shows where the calls do no work of their own: when the engine was built, 200 000 separate runs of `fib(15)` took about three times as long as the plain function. A recursion whose calls compute something pays nothing measurable.

Three things keep a recursion on one core: a pool of one worker (**`SCALY_WORKERS=1`**), a call made inside a `try` — a fault on another core could not reach its handler —, and a build with `-g`, whose debugger expects one body per function. The last one the compiler knows when it compiles, so **`scalyc --task-plan -g`** lists no pair; the first two are decided when the program runs.

5.4 An Uneven Tree

A loop splits its range into parts of equal size; a recursion splits where the problem does, and the halves may differ by orders of magnitude. Adaptive integration is such a problem: it halves an interval until two estimates of the area agree, so it goes deep where the function bends and stops early where it is flat. Here the function has forty narrow peaks on a slow ramp:

```
; forty narrow peaks on a slow ramp
function g(x: double) returns double
{
    var y x * 0.1
    var k 0
    while k < 40
    {
        let d x - ((k as double) * 0.0731 + 0.05)
        y := y + 1.0 / (d * d + 0.00001 * ((k % 7 + 1) as double))
        k := k + 1
    }
    y
}

function simpson(a: double, b: double) returns double
    (b - a) / 6.0 * (g(a) + 4.0 * g((a + b) * 0.5) + g(b))

function area(a: double, b: double, whole: double, eps: double, depth: int) returns d
{
    let m (a + b) * 0.5
    let l simpson(a, m)
    let r simpson(m, b)
```

```
    let diff l + r - whole
    if depth = 0 or (diff < 15.0 * eps and diff > 0.0 - 15.0 * eps)
        return l + r + diff / 15.0
    area(a, m, l, eps * 0.5, depth - 1) + area(m, b, r, eps * 0.5, depth - 1)
}

let total area(0.0, 3.0, simpson(0.0, 3.0), 0.0000000001, 60)
print "the area under g from 0 to 3: `total`"
```

```
$ scaly run area.scaly
the area under g from 0 to 3: 23048.968067792208
$ scalyc --task-plan area.scaly
task-plan: 0 for-loops, 0 parallelizable
fork-plan: 1 fork pairs
fork-plan: area.scaly:26:5: area + area
$ SCALY_WORKERS=1 scaly run area.scaly
the area under g from 0 to 3: 23048.968067792208
```

Nobody has to tell the engine where the work is. A call that turns out to be big is still noted while it runs, so it is offered on the next beat a core is free, however deep the recursion below it goes. With **`scaly run`**, one core took 0.35 seconds and all of them 0.09.

The result has every digit of the sequential one. The two halves of an interval may be computed on two cores, but their sum is formed where it is written, left half plus right half, so the floating-point additions happen in the same order either way.

6 Lanes

6.1 Four Numbers in One

A core can do more than one thing per instruction. Its vector registers hold several numbers side by side — two, four or more doubles, sixteen or thirty-two bytes —, and one instruction adds or multiplies all of them at once, each in its own *lane*. The cores of the previous chapters multiply the work by the number of cores; lanes multiply it once more, on each of them.

A compiler can sometimes find lanes in a loop by itself, but not in a loop that stops early or that takes a different path for each number. So Scaly lets a program say it: a vector type holds a fixed number of values as one value, and its operators work lane by lane.

```
let a f64x4(1.0, 2.0, 3.0, 4.0)
let b f64x4.splat(0.5)
let c f64x4.iota(10.0, 0.25)
let d a * b + c
print "`d[0]` `d[1]` `d[2]` `d[3]`"
```

```
let e a.with(2, 9.0) * 2.0
print "`e[0]` `e[1]` `e[2]` `e[3]`"

10.5 11.25 12 12.75
2 4 18 8
```

`f64x4(…)` gives each lane its value, `splat` the same value to every lane, and `iota(base, step)` counts up from `base`. `a * b + c` is four multiplications and four additions, done as one of each. A scalar on one side of an operator stands for itself in every lane, as the `2.0` does. `d[i]` reads a lane and stops the program with exit code 15 when there is no lane `i`; `a.with(i, x)` is a copy of `a` with one lane replaced. A vector is a value like a `double`: it is copied, passed and returned in registers, and it never lives on a page.

The types are named by their element and their number of lanes: `f64x2`, `f64x4`, `f64x8`; `f32x4`, `f32x8`, `f32x16`; `i64x2`, `i64x4`, `i32x4`, `i32x8`; `u8x16`, `u8x32`, `u32x4`, `u32x8`. A vector wider than the hardware is fine: on a core whose registers hold two doubles, an `f64x4` is two registers, and its operations are two instructions each.

6.2 Masks

A comparison of two vectors compares lane by lane, and its answer is a *mask* — `mask2` to `mask32`, one flag per lane. A mask answers questions about its lanes, combines with `and`, `or` and `not` lane by lane, and chooses between two vectors with `select`:

```
let a f64x4(1.0, 2.0, 3.0, 4.0)
let big a > 2.5
print "`big.count()` lanes above 2.5, any: `big.any()`, all: `big.all()`"
let clipped select(big, f64x4.splat(2.5), a)
print "`clipped[0]` `clipped[1]` `clipped[2]` `clipped[3]`"
print "sum `a.sum()`, largest `a.max_lane()`"

2 lanes above 2.5, any: true, all: false
1 2 2.5 2.5
sum 10, largest 4
```

Masks are how a vector program takes a different path per lane: it computes both paths for all lanes and keeps, with `select`, what each lane should have. And `any` and `all` are how it leaves a loop: when no lane has anything left to do. `sum`, `product`, `min_lane` and `max_lane` fold the lanes into one number; a sum of floating-point lanes adds them in order, lane 0 first, so it gives the digits of a plain loop over them.

6.3 Lanes from Memory

`f64x4.load(xs, i)` reads the elements `xs[i]` to `xs[i + 3]` of a `Slice`, `Vector` or `Array` as one vector, and `v.store(ys, i)` writes them. Each checks once that all four elements exist — a scalar loop checks every one. A dot product four elements at a time, with a fused multiply-add per step and the rest done one by one:

```
function dot(a: Vector[double], b: Vector[double]) returns double
{
    var acc f64x4.splat(0.0)
    var i: size_t 0
    while i + 4 <= a.length
    {
        acc := f64x4.load(a, i).fma(f64x4.load(b, i), acc)
        i := i + 4
    }
    var sum acc.sum()
    while i < a.length
    {
        sum := sum + a[i] * b[i]
        i := i + 1
    }
    sum
}

let n 1001
var a Vector[double](n as size_t)
var b Vector[double](n as size_t)
for i in n
{
    a.put(i as size_t, (i as double) * 0.5)
    b.put(i as size_t, 2.0)
}
print "a · b = `dot(a, b)`"
```

```
a · b = 500500
```

`v.fma(w, acc)` is `v * w + acc` with one rounding instead of two; `sqrt`, `abs`, `floor`, `min` and `max` work lane by lane as well. The four lanes of `acc` are four running sums, added up at the end. That is a different order of additions than a loop over one element at a time, and for numbers that are not whole it can change the last digits; the order is written in the program, so the result is the same on every run and every machine.

6.4 Mandelbrot, Eight Points at a Time

The program of Section 4.1 counts one point after the other, and each point leaves its loop when it escapes. With lanes, a row is counted eight points at a time, as two `f64x4` — two, because while one multiplication is still in flight the core can already start the other. The eight points go through their iterations together; every eight iterations the program asks whether any lane is still inside, and stops when none is. A point that escaped keeps iterating with the others; after fifty iterations its value is so large that it has become infinite or not a number, and the comparison with 4.0 sorts it out at the end:

```
function count8(x: int, y: int, n: int) returns int
{
    let step 2.0 / (n as double)
    let ci f64x4.splat(step * (y as double) - 1.0)
    let cr0 f64x4.iota(step * (x as double) - 1.5, step)
    let cr1 cr0 + 4.0 * step
    var zr0 cr0
    var zi0 ci
    var zr1 cr1
    var zi1 ci
    var s0 f64x4.splat(0.0)
    var s1 s0
    var block 0
    while block < 6
    {
        var k 0
        while k < 8
        {
            let r0 zr0 * zr0
            let i0 zi0 * zi0
            let ri0 zr0 * zi0
            s0 := r0 + i0
            zr0 := r0 - i0 + cr0
            zi0 := ri0 + ri0 + ci
            let r1 zr1 * zr1
            let i1 zi1 * zi1
            let ri1 zr1 * zi1
            s1 := r1 + i1
            zr1 := r1 - i1 + cr1
            zi1 := ri1 + ri1 + ci
            k := k + 1
        }
        if ((s0 <= 4.0) or (s1 <= 4.0)).any() = false
            return 0
        block := block + 1
    }
```

```
    var k 0
    while k < 2
    {
        let r0 zr0 * zr0
        let i0 zi0 * zi0
        let ri0 zr0 * zi0
        s0 := r0 + i0
        zr0 := r0 - i0 + cr0
        zi0 := ri0 + ri0 + ci
        let r1 zr1 * zr1
        let i1 zi1 * zi1
        let ri1 zr1 * zi1
        s1 := r1 + i1
        zr1 := r1 - i1 + cr1
        zi1 := ri1 + ri1 + ci
        k := k + 1
    }
    (s0 <= 4.0).count() + (s1 <= 4.0).count()
}

function row_count(y: int, n: int) returns int
{
    var count 0
    var x 0
    while x < n
    {
        count := count + count8(x, y, n)
        x := x + 8
    }
    count
}

let n 2000
var counts Vector[int](n as size_t)
for y in n
    counts.put(y as size_t, row_count(y, n))

var total 0
for c in counts
    total := total + c
print "mandelbrot `n`: `total` points inside"

$ scaly run mandelbrot.scaly
mandelbrot 2000: 1588053 points inside
$ scalyc --task-plan mandelbrot.scaly
task-plan: 2 for-loops, 2 parallelizable
```

```
task-plan: mandelbrot.scaly:71:1: parallel
task-plan: mandelbrot.scaly:75:1: parallel
$ SCALY_WORKERS=1 scaly run mandelbrot.scaly
mandelbrot 2000: 1588053 points inside
```

The count is the one of the scalar program. The row loop is the same plain `for` as before, and it still runs on every core: a vector is a value like a number, and `count8` is a function like `row_count` was. Built with **`scaly build --release`** and `n` at 8000, the scalar program took 4.1 seconds on one core and 0.54 on all ten; this one takes 0.76 on one core and 0.11 on all ten — the lanes bring five times, the cores another seven, together thirty-seven.

6.5 Lanes and Cores

Lanes and cores combine because they divide the work at different places: the lanes inside one round, the cores across the rounds. What the compiler checks for a `for` does not change. One thing a round may not do with lanes is store a vector into memory outside the round: the compiler proves that two rounds write different *elements* only for `put` at the loop's index, not for the four elements a store writes.

```
let n 1000
var xs Vector[double](n as size_t)
for i in n
    xs.put(i as size_t, i as double)

var ys Vector[double](n as size_t)
for k in n / 4
{
    let v f64x4.load(xs, (4 * k) as size_t)
    (v * 2.0).store(ys, (4 * k) as size_t)
}
print "ys[999] = `ys[999]`"

$ scaly run double.scaly
ys[999] = 1998
$ scalyc --task-plan double.scaly
task-plan: 2 for-loops, 1 parallelizable
task-plan: double.scaly:3:1: parallel
task-plan: double.scaly:7:1: blocked: SIMD store
```

Such a loop runs in order, on one core, with its lanes. A loop whose rounds load and compute and hand back numbers — a sum, a count, a slot of their own — runs on all cores with its lanes, as the rows of the Mandelbrot set do.

Which instructions the lanes become depends on the processor the program is compiled for, and

without being told, the compiler compiles for a generic one of the target's family: on x86-64 that means no AVX, even where the machine has it. **`scaly build --release -mcpu=native`** compiles for the machine at hand (or `-mcpu=` with a processor's name, for another one). On the laptop of these measurements it changed nothing: its vector registers are as wide as the generic ones.

7 Many Connections

7.1 Waiting Is Not Working

The programs of the previous chapters keep every core busy with arithmetic. A server is busy differently: most of the time its connections wait — for the next request of a client, for a database, for a file. Ten thousand open connections may need the processor for a few hundred of them at any moment. What scales a server is how little a waiting connection costs.

A thread per connection costs a stack of megabytes and a place in the operating system's scheduler, and ten thousand of them are too many. The usual way out is to write the server as callbacks or with `async` and `await`, and every function that may wait has to be written that way too: the waiting becomes part of each function's type, and the program splits into two kinds of code.

Scaly does neither. A connection is a *task*: a small stack of its own (128 KB, of which only what is touched costs memory) and a region of its own, run by a *scheduler* that switches between tasks in user space. When a task reads from a socket that has nothing to read, it is parked, and the scheduler runs another; when data arrives, the task continues where it stopped. The read is an ordinary call — the same call that, in a program without a scheduler, simply blocks.

7.2 A Server

The `http` package of the standard library is a server built that way. A handler takes the request and the response, a router maps paths to handlers, and `serve` runs it:

```
package http 0.2.0

use http.Request.Request
use http.Response.Response
use http.Router.Router

procedure hello(req: ref[Request], mutable res: ref[Response])
{
    let name req.param("name")
    res.send_text("hello, `name`")
}
```

```
var app Router.create(Page.allocate_page())
app.get("/hello/{name}", &hello)
serve(app, 8080)
```

Run, it answers **`curl localhost:8080/hello/book`** with `hello, book`, and goes on serving until it is stopped. The program the book checks is one that stops by itself: a server that answers each request after a pause, and a hundred clients in the same program that ask at once. The handler sleeps with `Scheduler.sleep`, which parks its task, not a thread; each client is a task too, on a scheduler of the program's own, and counts how many clients are waiting for an answer at the same moment:

```
package http 0.2.0

use http.Request.Request
use http.Response.Response
use http.Router.Router

; waits ms milliseconds without holding a thread, then answers ms
procedure delay(req: ref[Request], mutable res: ref[Response])
{
    let ms parse_int_prefix(req.param("ms"))
    Scheduler.sleep(ms)
    res.send_int(ms)
}

mutable port: int 0
mutable waiting: int 0
mutable most: int 0
mutable answered: int 0

; one client: a request, then the answer up to the end of the connection
procedure client() mutable waiting, most, answered
{
    var s TcpStream.connect("127.0.0.1", port)
    s.write("GET /delay/100 HTTP/1.1\r\nHost: here\r\nConnection: close\r\n\r\n")
    waiting := waiting + 1
    if waiting > most
        most := waiting
    var buf u8[256]
    let view Slice[u8](256, &buf[0])
    var got: size_t 0
    repeat
    {
        let n s.read(view.slice_from(got))
        if n <= 0
```

```
            break
        got := got + (n as size_t)
    }
    waiting := waiting - 1
    s.close()
    if got > 12 and buf[9] = 50 and buf[10] = 48 and buf[11] = 48
        answered := answered + 1
}

var app Router.create(Page.allocate_page())
app.get("/delay/{ms}", &delay)
var l TcpListener.listen(0)
port := l.port()
let server start(app, l)

let clients Scheduler.create(Page.allocate_page())
var k 0
while k < 100
{
    Scheduler.spawn(&client)
    k := k + 1
}
Scheduler.run()

server.shutdown()
server.wait()
print "`answered` of 100 answered, at most `most` waiting at once"

$ scaly run waits.scaly
100 of 100 answered, at most 100 waiting at once
$ SCALY_WORKERS=1 scaly run waits.scaly
100 of 100 answered, at most 100 waiting at once
```

`TcpListener.listen(0)` takes a free port on the local machine, and `start` serves it on threads of its own and returns at once, where `serve` would not return; `shutdown` and `wait` end it. The router and the clients' scheduler each get a page to live on. `&delay` and `&client` hand the procedures themselves to the router and the scheduler.

All hundred clients waited at the same time, with all cores and with one: a hundred requests that take a tenth of a second each were answered, built with **`scaly build --release`**, after 0.11 seconds. A thousand clients took 0.15 seconds: the server held a thousand parked tasks, not a thousand threads.

7.3 The Same Calls Everywhere

Look at `client` again: it connects, writes and reads with the calls a program without any scheduler would use, and nothing in its declaration says that it may wait. Whether a call blocks or parks is decided when it runs, not written in the program:

- outside a scheduler, `read` blocks the thread, as in any program;
- inside a task, it parks the task and lets the scheduler run the others until data arrives;
- a call the operating system can only do blocking — `connect`, which resolves a name and shakes hands, and the reads of a file — goes to a pool of helper threads, and the task parks until it is done.

So there is one kind of function. A procedure that reads a configuration file is called from a handler, from a test and from `main` alike, and code written before anyone thought of a server runs inside one unchanged. One limit is named: a socket is read and written by one task at a time, not by a reader and a writer in parallel.

7.4 One Scheduler per Core

A scheduler runs on one thread, and one thread uses one core. `serve` and `start` therefore run one scheduler per core — as many as SCALY_WORKERS allows — and the connections are spread over them. On Linux each scheduler has a listener of its own on the same port, and the kernel hands each new connection to one of them; on macOS and Windows one scheduler accepts and passes the connections round. A connection stays on the scheduler that took it, so its task never moves between cores, and its memory stays with one core's caches.

Each scheduler waits for its sockets with what the operating system offers for many at once — `io_uring` or `epoll` on Linux, `kqueue` on macOS, I/O completion ports on Windows — and none of that shows in the program. The checked program above ran its server on ten schedulers and on one, and printed the same.

7.5 A Region per Connection

Every task allocates from its own region (Section 3.4). For a server that settles what else would be its hardest question: a request builds its strings, its parsed headers and its answer in the region of the handler's call, and all of it is released at once when the handler returns — no collector walks the heap, no other connection notices. A connection that ends gives back its stack and its region, and the records of a finished task are reused for the next connection, so a server does not grow with the number of connections it has served.

Neither is shared between the schedulers, so they never take turns at an allocator or a lock for memory. A connection costs what it uses, its stack, and nothing per request that outlives the request.

8 The Graphics Card

8.1 Thousands of Small Cores

A graphics card is a machine of its own beside the processor: thousands of small cores that run one routine for every element of a large range at once, with memory of their own or, on many laptops, the processor's memory shared. What it runs well is exactly what the compiler already recognises in a `for` (Chapter 4): rounds that read what they are given, compute, and put one result into a slot of their own.

So a program does not say "run this on the graphics card". It writes the loop. Here is the program of Section 4.1 once more, with one change: it computes in `f32` and `i32`, the numbers every card has, where the original computed in `double` and `int`.

```
function row_count(y: i32, n: i32) returns i32
{
    let ci (2.0 * (y as f32) / (n as f32)) - 1.0
    var count: i32 0
    var x: i32 0
    while x < n
    {
        let cr (2.0 * (x as f32) / (n as f32)) - 1.5
        var zr: f32 0.0
        var zi: f32 0.0
        var i: i32 0
        var inside true
        while i < 50 and inside
        {
            let tr zr * zr - zi * zi + cr
            zi := 2.0 * zr * zi + ci
            zr := tr
            if zr * zr + zi * zi > 4.0
                inside := false
            i := i + 1
        }
        if inside
            count := count + 1
        x := x + 1
    }
    count
}

let n: i32 2000
var counts Vector[i32](n as size_t)
for y in n
```

```
    counts.put(y as size_t, row_count(y, n))

var total 0
for c in counts
    total := total + c
print "mandelbrot `n`: `total` points inside"
```

```
$ scaly run mandelbrot.scaly
mandelbrot 2000: 1588044 points inside
$ scalyc --gpu-plan mandelbrot.scaly
gpu-plan: 2 for-loops, 2 parallel, 1 kernels
gpu-plan: mandelbrot.scaly:31:1: kernel (metal): out counts; in n; calls row_count; n
gpu-plan: mandelbrot.scaly:35:1: no kernel: iterates a collection
$ scaly build --gpu mandelbrot.scaly -o mandelbrot
$ ./mandelbrot
mandelbrot 2000: 1588044 points inside
$ SCALY_GPU=on ./mandelbrot
mandelbrot 2000: 1588044 points inside
$ SCALY_GPU=off ./mandelbrot
mandelbrot 2000: 1588044 points inside
```

`--gpu-plan` says for every parallel `for` whether a card could run it: the loop over the rows is a *kernel* — a routine a card runs once per round —, with the vector it writes, the values it reads from around it, and the functions it calls. A program built with `--gpu` carries each of its kernels along, and decides while it runs, loop by loop, whether the card or the processor's cores compute it (Section 8.4). SCALY_GPU takes the decision away from it: **`on`** sends every kernel to the card, **`off`** none.

All three runs print the same count, and that is a promise, not luck: a loop goes to the card only if the program cannot tell where it ran (Section 8.3). The count is not the 1588053 of the chapter on loops — an `f32` rounds earlier than a `double`, and on the edge of the set that moves nine points to the other side —, but it is the count of *this* program wherever it runs.

Built with **`scaly build --release --gpu`** and `n` at 8000, the program took 4.0 seconds on one core, 0.52 on all ten, and 0.06 where the card counted the rows — 0.02 of them for the rows themselves.

The card is reached through Metal, on a Mac, and — for the loops that allow it — through WebGPU (Section 8.5). On a system without either, `--gpu` builds the program without its kernels, and it runs on the processor's cores as it always did.

8.2 What a Card Can Run

A card has no pages, no strings and no stack to speak of. It has numbers, arrays of numbers, and one routine. A parallel `for` is a kernel when its rounds get by with that:

- they compute with `bool`, `i32`, `u32` and `f32` — and with 64-bit integers on a card that has them (Section 8.5);
- they read numbers bound outside the loop, and elements of a `Vector`, `Slice` or `Array` of such numbers, at any index;
- they write with `put`, at exactly the loop's counter, into a `Vector` of such numbers, add to a sum, a product or a largest of such numbers (Section 4.4), and write nothing else;
- the functions they call take and answer such numbers, do not call themselves, and are made of the same stuff;
- nothing in them builds a record or a string, and nothing leaves the loop early.

Inside a round almost everything else is allowed: bindings, `if`, `while` and `for`, a `break` out of an inner loop. A loop that is no kernel is not an error. It runs on the cores, and `--gpu-plan` names what kept it from the card:

```
function scaled(v: i32, k: i32) returns i32
    v * k + 1

function caption(i: int) returns String
    "row `i`"

let n 1000
let k: i32 3
var src Vector[i32](n as size_t)
for i in n
    src.put(i as size_t, (i as i32) % 17)

var out Vector[i32](n as size_t)
for i in n
    out.put(i as size_t, scaled(src[i as size_t], k))

var wide Vector[int](n as size_t)
for i in n
    wide.put(i as size_t, i * i)

var halves Vector[double](n as size_t)
for i in n
    halves.put(i as size_t, (i as double) * 0.5)

var names Vector[String](n as size_t)
for i in n
    names.put(i as size_t, caption(i))

var sum: i32 0
```

```
for i in n
    sum := sum + out[i as size_t]
print "`out[999]` `wide[999]` `halves[999]` `names[999]` `sum`"

$ scaly run kernels.scaly
40 998001 499.5 row 999 24937
$ scalyc --gpu-plan kernels.scaly
gpu-plan: 6 for-loops, 6 parallel, 4 kernels
gpu-plan: kernels.scaly:10:1: kernel (metal, webgpu): out src
gpu-plan: kernels.scaly:14:1: kernel (metal, webgpu): out out; in k, src; calls scale
gpu-plan: kernels.scaly:18:1: kernel (metal): out wide; not webgpu: the counter i, us
gpu-plan: kernels.scaly:22:1: no kernel: halves is a Vector[double]
gpu-plan: kernels.scaly:26:1: no kernel: names is a Vector[String]
gpu-plan: kernels.scaly:30:1: kernel (metal, webgpu): reduces sum; in out
```

All six loops run on all cores; four of them could run on a card. The first two compute in 32 bits — the counter `i` is an `int`, but it is cast or used as an index on the spot, and the number of rounds fits into 32 bits. The third squares the counter itself, in 64 bits. The fourth wants `double`, which no card of today's backends has; the fifth builds a string in every round. The last is a sum, and a kernel all the same: the card computes what each round adds, and the adding is done where the order of it can be kept (Section 8.3).

8.3 The Same Numbers

A loop that prints one result on the processor and another on the card would make every run a question of where it happened to run. So the rule is the one that already holds between one core and ten: the program cannot tell. For whole numbers every card keeps it by itself. Floating point takes care on both sides, and the compiler takes it: it writes the kernel one operation at a time, in the order the processor computes them, and tells the card to round every one of them as the processor does — no multiplication fused with the addition after it, no sum reordered, no faster and coarser division or square root. What comes back is equal to the processor's result bit for bit, not merely close to it.

The other half of the promise is what happens when a round goes wrong. On the processor an index outside its vector stops the program with exit code 15, and a division by zero stops it too. A card cannot stop a program. Its round raises a flag instead and computes nothing; when the card is done and the flag is up, all of its results are thrown away, and the loop runs on the processor — which stops in the round it always stopped in, with the exit code it always had. Nothing a card computed is ever kept from a loop the processor would not have finished.

A sum is where a card would most naturally break the promise. Thousands of small cores add up thousands of numbers as a tree, in whatever order their work falls, and a floating-point sum depends on its order: the last digits of a tree are not those of a loop. So a card never adds up a loop's sum. It computes, for each round, what that round contributes, and hands back one number

per round; the processor then folds those numbers into the variable one by one, in the rounds' order — which is what it does with the rounds it runs itself (Section 4.4). The folding is a pass over memory and costs next to nothing beside the rounds. A product and a largest go the same way, and so do the sums of whole numbers, which no order could change.

The card works on copies. It gets the vectors a loop reads as they are when the loop begins, and the vectors it writes come back whole when it is done. For a loop the compiler calls parallel that is no different from what its rounds see on the cores: no round reads what another one writes.

8.4 Where a Loop Runs

A card is fast and far away. Handing it a loop means copying the loop's vectors over, waking the card, waiting for it, and copying the results back: a few tenths of a millisecond on the laptop of these measurements, whether the loop has eight rounds or eight million. And the first time a loop goes there its kernel has to be translated for the card, which takes some hundredths of a second. For the 8000 rows above that is nothing. For a small loop it is everything.

So the program decides, and it decides the way it already decides between one core and all of them (Section 4.6): it starts every loop on one core, measures the first rounds, and only a loop with enough work left is handed on. At that moment it knows what a round costs. It compares what the cores would need for the rounds that are left with what the card needs for the whole loop — as measured the last time this loop ran there, or, the first time, the price of a translation and one call — and the card gets the loop if it wins by a clear margin. A loop that ran on the card is asked at its very first round the next time. And a loop that is too small to be worth a translation, but comes back again and again, is tried on the card once the work it has brought adds up to one: from then on it is measured like any other, and goes where it is faster.

SCALY_GPU_REPORT makes a program say at its end where each loop went. The program of this chapter with its 2000 rows, and then with 8000:

```
$ SCALY_GPU_REPORT=1 ./mandelbrot
gpu: mandelbrot.scaly:31: 0 on the card, 1 on the processor
mandelbrot 2000: 1588044 points inside
$ SCALY_GPU_REPORT=1 ./mandelbrot8000
gpu: mandelbrot.scaly:31: 1 on the card, 0 on the processor; the card 21.983 ms for 8
mandelbrot 8000: 25402925 points inside
```

Two thousand rows are three hundredths of a second on ten cores: less than a translation costs, so they stay where they are. Eight thousand rows are half a second, and the card takes them. The line of a loop that went to the card says what it spent there — copying in, running, copying out — and what the translation of its kernel took; here the system had kept it from an earlier run. The times in such a line differ from run to run and from machine to machine, which is why they stand on standard error and not in what the program prints. A loop that never had enough work to be handed on is not in the report at all: nobody asked about it.

How much there is to lose shows a loop of eight rounds that a program runs three thousand times. Left alone, it never leaves its core. Forced to the card with **`SCALY_GPU=on`** it took 0.57 seconds for what the core does in a blink — three thousand journeys of two tenths of a millisecond each. **`on`** is for finding out what the card does with a loop, **`off`** for finding out what it costs to do without; a program that is to be fast sets neither.

8.5 The Numbers a Card Computes

Which loops a card can take depends on which numbers it has, and cards differ. Two rules cover them:

- *A card computes in 32 bits.* No card reached today has 64-bit floating-point numbers, and the cards a browser reaches (WebGPU) have no 64-bit integers either. A loop that computes with `double` stays on the processor, where its cores and lanes still scale it; one that computes with `int`, which is 64 bits, goes only to cards that have them. A `double` is never quietly computed as an `f32`: the digits would change with the place. A program that wants the card everywhere computes in `f32` and `i32`; the loop's own counter does not count, as long as the range it counts fits into 32 bits.
- *Floating point is exact or relaxed.* Unless the program says otherwise, a loop with floating-point numbers may only go where every operation is rounded exactly as on the processor. Metal can be told to; WebGPU cannot — its specification lets a card reorder a sum, fuse a multiplication with an addition, and be off by a few last bits in a division. A loop written `relaxed` may go to such a card as well: its results stay within the tolerances the card's specification promises, which is what most graphics and machine learning want.

```
function half(x: f32) returns f32
    x * 0.5

let n 1000
var exact Vector[f32](n as size_t)
var loose Vector[f32](n as size_t)
for i in n
    exact.put(i as size_t, half(i as f32) / 3.0)
relaxed for i in n
    loose.put(i as size_t, half(i as f32) / 3.0)
print "`exact[999] as f64` `loose[999] as f64`"

$ scaly run relaxed.scaly
166.5 166.5
$ scalyc --gpu-plan relaxed.scaly
gpu-plan: 2 for-loops, 2 parallel, 2 kernels
gpu-plan: relaxed.scaly:7:1: kernel (metal): out exact; calls half; not webgpu: float
gpu-plan: relaxed.scaly:9:9: kernel (metal, webgpu): out loose; calls half
```

`relaxed` stands before a loop or before a block, and holds for everything the loop's rounds compute, including the functions they call. It allows, it does not require: on the processor a relaxed loop computes exactly as before, and so it does under Metal, which rounds as the processor does whether it has to or not. Where a card of the looser kind computed it, a point on the edge of the Mandelbrot set may come out the other way — one whose iteration ends within a rounding error of 4.0. That is what the last words of the report at the head of this chapter say: the rows are a kernel for Metal as they stand, and would be one for WebGPU if the loop were relaxed. Counted there, the 2000 rows came to 1588048 points, four more than everywhere else.

A program built with `--gpu` carries a kernel in both languages where both may run it, Metal's and WebGPU's, and uses Metal unless told otherwise. **`SCALY_GPU_BACKEND=webgpu`** sends the loops that have a kernel for WebGPU there instead — through wgpu-native, the library that also sits under browsers, which the program opens when it first needs it and does without where it is not installed — and leaves the others on the processor. Whole numbers come back from that card exactly as from any other; a call takes two to three times as long as one through Metal. `--gpu-msl` and `--gpu-wgsl` print the kernels as the compiler writes them.

A browser is the other place with such a card. Built with **`scaly build --target wasm32-wasip1 --gpu`**, a program carries its kernels for WebGPU alone and asks the page it runs in for the card; a page that has none to give — or the command line — answers no, and every loop stays on the processor. The standard library brings the page's side along (`web/gpu-host.mjs` beside the WASI host): the program runs in a worker, the card belongs to a second one, and the loop waits for its rounds exactly as it does natively. The decision is the same too, taken where the processor has measured a round. A loop of whole numbers that keeps the one core a page's program has busy for 0.44 seconds came back from the card in 3 milliseconds, with the same numbers.

9 More Than One Machine

9.1 Where Memory Ends

Everything so far scaled inside one machine: the cores of the earlier chapters share its memory, and the compiler hands them work on its own. Past the machine's edge memory is no longer shared, a message takes a fraction of a millisecond instead of a few nanoseconds, and the other side may stop answering. Scaly does not hide that edge. Within a machine a `for` decides by itself where its rounds run; across machines the program says what goes where, and what it sends is a message.

A message is a region (Section 3.4): a value built on pages of its own, with everything it points to. A send ships those pages as they are, and the receiving side fixes every pointer in them to where the pages landed — with code the compiler generates for each message type. There is no serialization code to write and no schema to keep in step with the program.

9.2 Nodes and Channels

The program of this chapter counts the primes below three million on three worker nodes. One program plays both roles: started without an argument it coordinates, with one it works. The coordinator listens on a port, hands each worker that connects a share of the range, and prints what comes back:

```
define Share
(
    from: int
    to: int
)

define Count
(
    primes: int
    range: String
)

function is_prime(n: int) returns bool
{
    if n < 2
        return false
    var d 2
    while d * d <= n
    {
        if n % d = 0
            return false
        d := d + 1
    }
    true
}

function count_primes(from: int, to: int) returns int
{
    var count 0
    for k in to - from
        if is_prime(from + k)
            count := count + 1
    count
}

procedure work()
{
    let peer Cluster.connect("localhost", 7700, 5000)
    if peer = null
```

```
        return
    var shares Remote[Share](peer, 1)
    let counts Remote[Count](peer, 2)
    let share shares.receive()
    if share <> null
    {
        let n count_primes(share.from, share.to)
        counts.send(Count(n, "`share.from` to `share.to`"))
    }
    Cluster.disconnect(peer)
}

procedure coordinate(workers: int, limit: int)
{
    let listener Cluster.listen(7700)
    var peers Array[ref[NodePeer]]()
    var w 0
    while w < workers
    {
        let peer Cluster.accept(listener)
        if peer <> null
        {
            let shares Remote[Share](peer, 1)
            shares.send(Share(limit * w / workers, limit * (w + 1) / workers))
            peers.add(peer)
            w := w + 1
        }
    }
    var total 0
    for peer in peers
    {
        var counts Remote[Count](peer, 2)
        let c counts.receive()
        if c <> null
        {
            print "`c.primes` primes from `c.range`"
            total := total + c.primes
        }
        Cluster.disconnect(peer)
    }
    print "`total` primes below `limit`"
}

if os.arguments().length > 0
    work()
else
    coordinate(3, 3000000)
```

```
$ scaly build primes.scaly -o primes
$ ./primes work &
$ ./primes work &
$ ./primes work &
$ ./primes
78498 primes from 0 to 1000000
70435 primes from 1000000 to 2000000
67883 primes from 2000000 to 3000000
216816 primes below 3000000
```

Here the four nodes are four processes on one machine; on four machines the workers name the coordinator's host instead of `localhost`, and nothing else changes. `Cluster.connect` keeps trying for as long as its last argument says, in milliseconds, because a worker may start before the coordinator listens; `Cluster.accept` waits for the next node to connect. Either answers the connection to the other node, or `null`.

A `Remote[T]` is a channel to that node for messages of one type, named by a number both sides agree on: here 1 for the shares and 2 for the counts. `send` hands a message over and returns at once; `receive` waits for the next one and answers it, or `null` when the other node has said goodbye or is gone. `Cluster.disconnect` says goodbye. The coordinator first sends every share and only then collects, so the three workers count at the same time — and each of them counts its share with the plain `for` of Chapter 4, on all the cores of its machine.

9.3 A Message Is a Region

`counts.send(Count(n, "…"))` builds the `Count` — the record and the string in it — on a page of its own, and the send ships that page and releases it. What is sent is gone from the sender; nothing on the other side can point back into it. The receiving node's pages hold the same bytes at other addresses, the generated code fixes the pointers, and `receive` hands the message to the routine that asked: it belongs to that routine's region from then on and goes when that region goes, like any value the routine built itself.

Because the send builds the message, its argument is the construction, or a call that makes the message. A value built before would have its string on another page, which the send would not ship, so a name is refused:

```
let c Count(n, "`share.from` to `share.to`")
counts.send(c)      ; error: send builds the message on a page of its own
```

A message may hold records, unions, strings and the containers of the standard library — `Vector`, `Array`, `List`, `HashMap` — nested as deep as the program likes. What only makes sense in one process cannot be sent, and the compiler says so: a delegate, an open file, a fixed-size array.

9.4 One Program, One Build

Pages shipped as they are only make sense to a program that lays out its records the same way. So the nodes of a cluster run one build of one program — which is why the transcript builds once and starts the result four times. Two nodes check it before anything else: each sends a stamp of its build when it connects, and a node whose stamp differs is refused, with both stamps in the error. A new version of the program is a new cluster.

9.5 When a Node Fails

A node that crashes, loses its network or stops answering looks the same from the other side as one that said goodbye: its channels close, and a `receive` waiting on one of them answers `null`. That is why the coordinator asks `if c <> null`: with a worker killed in the middle of its share it prints the counts of the others and finishes. A silent node is noticed by its silence — the nodes exchange a heartbeat every half second when nothing else flows, and one silent for two and a half seconds is taken for gone. That is a suspicion, not a proof; a node that was only slow finds its connection closed.

A send never waits for an answer and is never retried: a message arrives at most once, the messages of one channel in the order they were sent, and a message sent to a node that is gone is dropped. Whether a lost share is sent to another worker is the program's decision, not the runtime's. The tests of the cluster runtime include such a program: a small transformer trained on several nodes, each computing the gradients of its part of a batch, one node summing them in a fixed order and sending the sum back. Its loss is the same to the last bit as on one node, and with one worker killed mid-run the others finish the training.

One limit is named: the nodes trust each other. A connection is neither authenticated nor encrypted, and a node checks the framing of what it receives, not the meaning — run a cluster only inside a network you trust.

III Language Reference

10 Lexical Structure

10.1 Overview

This chapter describes the lexical structure of Scaly: how source text is divided into tokens. The lexer transforms a sequence of characters into a sequence of tokens that the parser then processes.

10.2 Whitespace and Line Breaks

Whitespace characters (space, tab) separate tokens but are otherwise insignificant. Line breaks, however, are syntactically meaningful in Scaly — they can terminate statements and declarations.

A colon (`:`) can be used in place of a line break when multiple statements should appear on a single line.

10.3 Comments

Scaly supports two forms of comments:

- *Single-line comments* start with a semicolon (`;`) and extend to the end of the line.
- *Multi-line comments* start with `;*` and end with `*;`.

```
; This is a single-line comment

;* This is a
   multi-line comment *;
```

10.4 Literals

10.4.1 Integer Literals

Integer literals represent whole numbers. They can be written in decimal or hexadecimal notation.

```
42        ; decimal
0x2A      ; hexadecimal (same value)
```

10.4.2 Floating-Point Literals

Floating-point literals represent decimal numbers with a fractional part or an exponent.

```
3.14159       ; pi
2.5e10        ; scientific notation
1.0e-5        ; small number
```

10.4.3 String Literals

String literals are enclosed in double quotes. The supported escape sequences are `\"`, `\\`, `\'`, `\n`, `\r`, `\t`, `\0`, and `\uXXXX` (four hex digits, encoded as UTF-8).

```
"Hello, World!"
"Line 1\nLine 2"      ; with newline escape
"Tab:\tValue"         ; with tab
"Quote: \"nested\""   ; escaped quotes
"\u0041"            ; Unicode escape - the letter A
```

Strings can span multiple lines without any special syntax — newlines are simply included as part of the string content:

```
"This is a string
that spans multiple
lines."
```

A backtick inside a string embeds an expression: the string is built from its parts, and each embedded value is written in the text form of its type — a `String`, a `char`, the bytes of a `Slice[u8]`, an integer, a `bool`, or a floating-point number as the shortest text that reads back to it. A value of another type needs a conversion inside the backticks, and `\`` writes a backtick itself.

```
let n 42
let x 0.1
"The answer is `n`, not `n + 1`, and x is `x`"
                      ; The answer is 42, not 43, and x is 0.1
```

Inside the backticks a string is written with its quotes escaped, since an unescaped one would end the string around it: `"a `twice(\"ab\")` b"`.

A parameter that takes the bytes of a text is a `Slice[u8]`, a view that owns nothing. A string is handed to it as its view: a literal is that view from the start, and a `String` — a variable, a field, a string built with backticks — is passed as `s.as_slice()` would pass it, for as long as the call lasts. A binding or a field of type `Slice[u8]` takes the conversion written out.

```
function vowels(text: Slice[u8]) returns int
    ...
let name String("Ada")
vowels(name)                    ; the view of name
vowels("hello, `name`")         ; the view of the string built here
let v: Slice[u8] name.as_slice()
```

10.4.4 Character Literals

A character is written as a string literal of one character wherever a `char` is expected — a `char` parameter, a typed binding, or the cast `as char`. Escapes count as one character.

```
let a: char "a"        ; lowercase a
let nl "\n" as char    ; newline character
```

10.4.5 Boolean Literals

Boolean literals are `true` and `false`.

```
true
false
```

10.5 Operators and Special Symbols

10.5.1 The Equals Sign

Unlike many programming languages, Scaly reserves the equals sign (=) exclusively for equality comparison. It is *not* used for variable declarations or assignments.

```
; Variable declaration (NO equals sign)
var x 42                  ; declares x with value 42
let y: i32 100            ; with type annotation

; Assignment (its own sign)
x := 43                   ; assigns 43 to x

; Equality comparison
if x = 43                 ; tests if x equals 43
    print "x is 43"
```

This design eliminates the classic confusion between assignment and comparison that leads to bugs in C-style languages (`if (x = 42)` vs `if (x == 42)`).

NOTE 1 Because = is just an operator like any other, the standard library could define a unary = operator that returns its argument unchanged. This would allow the syntax `let x = 42` to parse as `let x (= 42)`, for those who prefer traditional declaration style.

10.6 Identifiers

Identifiers name variables, functions, types, and other entities. Scaly supports three forms of identifiers:

• *Alphanumeric identifiers* start with a letter and continue with letters, digits, or underscores.

• *Operator identifiers* consist of operator characters such as +, −, *, /.

• *String identifiers* are enclosed in single quotes and can contain any characters.

```
myVariable        ; alphanumeric
calculate_total   ; with underscore
++                ; operator identifier
'my identifier'   ; string identifier (with spaces)
```

The form of an identifier does not by itself decide what it names. A variable, a function, or an operator may carry any of the three forms; an operator-character name such as ++ can name an ordinary function, and a string identifier such as `'divided by'` can name an operator. What a name denotes is fixed by its declaration, not by how it is spelled (see the chapter on definitions).

11 Type System

11.1 Overview

Scaly currently recognizes a fixed set of primitive types — integers, floats, booleans, and pointer types — which the LLVM-based emitter maps to the corresponding LLVM types. A possible future direction, described in the following sections, is to make the type system fully backend-agnostic, with even the primitive types supplied by a backend-specific prelude rather than built into the compiler.

11.2 Intrinsic Types (Future Direction)

NOTE 2 The mechanism described in this section and the two that follow (*Type Aliases* and *Backend Preludes*) is a planned design, not current behavior. Today the compiler knows its primitive types directly and targets LLVM only.

An *intrinsic type* is a type whose implementation is provided by the backend, not by Scaly code. Under this design, intrinsic types would be declared using the `intrinsic` keyword:

```
define i32 intrinsic
define i64 intrinsic
define f64 intrinsic
```

The compiler would not interpret these definitions — it would simply record that these types exist and are intrinsic. The backend (Emitter) would be responsible for mapping intrinsic types to the target platform's representation.

This design would enable Scaly to target any backend:

• *LLVM*: `i32`, `i64`, `f32`, `f64`

• *JavaScript*: `number`, `string`, `boolean`

• *WebAssembly*: `i32`, `i64`, `f32`, `f64`

• *JVM*: `int`, `long`, `float`, `double`

11.3 Type Aliases (Future Direction)

Type aliases provide human-friendly names for intrinsic or compound types:

```
define bool i1
define char i32       ; Unicode scalar value
```

```
define int i64        ; Platform word size
define size_t u64
```

The alias and its target are interchangeable — `bool` and `i1` refer to the same type.

11.4 Backend Preludes (Future Direction)

Each backend provides a *prelude* file that defines the intrinsic types and their aliases for that target. The prelude is implicitly loaded before any user code.

Example 11-1. LLVM Prelude (excerpt)

```
; Intrinsic types (LLVM native)
define i1 intrinsic
define i8 intrinsic
define i16 intrinsic
define i32 intrinsic
define i64 intrinsic
define f32 intrinsic
define f64 intrinsic
define ptr intrinsic

; Human-friendly aliases
define bool i1
define char i32
define int i64
define size_t u64
```

Example 11-2. JavaScript Prelude (excerpt)

```
; Intrinsic types (JavaScript native)
define number intrinsic
define string intrinsic
define boolean intrinsic

; Aliases for compatibility
define int number
define float number
define bool boolean
```

11.5 Literal Type Inference

Numeric and other literals do not have an inherent type. Their type is inferred from context — the parameter a literal is passed to, the annotation on its binding, or the other operand of an operation:

```
function double(x: i32) returns i32 { x * 2 }

double(42)           ; 42 inferred as i32 from parameter type

let y: i64 100       ; 100 inferred as i64 from annotation
let z y + 50         ; 50 inferred as i64 to match y
```

When nothing constrains an integer literal, it defaults to `int` (the platform word size), so the common case needs no annotation:

```
let x 42             ; x is int
```

An integer constant — a literal, a `define` declared `int`, and an operation over such — keeps its exact value until it meets the type it is used as, and then it must fit that type. A constant that does not fit is a compile-time error, never a value cut to the width:

```
let a: u8 200 + 55   ; 255
let b: u8 300        ; error: the constant 300 does not fit u8
let c: u32 0 - 1     ; error: the constant -1 does not fit u32
let d: u8 ~0         ; 255: the complement in the target's width
```

Scaly does *not* support type suffixes on literals (such as `42i32`). Where a different width matters, annotate the binding (`let x: i32 42`) or cast (`42 as i32`).

11.6 Tuples and the Empty Tuple

A tuple is an anonymous record, written in parentheses: `(1, "one")`, or with labels, `(x: 1, y: 2)`. Its type is written the same way, `(int, String)`, and it may stand wherever a type may: as a parameter, a result, a field or a generic argument. A labelled component is read by its label, `t.x`; an unlabelled one is reached by decomposition, `let (a, b) t`. A routine returns several values as a tuple:

```
function divide(n: int, d: int) returns (int, int)
    (n / d, n % d)

let (q, r) divide(17, 5)     ; q is 3, r is 2
```

A parenthesized single value is that value, so `(a)` is `a`. The empty tuple `()` is the type of no value: a routine that answers nothing returns `()`, which the word `void` also names. It holds no bytes, so `Result[(), Error]` or `Vector[()]` cost nothing beyond their other parts. A call that answers `()` where a type is declared is a compile-time error.

11.7 References

By default, values flow through functions by value: a function parameter gets read-only access to its argument, and a function returns a value.

To refer to an existing value without copying it, Scaly provides the reference type `ref[T]` — a non-owning, borrowed reference to a `T`. References are used for parameters and for nullable fields, written `ref[T]?`. Like `pointer[T]`, a reference is represented as a machine pointer, but it carries borrow semantics rather than the unchecked, low-level meaning of a raw pointer.

11.7.1 Low-Level Pointers

For low-level implementation of data structures like `Page`, `HashMap`, or `List`, the type `pointer[T]` is available. This is an escape hatch for implementors, not for everyday code.

```
; Low-level list node (internal implementation)
define Node[T]
(
    data: T
    next: pointer[Node[T]]    ; raw pointer for linked structure
)
```

11.7.2 Nullable Values

For nullable values, use `Option[T]` from the standard library — a proper sum type with `Some(value)` and `None` variants, matched with `choose`:

```
let o Option[int].Some(7)
choose o
```

```
    when v: Some
        print "got a value"
    else
        print "empty"
```

For many element types the compiler optimizes `Option[T]` to a simple nullable pointer — no space overhead for the tag (see Section 11.12.4).

11.8 Parameter Passing

All *function* parameters are *borrowed* — functions receive read-only access to their arguments. The caller retains ownership.

```
define Point ( x: int, y: int )

function distance_squared(a: Point, b: Point) returns int
{
    let dx a.x - b.x
    let dy a.y - b.y
    dx * dx + dy * dy
}

let origin Point(0, 0)
let target Point(3, 4)
distance_squared(origin, target)    ; origin and target unchanged
```

11.8.1 Implementation

The implementation of parameter passing depends on the execution context, but the semantics remain identical:

- *Same thread/stack*: A pointer is passed. No copying occurs. The function reads through the pointer.
- *Different thread/GPU/remote*: The entire data tree is copied to the target execution context. The function still has read-only access — same semantics, different mechanism.

This design means code doesn't change based on where it executes. A function that works locally works identically when distributed.

11.8.2 Mutation via Procedures

To modify data, use a `procedure` instead of a function, and mark each parameter it writes `mutable` — the receiver `this` included. A field that may change after its object is built is declared `var`; every other field is fixed once the object exists. The caller sees the change:

```
define Point(var x: int, var y: int)

procedure move(mutable p: Point, dx: int)
    p.x := p.x + dx

var pos Point(0, 0)
move(pos, 5)
print "pos.x is now `pos.x`"
```

The compiler checks the marks against the bodies: a procedure that writes a parameter it does not mark, a function that writes anything its parameters reach, and a store into a field that is not `var` are errors. So the distinction between functions (pure, read-only) and procedures (may mutate) is explicit in the code, and a reader knows from a signature which arguments a call may change.

11.9 Constructors

Constructors create instances of types. Scaly provides both implicit and explicit constructors.

11.9.1 Implicit Constructors

If all members of a type are public, an implicit constructor is generated that takes all fields as parameters in declaration order:

```
define Point ( x: int, y: int )

let p Point(10, 20)    ; implicit constructor
```

11.9.2 Explicit Constructors

Use `init` for explicit constructors when you need default values for some fields or different construction signatures. An explicit `init` coexists with the implicit all-fields constructor:

```
define Point
```

```
(
    x: int
    y: int
)
{
    init(value: int)        ; convenience constructor
    {
        this.x := value
        this.y := value
    }
}

let p1 Point(10, 20)    ; implicit constructor
let p2 Point(5)         ; convenience init - Point(5, 5)
```

11.9.3 The `this` Prefix

The `this.` prefix is optional when unambiguous, but recommended for clarity:

```
init(x: int, y: int) {
    this.x := x    ; clear: field x gets parameter x
    this.y := y
}
```

When parameter names shadow field names, `this.` is required to disambiguate. Avoid `x := x` — it's confusing even if technically resolvable.

11.9.4 Complete Initialization

Every constructor should initialize all fields. The compiler does not yet enforce this: a field an `init` forgets to set contains an undefined value, not an implicit default (no automatic 0, false, or null). Enforcement as a compile-time error is planned; until then, treat a missing assignment in an `init` as a bug.

11.9.5 Constructor Return

`init` implicitly returns `this`, enabling direct binding:

```
let p Point(10, 20)    ; init returns the new Point
```

11.9.6 Types That Allocate

Some types are value types (the struct itself can live on the stack) but allocate internal data — `String`, `Array`, and the other containers. Their constructors are ordinary `init` definitions; the region their internal data is allocated in is chosen by the compiler from how the constructed value is used (see Section 11.14). No special constructor syntax is involved.

11.10 Lambdas and Closures

A lambda is an anonymous function defined with backslash syntax — parameters, a colon, and a body:

```
\x: x * 2                    ; single parameter
\x y: x + y                  ; multiple parameters
```

A lambda bound with `let` is called like a named function. Its parameter types are inferred at each call site (the lambda is specialized per call, like a local generic function):

```
let inc \x: x + 1
inc(41)                      ; 42
```

11.10.1 Capturing Variables

Lambdas can capture variables from their enclosing scope. Captures are *by value*: the lambda receives a copy at the call, so later changes to the original are not seen by the closure, and an assignment to a captured name inside the lambda changes only the closure's copy — the original stays untouched:

```
let multiplier 10
let scale \x: x * multiplier    ; captures multiplier by value

scale(5)    ; returns 50
```

11.10.2 Function Types: Delegates

To pass a lambda as a value — to a parameter, into a field, or as a return value — the function type must be named. A `delegate` definition supplies that name:

```
define IntFn delegate(int) returns int

function apply(f: IntFn, x: int) returns int
    f(x)

let g: IntFn \x: x + 1
apply(g, 5)                  ; 6
```

A delegate-typed value is a closure: a record holding the captures plus the code to run. Calling through it is an indirect call.

A named function becomes such a value where a delegate type is declared, chosen among its overloads by the delegate's signature; so does a method bound to its receiver, written without its tuple:

```
function twice(n: int) returns int
    n * 2

apply(twice, 21)             ; 42
let c Counter(40)
apply(c.plus, 2)             ; c.plus is the method bound to c
```

Bound to a name without a delegate type, a function stays a compile-time value: `let g twice` costs nothing, and `g(21)` is a direct call.

11.10.3 Closure Lifetime

A closure bound to a local variable lives on the stack frame and dies with it. A closure *returned* from a function must outlive the frame, so the compiler allocates it on the caller's region automatically — factory functions work:

```
define IntFn delegate(int) returns int

function make_adder(n: int) returns IntFn
{
    return \x: x + n         ; allocated on the caller's region
}

let add5 make_adder(5)
let r add5(2)                ; 7
```

What is *not* allowed is smuggling a stack-allocated closure out of its frame — returning a local binding, storing it into an object that outlives the frame, or passing it to a callee that stores it. The compiler rejects these with a hard error:

```
function bad(n: int) returns IntFn
{
    let g: IntFn \x: x + n
    return g
    ; error: a closure bound to a local lives on the stack frame and
    ; cannot be returned - return the lambda directly (return \x: ...)
    ; so it is allocated on the caller's region
}
```

11.10.4 Implementation

A named-called lambda costs nothing at runtime: each call site is monomorphized into a direct call, with captures passed as leading arguments. A delegate-typed closure is a small record — a code pointer plus the captured values — and calling it is one indirect call. No garbage collector is involved either way.

11.11 Error Handling

Scaly uses explicit error handling. There are no exceptions or stack unwinding — errors are values returned from functions or procedures.

11.11.1 The `throws` Clause

Functions or procedures that can fail declare their error type with `throws`:

```
function parse(input: String) returns AST throws ParseError
{
    if invalid(input)
        throw ParseError.InvalidSyntax(String("expected expression"))
    ...
}
```

Under the hood, this is equivalent to returning `Result[AST, ParseError]`, but with

dedicated syntax for clarity.

11.11.2 Single Error Type

A function can only throw one error type. To represent multiple error kinds, use a union:

```
define FileError union
(
    NotFound: String
    PermissionDenied: String
    IoError: String
)

function read_file(path: String) returns String throws FileError
{
    ...
}
```

This ensures a clear error signature and enables the `try`/`when` pattern for handling specific variants.

11.11.3 The `try`/`when` Pattern

Handle errors with `try` and `when` clauses. On success, the binding holds the result and the `when` arms are skipped; on failure, the matching arm runs with the error value bound:

```
try let v read_thing(false)
    when e: NotFound
        print "not found"
    else
        print "other error"
```

If not all error variants are covered by `when` arms, an `else` clause is required.

11.11.4 Error Propagation

Use `else throw` to re-throw errors to the caller:

```
function process(x: int) returns int throws E
```

```
{
    try let v inner(x)
        else throw    ; re-throws E to the caller

    v * 2
}
```

When the error type matches exactly, the error propagates automatically. This simplifies deeply nested code like parsers and visitors.

11.11.5 Error Lifetime

Thrown values live in an exception region the caller provides. The compiler infers this from throw position; the explicit thrown-lifetime suffix `!` may also be written on the thrown construction:

```
throw ParseError.InvalidSyntax(msg)!   ; explicit thrown lifetime
```

11.12 Union Types

A union type (also called sum type or tagged union) can hold one of several variants. Each variant optionally carries a payload of a single type (use a struct for several fields):

```
define Rectangle ( width: f64, height: f64 )

define Shape union
(
    Circle: f64
    Box: Rectangle
    Empty
)

let s Shape.Circle(5.0)
```

11.12.1 Memory Layout

A union is stored as a tag plus the largest variant:

```
Shape = { tag: u8, data: [size of largest variant] }
```

Unions can contain other unions. Nested unions contribute their full size (tag + data) when computing the parent union's size.

11.12.2 Pattern Matching with `choose`

Use `choose-when` to match on variants. Each `when` clause names a binding, then the variant, then the result; the binding is bound to the variant's payload:

```
choose s
    when r: Circle: r * r * 3.14
    when b: Box: b.width * b.height
    else 0.0
```

If not all variants are covered, an `else` clause is required.

11.12.3 Option Type

`Option[T]` is the standard library's union for nullable values:

```
define Option[T] union
(
    Some: T
    None
)

let o Option[int].Some(7)
choose o
    when v: Some: process(v)
    else handle_not_found()
```

11.12.4 Option Optimization

For non-pointer types, `Option[T]` optimizes to a pointer:

• `None` = null pointer

• `Some(value)` = pointer to value

Note that if T is itself a pointer, the Option Optimization leads to a pointer to another pointer which could be null itself.

11.13 Generics

Scaly supports generic types with type parameters in square brackets:

```
define Pair[T] ( first: T, second: T )
{
    function sum(this) returns T
        first + second
}

let p Pair[int](20, 22)
p.sum()                      ; 42
```

The type arguments are written where a generic type is constructed: they are not inferred from the values, and `Pair(20, 22)` is an error that says so. A tuple type is an argument like any other: `Array[(n: int, s: String)]()`.

A record declared inside a generic record is generic in the outer record's parameters: `Tally` inside `Shell[T]` holds a `T`, and each instance of `Shell` has its own. It is named through an instance of the outer record, or with the arguments at the end, the outer record's first; both spellings are the same type, for a value and for a type alike:

```
define Shell[T] ( item: T )
{
    define Tally ( n: T )
    {
        function get(this) returns T
            n
    }
}

let t: Shell[int].Tally Shell[int].Tally(5)
let u: Shell.Tally[int] t
print "`t.get() + u.get()`"    ; 10
```

The standard library's containers are generic the same way: `Vector[T]`, `List[T]`, `HashMap[K, V]`, `Option[T]`, `Result[T, E]`.

11.13.1 Monomorphization

Generics are implemented via *monomorphization*: each concrete instantiation becomes a completely separate type at compile time. `Pair[int]` and `Pair[String]` share no code at runtime — each has its own specialized implementation.

Benefits:

• No runtime overhead — no type descriptors or vtables

• Full optimization — the compiler sees concrete types

• No boxing — primitives stay primitives

Trade-offs:

• Larger binaries — each instantiation duplicates code

• Longer compile times — more code to generate

11.13.2 Name Mangling

The Planner generates unique mangled names for each instantiation, following Itanium ABI conventions:

```
List[int]            → _ZN4ListIiE...
List[String]         → _ZN4ListI6StringE...
HashMap[String, int] → _ZN7HashMapI6StringiE...
```

These names are compatible with `c++filt` for debugging.

11.14 Lifetimes and Memory Regions

Scaly uses Region-Based Memory Management (RBMM). Instead of garbage collection or manual malloc/free, values that need the heap are allocated in memory regions (pages) that are deallocated in bulk when their owning scope exits.

11.14.1 Allocation Is Inferred

Ordinary code carries no memory-management annotations at all. A constructed value lives on the stack when that suffices; when its internals allocate, or the value must outlive the statement that created it, the compiler places the allocation in an appropriate region:

• A value used only within the current function gets a *local* region, freed when the scope exits.

• A value returned from a function is placed in the *caller's* region, so it survives the return.

• A thrown error value is placed in the exception region the caller provides.

The standard library and the Scaly compiler itself are written in exactly this annotation-free style.

11.14.2 Escape Checking

The compiler proves that no reference outlives the memory it points into. A reference to a local that would escape the frame — through a return, through a store into an object that outlives the frame, or by being passed to a callee that stores it — is a hard compile-time error:

```
function dangling() returns pointer[int]
{
    var local 5
    return &local
}
; error: reference into a local page escapes via return - allocate the
; target with '#' (caller page) or '^name' (region) so it outlives the use
```

The check is interprocedural (it follows the value through calls into callees that store their parameters, and through closures) and conservative: code it accepts is safe.

11.14.3 Explicit Lifetime Suffixes

When a value must be placed in a region the inference would not choose — most commonly a heap object returned by pointer — an explicit lifetime suffix on the constructor call says where it lives. The suffix goes between the type and the argument list:

Caller (`#`)

> The value is allocated in the caller's region. It survives the function return and is freed with the caller's scope. Used for returning heap-allocated values by pointer.

Reference (`^name`)

> The value is allocated in the same region as the named value — for tying an allocation's lifetime to a specific longer-lived object, such as a collection it is stored into.

Thrown (`!`)

> The value is allocated in the exception region. The compiler infers this for values in throw position, so writing it is rarely necessary.

```
define Node ( data: int )

function create_node(data: int) returns pointer[Node]
    Node#(data)              ; allocated in the caller's region

let n create_node(7)
(*n).data                    ; 7 — valid in the caller's scope
```

11.14.4 By-Value Returns

Functions can return values by value — the simplest and most common case, and the reason most code needs no lifetime suffix at all:

```
function create_point(x: int, y: int) returns Point
{
    Point(x, y)    ; constructed and returned by value
}
```

By-value return avoids region allocation entirely. The value is constructed directly in the caller's stack frame or register.

12 Grammar

This chapter presents the complete grammar of the Scaly programming language in a formal notation. Each production defines a syntactic construct, with nonterminals linking to their definitions.

Notation:

• *identifier*, *literal* — terminal symbols from the lexer

• `'keyword'` — keyword terminals

• `X | Y` — alternatives

• `X?` — optional

• `X+` — one or more

• `X*` — zero or more

Program = *File* *Statement*`*`

File = *Package** *Use** *Declaration**

Declaration = *Symbol* `':'?`

Symbol = *Private* | *Definition* | *Function* | *Operator* | *Trait* | *Macro* | *Module*

Private = `'private'` *Export*

Export = *Definition* | *Function* | *Operator* | *Trait* | *Module*

Member = *Constituent* `':'?`

Constituent = *Definition* | *Function* | *Procedure* | *Operator* | *Implement* | *Trait* | *Macro* | *Module*

Definition = `'define'` `IDENTIFIER` *GenericParameters*`?` `':'?` *Attribute** *Concept*

GenericParameters = `'['` *GenericParameter*`+` `']'`

GenericParameter = `IDENTIFIER` *Attribute** `','`

Concept = *Class* | *Namespace* | *Union* | *Constant* | *Delegate* | *Intrinsic*

Class = *Structure* `':'?` *Body*`?` `':'?`

Body = `'{'` *Use** *Init** *DeInit*`?` *Member** `'}'` `':'?`

Namespace = `'{'` *Use** *Declaration** `'}'` `':'?`

Union = `'union'` `':'?` `'('` *Variant*`+` `')'` `':'?` *Body*`?` `':'?`

Variant = `IDENTIFIER` *Attribute** *TypeAnnotation*`?` `','` `':'?`

Constant = *Type* *Operand** `':'?`

Delegate = `'delegate'` *ParameterSet*`?` *Attribute** *Returns*`?` *Throws*`?`

GenericArguments = `'['` *GenericArgument** `']'`

GenericArgument = *Type* `','`

Optional = `'?'`

ParameterSet = *Parameters* | *Type*

Parameters = `'('` *Item** `')'`

Item = `IDENTIFIER` *TypeAnnotation*`?` `','` `':'?` *Attribute** `':'?`

Returns = `'returns'` *Type* *Attribute**

Throws = `'throws'` *Type* *Attribute**

Use = `'use'` *Name* `':'?`

Implement = `'implement'` *Type* *Attribute** `':'?` `'{'` *Use** *Method** `'}'` `':'?`

Trait = `'trait'` *Name* *Extends*`?` *Attribute** `'{'` *Use** *Method** `'}'` `':'?`

Method = *Function* | *Procedure* | *Operator*

Init = `'init'` *ParameterSet*`?` `':'?` *Action* `':'?`

DeInit = `'deinit'` `':'?` *Action* `':'?`

Function = `'function'` *Target*

Procedure = `'procedure'` *Target*

Operator = `'operator'` *Target*

Target = *Named* | *Routine*

Named = `IDENTIFIER` *Routine*

Routine = *GenericArguments*`?` *Lifetime*`?` *ParameterSet*`?` *Attribute** *Returns*`?` `':'?` *Throws*`?` `':'?` *Implementation*

Implementation = *Action* | *Extern* | *Instruction* | *Intrinsic*

Extern = `'extern'`

Instruction = `'instruction'` `':'?`

Intrinsic = `'intrinsic'` `':'?`

Extends = `'extends'` *Extend**

Extend = *Type* `','`

Macro = `'macro'` `IDENTIFIER` *Model* *Operand*+

Attribute = `ATTRIBUTE` *Model* `':'?`

Model = *Literal* | *Name* | *Object* | *Vector*

Module = `'module'` `IDENTIFIER` `':'?`

Package = `'package'` *Name* *Version*`?` *Source*`?` `':'?`

Version = `LITERAL` `'.'` `LITERAL`

Source = `LITERAL`

Initializer = `'('` *Operand*+ `')'`

Operand = *Expression* *MemberAccess**

MemberAccess = `'.'` *Name*

Expression = *Literal* | *Type* | *Object* | *Vector* | *Block* | *If* | *Match* | *Lambda* | *For* | *While* | *Choose* | *Try* | *Repeat* | *SizeOf* | *Is*

Literal = `LITERAL`

Object = `'('` *Component** `')'`

Component = *Operand*+ *Attribute** *Value*`?` `','`

Value = `':'` *Operand*+ *Attribute**

Vector = `'['` *Element*+ `']'` *Lifetime*`?`

Element = *Operand*+ *Attribute** `','`

Block = `'{'` *Use** *Statement** `'}'`

If = `'if'` *Operand*+ `':'` *Command* *Else*`?`

Else = `'else'` `':'`? *Command*

Match = `'match'` *Operand*+ `':'` *Branch*+ *Else*`?`

Branch = *Case*+ `':'` *Statement*

Case = `'case'` *Operand*+

Statement = *Command* `':'`?

While = `'while'` *Condition* `':'` *Label*`?` *Action*

Choose = `'choose'` *Operand*+ `':'` *When** *Else*`?`

Try = `'try'` *Condition* `':'` *When** *Else*`?`

Condition = *Operation* | *Let*

When = `'when'` `IDENTIFIER` `':'` *Name* `':'`? *Command* `':'`?

Command = *Operation* | *Let* | *Var* | *Mutable* | *Set* | *Continue* | *Break* | *Return* | *Throw*

Let = `'let'` *Binding*

Var = `'var'` *Binding*

Mutable = `'mutable'` *Binding*

Binding = `IDENTIFIER` *BindingAnnotation*`?` *Operand*+

BindingAnnotation = `':'` *BindingSpec*

BindingSpec = *Structure* | *Type* | *Array*

Array = `'['` *Type*`*` `']'`

Structure = `'('` *Part*`*` `')'`

Part = *Field* | *Property*

Field = `'private'` *Property*

Property = `IDENTIFIER` *TypeAnnotation* *Initializer*`?` `','` `':'`? *Attribute*`*` `':'`?

TypeAnnotation = `':'` *Type*

Continue = `'continue'` *Loop*`?` `':'`?

Break = `'break'` *Loop*`?` *Operand*`*` `':'`?

Loop = `'loop'` `IDENTIFIER`

Return = `'return'` *Operand*`*`

Throw = `'throw'` *Operand*`*`

Lambda = `'lambda'` *Operand*`+` `':'` *Action*

For = `'for'` `IDENTIFIER` *TypeAnnotation*`?` `'in'` *Operand*`+` `':'` *Label*`?` *Action*

Label = `'label'` `IDENTIFIER` `':'`?

Repeat = `'repeat'` *Label*`?` *Action*

Action = *Operation* | *Set*

Operation = *Operand*+

Set = `'set'` *Operand*`+` `':'` *Operand*+

SizeOf = `'sizeof'` *Type*

Is = `'is'` *Name*

Type = *Name* *GenericArguments*`?` *Optional*`?` *Lifetime*`?`

Name = `IDENTIFIER` *Extension*`*`

Extension = `'.'` `IDENTIFIER`

Lifetime = *Call* | *Local* | *Reference* | *Thrown*

Call = `'#'`

Local = `'$'`

Reference = `'^' IDENTIFIER`

Thrown = `'!'`

13 Definitions

13.1 Overview

Definitions declare new types that can be used throughout a program. Types can have properties, methods, and nested definitions.

13.2 Functions, Procedures, and Operators

A routine is introduced by one of three keywords, and that keyword — not the spelling of the name — decides how the routine behaves:

- *function* declares a pure function: its inputs are borrowed and immutable, and it returns a result without side effects. A function is applied prefix, by writing its name before a parenthesized argument list.
- *procedure* declares a routine that may mutate its inputs and have side effects (builders, I/O).
- *operator* declares an operator: a pure routine applied infix between its two operands (or prefix, for a unary operator).

The three identifier forms from the lexical chapter — alphanumeric, operator-character, and single-quoted string identifiers — are interchangeable as names here. A function may be named with operator characters, and an operator may be named with letters or a quoted string; what makes one a function and the other an operator is the declaration keyword, never the first character of the name.

```
function 'add seven'(n: int) returns int      ; string-named function
    n + 7

function ++(n: int) returns int               ; operator-char-named function
    n + 1

let a 'add seven'(35)                          ; applied prefix -> 42
let b ++(41)                                   ; applied prefix -> 42
let c ++ 41                                    ; one argument needs no parentheses ->
let c ++ 41                                    ; one argument needs no parentheses ->
```

Operators may be declared at the top level, not only as members of a type. Like functions, they accept any of the three name forms:

```
operator <+>(left: int, right: int) returns int    ; operator-char name
    left + right

operator 'divided by'(left: int, right: int) returns int   ; quoted name
    left / right

let c 40 <+> 2              ; infix -> 42
let d 84 'divided by' 2     ; infix -> 42
```

Built-in arithmetic, comparison, and logical operators carry their usual precedences. A custom operator that no level of the operation names — a text or single-quoted name, or a symbol such as == — binds most weakly of all, on a level of its own below `||`, so the built-in arithmetic in an expression combines before it. For example, given an operator `'minus'` that subtracts its right operand from its left, `20 'minus' 3 + 5` parses as `20 'minus' (3 + 5)` and yields `12`.

An operator's right parameter may be declared `lazy`: the caller's operand is handed over unevaluated, and the operator's body is put in where the operator stands. The standard library declares `and` and `or` on `bool` this way, so the right operand runs only where the left one does not decide:

```
operator and(left: bool, right: lazy bool) returns bool
    if left: right else false

let ok p <> null and p.x = 7   ; p.x is read only where p is there
```

The right operand is planned where the body puts it, so a null test on the left proves its name for it, as an `if` proves it for its branch. A free function and a method may declare a lazy parameter too: the call is the body put in where it stands, ordinary arguments evaluated once in the order they are written, a lazy one where the body names it:

```
function or_else(v: int, fallback: lazy int) returns int
    if v <> 0: v else fallback

let n or_else(cached, expensive())   ; expensive() runs only where cached is 0
```

13.3 Definitions

Definitions declare new types that can be used throughout a program.

13.3.1 Structure Definitions

The define keyword creates a new structure type with named properties.

Input	Result
`define Point(x: int, y: int)` `Point(3, 4)`	`{"_type":"Point","x":3,"y":4}`
`define Point(x: int, y: int)` `Point(3, 4).x`	`3`

`define Point(x: int, y: int)`
`let p Point(3, 4)`
`p.x + p.y`
	`7`

`define Point(x: int, y: int)`
`define Rectangle(origin: Point, width: int, height: int)`
`Rectangle(Point(0, 0), 10, 5).width`
	`10`

13.3.2 Union Definitions

Unions define types with multiple variants. Each variant can optionally carry a value.

Input	Result
`define Option union(Some: int, None)` `Some(5)`	`{"_type":"Option","_variant":"Some","value"`
`define Option union(Some: int, None)` `None`	`{"_type":"Option","_variant":"None"}`
`define Result union(Ok: int, Err: Str` `Ok(42)`	`{"_type":"Result","_variant":"Ok","value":4`
`define Result union(Ok: int, Err: Str` `Err("error")`	`{"_type":"Result","_variant":"Err","value":`
`define Option union(Some: int, None)` `Option.Some(5).value`	`5`

13.3.3 Function Names

A routine's role is set by its declaration keyword, not by how its name is spelled. A function may be named with ordinary letters, with a single-quoted string identifier (which may contain spaces),

or with operator characters; in every case it is an ordinary function, applied by writing its name before a parenthesized argument.

Input	Result

`function 'add seven'(n: int) returns int`
`    n + 7`
`'add seven'(35)`
	`42`

`function ++(n: int) returns int`
`    n + 1`
`++(41)`
	`42`

13.3.4 Operator Definitions

The operator keyword declares an operator, applied infix between its two operands. Like functions, operators may be named with operator characters or with a text or single-quoted name. Operators may be declared at the top level, not only inside a type. An operator that no level of the operation names, a text-named one among them, binds more weakly than every level, so the arithmetic combines first.

Input	Result

`operator <+>(left: int, right: int) returns int`
`    left + right`
`40 <+> 2`
	`42`

`operator 'plus'(left: int, right: int) returns int`
`    left + right`
`40 'plus' 2`
	`42`

`operator 'divided by'(left: int, right: int) returns int`
`    left / right`
`84 'divided by' 2`
	`42`

`operator 'minus'(left: int, right: int) returns int`
`    left - right`
`20 'minus' 3 + 5`
	`12`

13.3.5 Lazy Parameters

A parameter declared `lazy` before its type receives its argument unevaluated: the routine's body is put in where the call stands, and the argument runs where the body names it, and only there. This is how the standard library declares `and` and `or`, and it works for an ordinary function as well.

Input	Result

`function or_else(v: int, fallback: lazy int) returns int`
`    if v <> 0: v else fallback`
`procedure loud() returns int`
`{`
`    print "evaluated"`
`    7`
`}`
`or_else(5, loud())`
	5

`function or_else(v: int, fallback: lazy int) returns int`
`    if v <> 0: v else fallback`
`or_else(0, 6 * 7)`
	42

14 Expressions

14.1 Overview

Expressions are the core computational units of Scaly. Every expression evaluates to a value; one that answers nothing answers the empty tuple `()`. An expression is an *operation*: a sequence of things — values, names, operator names, tuples, vector literals and blocks — that the compiler reads from left to right. It holds one current value, and the *type* of that value says what the next thing means:

current value	next thing	meaning
none, a prefix, or one waiting for its operand	an operator name	a prefix: it takes the next value (`-x`, `a - -2`, `not not a`)
a value	an operator its type declares	that operator; it waits for its right operand
a function	a tuple	the call; the tuple is the argument list
a function	one other value	the call with that one argument (`print "hi"`, `twice -5`)
a type	a tuple	the construction (`Point(1, 2)`)
a type	a vector literal	the generic instance (`Vector[int]`)

current value	next thing	meaning
a value	a vector literal	indexing (`v[i]`)
any	`as T`, `is T`	the cast or the test of the whole operation so far

Parentheses make a tuple (and a tuple of one value is that value, so they also group), brackets a vector literal, braces a block, and the dot a member. Nothing else structures a sequence: there is no precedence table in the compiler. How strongly operators bind is stated by the standard library, as a protocol of *pending* types: `a +` answers a value that waits for its right operand, and the protocol decides what an operator after it means. The levels, strongest first, are listed under Section 14.3.

Two consequences are worth knowing before reading the rest of this chapter:

- *Application is stronger than waiting.* Before an operator takes what follows, that thing takes what is applied to it: `a + f(1)` adds the call's result, `a + v[i]` the element.
- *A call without parentheses takes exactly one thing.* `print f(x)` is `(print f)(x)`, which the compiler reports; an argument that is itself a call, an indexing or a construction takes its parentheses: `print(f(x))`.

14.2 Literals

Expressions are the core computational units of Scaly. Every expression evaluates to a value.

14.2.1 Boolean Literals

The boolean literals are true and false.

Input	Result
`true`	`true`
`false`	`false`

14.2.2 Integer Literals

Integer literals are sequences of decimal digits, optionally preceded by a minus sign for negative numbers.

Input	Result
`42`	`42`

Input	Result
`0`	`0`
`-7`	`-7`

14.2.3 Hexadecimal Literals

Hexadecimal literals start with 0x or 0X followed by hexadecimal digits (0-9, a-f, A-F).

Input	Result
`0xFF`	`255`
`0x1A`	`26`
`0x0`	`0`

14.2.4 Floating-Point Literals

Floating-point literals contain a decimal point separating the integer and fractional parts.

Input	Result
`3.14`	`3.14`
`2.0`	`2.0`
`0.5`	`0.5`

14.2.5 String Literals

String literals are enclosed in double quotes. They may span multiple lines, with line breaks preserved in the resulting string.

Input	Result
`"hello"`	`hello`
`""`	

14.2.6 Character Literals

A character is written as a string literal of one character wherever a char is expected: a char parameter, a typed binding, or the cast `as char`.

Input	Result
`"a" as char`	`a`
`"Z" as char`	`Z`

14.3 Operations

An operation is a sequence of things: values, names, operator names, tuples in parentheses, vector literals in brackets and blocks in braces, each perhaps followed by a member access with a dot. The compiler reads the sequence from left to right and holds one current value; the TYPE of that value says what the next thing means. A tuple after a function is the call, a tuple after a type the construction, a vector literal after a value the indexing, an operator name after a value one of the operators its type declares. Parentheses, brackets, braces and the dot are what structure a sequence, and nothing else does. An operation ends with a colon, a line break, or anything that is not part of it.

14.3.1 Prefix Operators

Where a value is expected an operator name is a prefix: it takes the next value as its operand. A value is expected at the start of an operation, after an operator, after a function, and after another prefix, so `a - -2` subtracts minus two, `twice -5` is `twice(-5)`, and `not not a` reads as written. An = at the start of an operation does nothing, so `let a = 42` reads as `let a 42`.

Input	Result
`-7`	`-7`
`-(-3)`	`3`
`+ 3`	`3`
`10 - -2`	`12`
`- -2`	`2`
`not not true`	`true`

`function twice(n: int) returns int`
`  n * 2`
`twice -5`
	`-10`

`= 6 * 7`	`42`

14.3.2 Operator Chains

Several operators in a row combine by their binding strength, which the standard library states as

a protocol of pending types: the compiler itself has no precedence table. Strongest first, the levels are `* / % div mod`; `+ -`; `<< >>`; `< > <= >=`; `= <>`; `&`; `xor`; `|`; `and &&`; `or ||`. An operator a type declares that no level names, such as == or a text name, binds most weakly of all. Operators of one level combine from left to right.

Input	Result
`3 + 4`	`7`
`10 - 3`	`7`
`6 * 7`	`42`
`84 / 2`	`42`
`17 % 5`	`2`
`3 + 4 * 2`	`11`
`20 - 5 - 3`	`12`
`1 << 2 + 1`	`8`
`1 + 2 * 3 = 7`	`true`
`1 < 2 and 3 < 4`	`true`

14.3.3 Application Before Operators

Before an operator takes the thing that follows it, that thing takes what is applied to it: a function its tuple, a value its vector literal, anything its members. So `a + f(1)` adds the call's result, and `a + v[i]` the element. A function called without parentheses takes exactly one thing; an argument that is itself a call, an indexing or a construction takes its parentheses, `print(f(x))`.

Input	Result

`function twice(n: int) returns int`
`    n * 2`
`3 + twice(4) * 2`
	`19`

`function twice(n: int) returns int`
`    n * 2`
`twice 21`
	`42`

14.3.4 Parentheses and Grouping

Parentheses around a single value group it: `(a)` is `a`. The group is read as an operation of its own, so a prefix may begin it inside a longer chain, and it combines first.

Input	Result
`5 * (- 2)`	`-10`
`2 * (3 + 4)`	`14`
`(2 + 3) * (4 + 1)`	`25`

14.3.5 Casts Take the Whole Operation

A cast `as T` and a test `is T` apply to the whole operation before them: `a + b as T` is `(a + b) as T`. A cast of the right side alone takes parentheses.

Input	Result

`let a: u8 200`
`let b: u8 100`
`a + b as int`
	`44`

`let a: u8 200`
`let b: u8 100`
`a + (b as int)`
	`300`

14.3.6 Identifier Shapes

Identifiers can have name shape (alphanumeric) or operator shape (symbols). The shape is purely lexical and does not determine behavior: what a name means depends on its declaration and on what the operation holds when it is reached.

Input	Result
`10 div 3`	`3`
`abs (- 5)`	`5`

14.3.7 Let Bindings

Let bindings introduce local variables. The syntax is let followed by a name and a value expression (no equals sign). Multiple statements are separated by colons or newlines.

Input	Result
`let x 5` `x`	`5`
`let x 5` `x + 3`	`8`

Input	Result

`let x 5`
`let y 3`
`x + y`
	`8`

`let x 2 + 3` `x * 2`	`10`

14.4 Tuples and Function Values

A tuple is a value: an anonymous record written in parentheses. Its components may carry labels, and its type is written the same way, `(int, String)`. The empty tuple `()` is the type of no value, which a routine without a result answers. Functions are values too: a name bound to a function, or a method bound to its receiver, is called later like the function itself.

14.4.1 Tuples

A labelled component is read by its label. Unlabelled components are reached by decomposition, `let (a, b) t`; a tuple is not an array and has no indexing. A routine returns several values as a tuple.

Input	Result
`let t (x: 3, y: 4)` `t.x * t.y`	`12`
`let (a, b) (6, 7)` `a * b`	`42`

`function pair() returns (int, int)`
`    (3, 4)`
`let (p, q) pair()`
`p + q`
	`7`

14.4.2 Applying a Bound Tuple

A tuple bound to a name stands for its components where it is applied: `f args` calls f with them, and `Point(args)` builds the record from them. Its components are evaluated once, when it is bound.

Input	Result

Input	Result

`function add(a: int, b: int) returns int`
`    a + b`
`let args (40, 2)`
`add args`
	42

`define Point(x: int, y: int)`
`let args (1, 2)`
`let p Point(args)`
`p.y`
	2

14.4.3 Functions as Values

`let g twice` binds the function at compile time and costs nothing; `g(21)` is a direct call. `x.m` without a tuple is the method bound to its receiver. Where a delegate type is declared, a named function becomes a closure value, chosen by the delegate's signature.

Input	Result

`function twice(n: int) returns int`
`    n * 2`
`let g twice`
`g(21)`
	42

`define Counter(n: int)`
`{`
`    function plus(this, k: int) returns int`
`        n + k`
`}`
`let c Counter(40)`
`let m c.plus`
`m(2)`
	42

`define IntFn delegate(int) returns int`
`function apply(f: IntFn, x: int) returns int`
`    f(x)`
`function twice(n: int) returns int`
`    n * 2`
`apply(twice, 21)`
	42

14.5 Comparison Operators

Comparison operators compare two values and return a boolean result.

14.5.1 Equality

The equality operator (=) tests if two values are equal. The inequality operator (not equal to) tests if two values are different.

Input	Result
`5 = 5`	`true`
`5 = 3`	`false`
`5 <> 3`	`true`
`5 <> 5`	`false`

14.5.2 Relational Operators

Relational operators compare the ordering of values.

Input	Result
`3 < 5`	`true`
`5 < 3`	`false`
`5 > 3`	`true`
`3 > 5`	`false`
`5 <= 5`	`true`
`5 <= 3`	`false`
`5 >= 5`	`true`
`3 >= 5`	`false`

14.6 Boolean Operators

Boolean operators combine boolean values.

14.6.1 Boolean Negation

The word operator `not` negates a boolean value. The tilde is the bitwise complement, which on a bool is the same. The exclamation mark is not negation in Scaly: it is the sigil of the thrown lifetime.

Input	Result

Input	Result
`not true`	`false`
`not false`	`true`
`~true`	`false`
`~false`	`true`

14.6.2 Boolean And/Or

The operators `and` and `or` (also spelled `&&` and `||`) are declared by the standard library with a lazy right operand: the right side is evaluated only where the left one does not decide. The bitwise operators `&`, `|` and `xor` combine two bools as well, and evaluate both sides.

Input	Result
`true and false`	`false`
`false or true`	`true`

`procedure noisy() returns bool`
`{`
`  print "evaluated"`
`  true`
`}`
`false and noisy()`
	`false`

`procedure noisy() returns bool`
`{`
`  print "evaluated"`
`  false`
`}`
`true or noisy()`
	`true`

`true & true`	`true`
`true & false`	`false`
`false | false`	`false`
`true | false`	`true`
`true xor true`	`false`

14.7 Bitwise Operators

Bitwise operators manipulate integer values at the bit level.

14.7.1 Bit Shifts

The left shift operator shifts bits to the left, effectively multiplying by powers of two. The right shift operator shifts bits to the right, effectively dividing by powers of two.

Input	Result
`1 << 4`	`16`
`5 << 3`	`40`
`16 >> 2`	`4`
`100 >> 3`	`12`

14.8 Lambdas

A lambda is an anonymous function written with a backslash: `\params: body`. Parameters are space-separated. A lambda bound to a name and applied to arguments is monomorphized at the call site for those argument types, so the same lambda works at any argument type.

14.8.1 Lambda Application

A lambda bound to a name can be called like a function.

Input	Result
`let inc \x: x + 1` `inc(5)`	`6`
`let add \x y: x + y` `add(3, 4)`	`7`

`let dbl \x: x + x`
`let a dbl(10)`
`let b dbl(21)`
`a + b`
	`62`

14.8.2 Captures

A lambda captures enclosing variables by value. The captured value is fixed when the lambda is applied.

Input	Result

Input	Result

`let n 10`
`let f \x: x + n`
`f(5)`
	`15`

`let a 3`
`let b 4`
`let g \x: x + a + b`
`g(10)`
	`17`

14.8.3 Captures in Braced Bodies

Capture discovery reaches inside a braced body, and a body-local binding that shadows an enclosing name is the local, not a capture.

Input	Result

`let n 10`
`let cap \x: { x + n }`
`cap(5)`
	`15`

`let n 10`
`let shade \x: { let n 99`
`x + n }`
`shade(5)`
	`104`

14.8.4 Embedded Calls

A lambda call may appear anywhere inside a larger expression, not only as the whole right-hand side of a binding. Like every application, each call takes its arguments before the operators around it combine.

Input	Result

`let inc \x: x + 1`
`let add \x y: x + y`
`inc(5) + add(3, 4)`
	`13`

`let inc \x: x + 1`
`let n 100`
`n + inc(9)`
	`110`

`let n 100`
`let cap \x: x + n`
`cap(1) + cap(2)`
	`203`

14.8.5 Delegate Values

A `delegate` names a function type. A non-capturing lambda assigned to a delegate-typed binding is a function-pointer value that can be passed to a function and called indirectly. A lambda literal may also be passed straight to a delegate parameter, and so may a named function.

Input	Result

`define IntFn delegate(int) returns int`
`function apply(f: IntFn, x: int) returns int`
`    f(x)`
`let g: IntFn \x: x + 1`
`apply(g, 5)`
	6

`define IntFn delegate(int) returns int`
`function twice(f: IntFn, x: int) returns int`
`    f(f(x))`
`let dbl: IntFn \x: x + x`
`twice(dbl, 20)`
	80

14.8.6 Closures

A capturing lambda assigned to a delegate type is a true closure value carrying the captured variables. (Closures returned from a function escape onto the caller's region -- see the escaping-closure AOT regression test.)

Input	Result

`define IntFn delegate(int) returns int`
`function apply(f: IntFn, x: int) returns int`
`    f(x)`
`let n 10`
`let add: IntFn \x: x + n`
`apply(add, 5)`
	15

14.9 Choose Expressions

Choose expressions pattern match on union types, allowing different code paths based on which variant is present.

14.9.1 Choose on Variant Construction

Choose can match directly on constructed variants.

Input	Result
`define Option union(Some: int, None)` `choose Option.Some(7): when v: Some: v else 0`	7
`define Option union(Some: int, None)` `choose Option.None: when v: Some: v else 0`	0

14.9.2 Choose with Binding

When a variant carries a value, the value can be bound to a name.

Input	Result
`define Result union(Ok: int, Err: int)` `choose Result.Ok(42): when x: Ok: x + 1 else 0`	43

14.9.3 Multiple When Clauses

Choose can have multiple when clauses to match different variants.

Input	Result
`define Result union(Ok: int, Err: int)` `choose Result.Ok(10): when x: Ok: x when e: Err: e + 100`	10
`define Result union(Ok: int, Err: int)` `choose Result.Err(5): when x: Ok: x when e: Err: e + 100`	105

14.10 Optional Types

Optional types use null pointer optimization (NPO) for efficient representation.

14.10.1 Is Null Check

The is null expression checks if an optional value is None.

Input	Result

`define Option[T] union: (Some: T, None)`
`function check(x: int?) returns int: if x is null: 0 else 1`
`check(Option[int].None)`
	0

`define Option[T] union: (Some: T, None)`
`function check(x: int?) returns int: if x is null: 0 else 1`
`check(Option[int].Some(42))`
	1

14.10.2 Optional Type Sugar

T? is syntactic sugar for Option[T].

Input	Result

`define Option[T] union: (Some: T, None)`
`function unwrap(x: int?) returns int: choose x: when v: Some: v else 0`
`unwrap(Option[int].Some(42))`
	42

14.10.3 Generic Option Types

Generic unions like Option[T] work with choose for pattern matching.

Input	Result
`define Option[T] union: (Some: T, None)` `choose Option[int].Some(42): when v: Some: v else 0`	42
`define Option[T] union: (Some: T, None)` `choose Option[int].None: when v: Some: v else -1`	-1

14.11 Variant Construction

Union variants can be constructed explicitly using Type.Variant syntax.

14.11.1 Construct Variants

Variants are constructed with Type.Variant(value) syntax.

Input	Result
`define Result union: (Ok: int, Error: int)` `choose Result.Ok(42): when x: Ok: x else 0`	42
`define Result union: (Ok: int, Error: int)` `choose Result.Error(100): when e: Error: e else 0`	100

14.11.2 Variant Is Expression

The is expression tests if a union value matches a variant.

Input	Result

Input	Result
`define Result union: (Ok: int, Error: int)` `if Result.Ok(50) is Ok : 1 else 0`	1
`define Result union: (Ok: int, Error: int)` `if Result.Error(33) is Ok : 1 else 0`	0

15 Control Flow

15.1 Overview

Conditions and loops are expressions like every other construct: an `if` with both branches has a value, and a `repeat` loop answers the operand of the `break` that ends it. A condition needs no parentheses; a body of one statement needs no braces, a longer one is a block in braces that begins on a line of its own. `if cond stmt` without an `else` runs the statement when the condition is *true*, and control continues after it.

Conditions are written as `and`/`or` chains. A null test proves its name for what follows it in the chain: in `p <> null and p.x = 7` the right side reads `p.x` only where `p` is there.

15.2 If Expressions

If expressions evaluate a condition and return one of two values.

15.2.1 Basic If

The if expression takes a boolean condition and two branches.

Input	Result
`if true : 1 else 2`	1
`if false : 1 else 2`	2
`if 5 > 3 : 10 else 20`	10

15.2.2 If with Bindings

If expressions can use values from let bindings.

Input	Result

Input	Result
`let x 5` `if x > 3 : x * 2 else x`	`10`

`let a 7`
`let b 3`
`if a > b : a else b`
	`7`

15.2.3 Else-If Chains without a Final Else

An if statement may chain an else if without a final else. Such a chain yields no value; when no condition holds, the statement falls through and any bindings keep their prior values. (Regression: the emitter used to merge the arms' values into a dead phi node that failed LLVM verification when the inner arm's value was computed inside its own branch.)

Input	Result

`function clamp(dmin: int) returns int`
`{`
`    var adj 7`
`    if dmin > 100`
`        adj := 0`
`    else if dmin > 50`
`        adj := dmin + 1`
`    adj`
`}`
`clamp(200) * 10000 + clamp(60) * 100 + clamp(10)`
	`6107`

15.3 While Loops

While loops repeat as long as a condition is true.

15.3.1 Basic While

A while loop evaluates its body repeatedly while the condition is true.

Input	Result

`var x 0`
`while x < 3 : x := x + 1`
`x`
	`3`

`var x 10`
`while x < 5 : x := x + 1`
`x`
	`10`

15.3.2 While with Accumulation

While loops can accumulate values across iterations.

Input	Result

`var sum 0`
`var i 1`
`while i <= 3 : { sum := sum + i: i := i + 1 }`
`sum`
	`6`

`var result 1`
`var n 5`
`while n > 0 : { result := result * n: n := n - 1 }`
`result`
	`120`

15.4 For Loops

For loops iterate over a range of integers.

15.4.1 Basic For

A for loop iterates from 0 to n-1 where n is the range expression.

Input	Result

`var count 0`
`for i in 3 : count := count + 1`
`count`
	`3`

`var x 42`
`for i in 0 : x := 0`
`x`
	`42`

15.4.2 For with Accumulation

For loops can accumulate values using the loop variable.

Input	Result

`var sum 0`
`for i in 5 : sum := sum + i`
`sum`
	`10`

Input	Result

`var prod 1`
`for i in 4 : prod := prod * (i + 1)`
`prod`
	`24`

15.5 Break and Continue

Break exits a loop early, continue skips to the next iteration.

15.5.1 Break Statements

Break exits the innermost enclosing loop.

Input	Result

`var sum 0`
`for i in 10 : { if i = 3 : break else sum := sum + i }`
`sum`
	`3`

`var x 0`
`while x < 100 : { x := x + 1: if x = 5 : break }`
`x`
	`5`

15.5.2 Continue Statements

Continue skips the rest of the current iteration and continues with the next.

Input	Result

`var sum 0`
`for i in 6 : { if i % 2 = 1 : continue else sum := sum + i }`
`sum`
	`6`

`var sum 0`
`var i 0`
`while i < 10 : { let current i: i := i + 1: if current % 2 = 1 : continue else sum :`
`sum`
	`20`

15.5.3 Nested Loops

By default break and continue affect only the innermost loop.

Input	Result

Input	Result

`var count 0`
`for i in 3 : { for j in 10 : { if j = 2 : break else count := count + 1 } }`
`count`
	`6`

`var count 0`
`for i in 3 : { for j in 5 : { if j % 2 = 1 : continue else count := count + 1 } }`
`count`
	`9`

15.5.4 Labeled Break and Continue

A loop may carry a label written `label NAME` after its colon. From within a nested loop, `break loop NAME` exits the named loop and `continue loop NAME` jumps to its next iteration, rather than acting on the innermost loop.

Input	Result

`var count 0`
`for i in 3 : label outer { for j in 5 : { if j = 2 : break loop outer else count :=`
`count`
	`2`

`var count 0`
`for i in 3 : label outer { for j in 5 : { if j = 0 : continue loop outer else count`
`count`
	`0`

15.6 Repeat Loops

A repeat loop runs its body forever and exits only through break (or return). Because break is the only way out, a break carrying a value (`break v`) makes the repeat an expression that yields that value.

15.6.1 Repeat and Break Value

The value of a repeat expression is the operand of the break that exits it.

Input	Result

`var i 0`
`let answer repeat { if i = 5 : break i + 100 else i := i + 1 }`
`answer`
	`105`

`var i 0`
`repeat { if i = 3 : break else i := i + 1 }`
`i`
	`3`

15.7 Mutable Variables

A binding introduced with var may be assigned a new value; one introduced with let may not. The assignment is written `target := source`, in the Algol tradition, and is a statement of its own: it introduces no name and is never part of a condition. The equals sign always compares.

15.7.1 Var and Assignment

Var creates a mutable variable, and `:=` assigns it.

Input	Result
`var x 5` `x`	`5`

`var x 5`
`x := 10`
`x`
	`10`

`var x 3`
`x := x + 1`
`x`
	`4`

`var a 2`
`var b 3`
`a := a * b`
`a`
	`6`

15.8 Match

`match` compares one value against a list of `case` alternatives and takes the first that is equal; one case may name several values, and `else` takes the rest. It works on numbers, characters and strings, which it compares by content:

```
function kind(n: int) returns int
    match n
        case 1: 10
        case 2 case 3: 20
        else 0

let word "b"
match word
    case "a": print "A"
    case "b": print "B"
    else print "?"
```

A union is matched by its variants with `choose`, described under Section 11.12.

16 Modules and Packages

16.1 Overview

A Scaly program can be split across several source files, and it can depend on other packages. There are no build files, manifests, or configuration files — three statements inside the source itself organize everything:

- `module` loads another source file into the current program or package.
- `package` declares a dependency on another package.
- `use` makes types defined in another module visible in the current file.

16.2 Modules

Every source file is a module. The statement `module shapes` loads the file `shapes.scaly` from the directory of the file that declares it and makes it part of the program:

```
; shapes.scaly
define Box(width: int, height: int)
{
    function area(this) returns int
        width * height
}

; myprog.scaly
module shapes
use myprog.shapes.Box

let b Box(3, 4)
if b.area() = 12
    print "area is 12"
```

A module may declare child modules of its own. Those live in a directory named after the parent module: if `compiler.scaly` declares `module lexer`, the child is loaded from `compiler/lexer.scaly`. This mirrors how the standard library and the compiler themselves are laid out.

Top-level statements of a loaded module are ignored — only its definitions become part of the program. Programs have top-level statements; libraries consist of definitions only.

16.3 Packages

A package is a source distribution. Its version is encoded in its filesystem path — `packages/scaly/0.2.1/` — and that path is the single source of truth; there is no separate version manifest. The statement

```
package scaly 0.2.0
```

declares a dependency on that package. A package is looked for among the packages of the installation, `$SCALY_HOME/packages/` (the installed programs set SCALY_HOME themselves), and then among the project's own, `./packages/` under the directory the compiler runs in. So a project brings the packages the installation does not have by putting them beside its sources, and it cannot put a package of its own in front of an installed one of the same name. Without SCALY_HOME only the current directory is searched. A declared package that is nowhere fails the compile with an error naming the places.

The version in a declaration is a *minimum*. One compilation takes one version of a package, and where a program reaches the same package through several declarations — its own and those of the packages it uses — it takes the highest version any of them names: a program that asks for `json 0.1.5` and uses a package that asks for `json 0.1.2` is built, with `0.1.5` for both. That holds among versions that go together — within one major version, and below 1.0 within one second number (Section 16.3.3 says why). Such versions form a *line*. Two lines of one package — a package that asks for `json 0.1` beside one that asks for `json 0.2` — are two packages to the build: both are compiled and linked, each asker gets its own, and a name of `json` written in a file means the line that file's package declares. What does not go: a value of one line handed to a routine of the other (they are different types), one file that declares both lines, and two lines of a package that brings C or assembly files. Each line has its own state — a cache or a counter kept in a package's global exists once per line — and what builds this way is not raised, so **scaly publish** says when a newer line of a dependency exists. Nothing else decides: there is no lock file and no solver, the versions follow from the declarations alone, and adding a dependency can raise a version but never lower one. A package's root file declares the packages it uses itself, so a program names only the packages it calls directly; circular package dependencies are forbidden.

The standard library package `scaly` is the implicit *prelude*: every program depends on it automatically, and its types are visible everywhere without any `use` statement. Pass `--no-prelude` to compile without it, for bare-metal or embedded targets. See Appendix B for what the standard library provides.

16.3.1 Packages from a git repository

A declaration may name where its package comes from, as a string after the version:

```
package json 0.1.3 "github.com/someone/json"
```

The source is a git repository that holds the package at `packages/json/0.1.3/` — the layout a project's own packages have. One repository serves every version it has published (Section 16.3.3). `"github.com/someone/json"` stands for `https://github.com/someone/json.git`; an address with a scheme, a `user@host:path` address and a local path are taken as they are, and whatever git needs to reach a private repository is git's business. A version is required where a source is given.

scaly run, **scaly test** and **scaly build** fetch what a program declares before they compile it, and what the fetched packages declare in their turn; **scalyc** never reaches the network and says which commands do. This needs **git** on the PATH. A fetched package is kept under `~/.scaly/packages/` (SCALY_PACKAGES names another place), by source, name and version, with its files read-only; nothing is written into the project. It is the third place a package is looked for, after the installation and the project's `./packages/` — so copying a package there is how a project keeps its own copy, and editing that copy is how it patches a dependency.

A version the package's record names is taken from the commit named there, any other from the head of the default branch. A published version does not change (Section 16.3.3). The first fetch remembers the commit and the content of the version's directory beside it; a version that is fetched again and is no longer what it was is refused, with both commits named. One package comes from one source: a program that reaches it through two different sources is refused, while two spellings of one address are one source.

A package may bring C or assembly files, which are compiled on the machine that builds it; the tool says so when it fetches one.

What a package defines is the package's own: two packages may each define a function, a record with its methods or a mutable global of one name, and each keeps its own. A record of one package is another type than a same-named record of another, and a name written in a package means that package's own before anything a `use` brings in. A program may hold values of both, call their methods and hand each back to its package; handing one package's record to the other is refused like any wrong type. The standard library's names and a program's own are not marked.

A package's constants and mutable globals are its own as well. A mutable global is one cell wherever it is read: a program that names a package's counter reads the counter the package's routines write. A bare name means the global of the package it is written in before another package's of the same name; written where two packages in reach both declare it and neither is the writer's own, it is refused, since nothing says which is meant.

A `use` makes a name writable — as a type, in a construction, before a static call. A value brings its type along: its methods and fields are reached without naming the type, so `let a make_alpha(1)` and `a.grown()` need no `use` of what `make_alpha` returns.

16.3.2 The cache

A package is compiled once. The first program that uses it has it compiled into the cache, `~/.scaly/cache` (SCALY_CACHE names another place); every later build links what lies there. Beside the compiled package lies its *interface* — the package as a caller sees it, every declaration and no body — and that is what the compiler reads from then on instead of the sources. Nothing of this is written into a project or a package, and nothing of it is ever committed.

Each entry is kept under a key over everything that went into it: the compiler, every file of the package, and the versions of the packages it declares. So an entry is never wrong, only no longer asked for — a changed package or a new compiler leaves the files of the state before. **`scaly clean`** empties the cache; what is needed is made again by the next build. The packages that were fetched (`~/.scaly/packages`) and the programs that were installed (`~/.scaly/bin`) are not the cache and stay.

16.3.3 Versions, and what publishing means

The rules of a package's versions are few, and the tool checks them.

A version is a directory. `packages/json/0.1.3/` is version 0.1.3 of `json`, and the next version is another directory beside it. There are no tags, no registry and no manifest.

Published is what the package's record names. Beside its version directories a package keeps the file `packages/json/published`, a line for each published version:

```
0.1.2 8c1f0e…  5d2a97…
0.1.3 b41c77…  0e93ab…
```

The version, the commit its directory lay at when it was published, and the git tree of that directory. **scaly publish** writes the lines; nobody else needs to. A version directory *without* a line is a version being written: it may change from commit to commit, on the default branch too, and a program that declares it gets it as it stands — and is told when it has changed since.

A published version does not change. Its line stays as it is, and its directory, while it is in the tree, stays the tree the line names. A correction, however small, is a new version: copy the directory, change the copy.

A published version may leave the tree. The line names the commit that holds it, and a build that declares the version fetches that commit. So a repository need not carry every version it ever published side by side: the directories being worked on are in the tree, the others in its history. Removing one is an ordinary commit that deletes the directory and leaves the record alone.

The number says what changed. A new version is compared with the one before it — the highest lower version, in the tree or in the record — by what the two *declare*: every function, procedure, operator and initializer with its parameters and its result, every record with its fields, every union

with its variants, every constant, global and alias. Bodies are not part of it, a generic routine's body neither. A package it declares counts too, by its line: a version that moves from `json 0.1` to `json 0.2` has changed what it declares, because a program that uses both packages must follow. Adding or dropping a dependency changes nothing a user had.

against the version before	from 1.0 on	below 1.0
a declaration is gone or reads differently	the first number rises	the second number rises
declarations were added, none is gone	the second number rises	the third number rises
bodies only	the third number rises	the third number rises

A version below 1.0 has promised nothing yet, so there a breaking change costs the second number; 1.0.0 is the statement that what the package declares is meant to stay. A build follows the same reading: the highest version asked for stands in for the lower ones where the table says nothing was taken away — within one first number from 1.0 on, within one second number below it. `0.2.0` does not stand in for `0.1.5`, and a program that reaches both is told so.

The comparison is deliberately strict. Declarations are compared as they are written, comments and line breaks apart: a renamed parameter, a constant's new value, a field added to a record all count as a declaration that reads differently. And nothing in a package is private, so every declaration counts.

```
scaly publish --check
scaly publish
```

Both run at the root of the package's repository and speak with the remote `origin` (another remote, an address or a path may follow). **`scaly publish --check`** changes nothing: it says whether every published version is still what its line names, whether the remote's record is still all here, and whether every new version's number says what it changed. A version that understates its change is refused with the declarations that are gone and the number it takes at least; a new version that does not compile is refused too. A new version of another line (0.2.0 after 0.1.x, 2.0.0 after 1.x) may change anything, so the version before is not compared — it may be written in a language the compiler of today no longer reads. Where a new version declares a package of which a newer line has been published meanwhile, it says so in a note — the declaration is not wrong, but no build will take the newer line until its author raises it. It answers 0 when all is well, 1 when a rule is broken and 2 when it could not ask — no repository, or the remote out of reach. It belongs in the package's own test run, so a change to a published version is noticed before it is pushed and not by the first program that fetches it.

`scaly publish` checks the same and then publishes what is new: each new version gets its line — the commit is the current one, so the version must be committed —, the record is committed, and both commits are pushed. From then on the version is fixed.

Making the next version of a package is therefore: give `packages/json/0.1.3/` the number the change takes, work and commit there as long as it takes, and **`scaly publish`** when it is done. The directory may be copied, and then both versions lie side by side; it may as well be *renamed*, because the published one lives on at its commit, and then the tree holds one directory per package. **`scaly next json 0.1.4`**, at the root of the repository, does that: it renames the directory with git and rewrites every tracked file that names it (a script, a test). It wants a tree without uncommitted changes and a number higher than the one that is there, and it leaves a published directory of another package as it was published. A package that is released together with a program — a compiler with its parser, say — publishes a version with each release and works on the next one in between.

A declared version is a minimum. `package json 0.1.3` asks for 0.1.3 or what stands in for it. Among the declarations of one build that is the highest version asked for; and where a place a package is looked for holds no directory of the version itself, it is the highest version of that line there that is not lower. So the declarations need not follow when a package they name begins its next version: a program that says `package json 0.1.3` builds against a tree or an installation that has `packages/json/0.1.4/` alone. A version of another line never stands in, and neither does a lower one.

16.3.4 Starting a package

```
scaly new shapes --lib
```

creates `shapes/packages/shapes/0.1.0/shapes.scaly`: a package with one function and its test. Its root file declares the packages it uses — the standard library too, which only a program gets without saying so:

```
package scaly 0.2.0

define shapes
{
    function greeting(name: String) returns String
        "Hello, `name`!"

    function test_greeting() returns int
    {
        if greeting("Scaly") <> "Hello, Scaly!"
            return 1
        0
    }
```

```
}
```

`scaly test packages/shapes/0.1.0/shapes.scaly` runs its tests. A program in the directory that holds `packages/` uses it with `package shapes 0.1.0`; once the directory is a git repository somewhere, any program does, by naming that repository as the source.

16.3.5 The programs of a package

A package that is a library first may still bring programs — a parser its validator and its normalizer, say. The programs a package offers are the files of the directory `programs/` beside its root file, each a program named after its file:

```
packages/shapes/0.1.0/
    shapes.scaly              the library
    shapes/                   its modules
    programs/
        shapes.scaly          the program "shapes"
        shapes_check.scaly    the program "shapes_check"
```

A program there is an ordinary program that declares the package it belongs to, without a source — its package and the others of the same repository lie beside it:

```
package shapes 0.1.0

print(shapes.greeting("world"))
```

One of the programs may be called like the package. Other program files a package keeps — drivers for its tests, experiments — stay outside `programs/` and are not offered.

```
scaly install github.com/someone/shapes shapes 0.1.0
scaly install github.com/someone/shapes shapes 0.1.0 shapes_check
```

fetches the package from that repository, builds its programs — all of them, or the one named — optimised as **`scaly build --release`** does, and puts them beside **scaly** itself, in `~/.scaly/bin` (SCALY_BIN names another place). A package without `programs/` is a library, and **scaly install** says so.

`scaly install` without arguments lists what it has installed — program, package, version and source —, and **`scaly uninstall shapes_check`** removes one of those. It goes by its own record and removes nothing else of that directory.

16.4 Imports and Visibility

Type definitions are private to their module by default. A type defined in another module — even one loaded into the same program with `module` — is hidden until the current file imports it with a `use` statement. This keeps every file's dependencies explicit: the list of `use` statements at the top of a file is a complete inventory of the outside types the file relies on.

16.4.1 What Is Visible Without an Import

Three kinds of types need no `use` statement:

- Types defined *in the same file*.
- Types of the *prelude* — the standard library (`String`, `Vector`, `Option`, …) is always visible.
- The namespace of a *declared child module*. When a file declares `module geometry` and `geometry.scaly` defines a namespace of the same name, that namespace is visible in the declaring file, so its members can be reached qualified:

```
; geometry.scaly
define geometry
{
    function unit_area() returns int
        1
}

; myprog.scaly
module geometry

let a geometry.unit_area()   ; qualified access, no use needed
```

Visibility applies to named type definitions. Free functions are resolved through their namespaces and are not gated separately.

16.4.2 Importing a Single Type

A `use` statement names a type by its full dotted path. The path starts with the program or package name, followed by the module path, ending in the type name:

```
use myprog.shapes.Box            ; Box from module shapes of program myprog
use scalyc.compiler.Modeler      ; Modeler from package scalyc
```

A module that is named after the single type it defines — the file `Modeler.scaly` defining `Modeler` — is addressed once, as the path spells it; the module and the type do not appear as

two separate path segments.

16.4.3 Importing Everything from a Module

A trailing `*` imports every type a module defines:

```
module shapes
use myprog.shapes.*

let b Box(3, 4)              ; Box is visible through the glob
if b.area() = 12
    print "glob works"
```

Globs are convenient for a module whose types are used pervasively. Prefer single-type imports when a file uses only one or two names from a module — they document the dependency more precisely.

16.4.4 When a Type Is Not Visible

Referencing a type that exists in the program but has not been imported fails the compile. The error names the module that defines the type and states the exact `use` line that would import it:

```
myprog.scaly:3:7: error: function not found: Box - defined in
    myprog.shapes but not visible here; add: use myprog.shapes.Box
```

Two compiler flags support working with visibility: `--no-use-vis` disables enforcement entirely (every type in the program is visible everywhere, the pre-visibility behavior), and `--use-vis` prints an advisory report of every reference that strict visibility would reject — useful when migrating an existing codebase, to collect the needed `use` lines before turning enforcement on.

A. The Tools

Scaly comes as three programs: **scaly**, the tool that runs, builds and tests programs; **scalyc**, the compiler by itself; and **scalyls**, the language server an editor talks to. This appendix says how they are installed and what each of them takes.

A.1 Installation

The installer — `install.sh` on macOS and Linux, `install.ps1` on Windows (Section 2.1.1) — puts everything under `~/.scaly` (`%USERPROFILE%\.scaly` on Windows) and the three commands on your PATH. It needs no administrator rights for that. The standard library and

the standard packages — `http`, `https`, `tls`, `json`, `compress`, `pg`, `redis` and `h3` — come with it as sources; a package is compiled the first time a program uses it and kept in `~/.scaly/cache`.

Before it fetches anything, the installer shows Scaly's license (MIT, with its disclaimer of warranty) and asks whether you accept it; without that, nothing is installed. Where nobody can answer — a Dockerfile, CI — **`SCALY_ACCEPT_LICENSE=1`** accepts it:

```
curl -fsSL https://scaly.io/install.sh | SCALY_ACCEPT_LICENSE=1 sh
```

A.1.1 Ready to run, or built on your machine

For these systems the installer downloads the programs ready to run, which takes a few seconds:

- macOS 26 on Apple silicon, macOS 15 and newer on Intel
- Linux with glibc 2.35 or newer (RHEL 9, Debian 12, Ubuntu 22.04 and later, Fedora, openSUSE, Arch, NixOS), on arm64 and x86-64
- Windows 10 and 11, on x64 and arm64

Building a program needs a C compiler, which links it: Apple's Command Line Tools on macOS, clang on Linux, the Visual Studio Build Tools on Windows. Where it is missing, the installer fetches it, and the system asks for your password (on Windows, for administrator rights) once. On macOS and Linux, **`SCALY_BUILD_TOOLS=skip`** leaves that out.

On a macOS or a Linux that is not in the list — an older version, another distribution, or one where the ready-made programs do not start — the installer builds Scaly on your machine instead. Scaly ships its compiler as LLVM IR, the *seed* (Appendix C), so that route needs LLVM 21 and takes about half a minute:

```
brew install llvm@21                                    ; macOS
sudo apt install llvm-21 llvm-21-dev lld-21 clang     ; Ubuntu and Debian
```

The installer says which of the two routes it takes; **`SCALY_FROM_SEED=1`** asks for the second one anywhere.

A.1.2 Versions, updates, removal

The installer takes the newest version of the toolchain. A version that is out does not change afterwards, so a particular one can be asked for by its number and is the same every time (versions before 0.2.29 are no longer offered for download):

```
curl -fsSL https://scaly.io/install.sh | SCALY_VERSION=0.2.29 sh
```

The toolchain's number is its own. The packages it brings — the standard library among them — have theirs, and a program's `package scaly 0.2.0` goes on meaning the standard library of whatever toolchain is installed, as long as that is of the same line (Section 16.3.3).

Running the installer again replaces the toolchain with the current one. To remove Scaly, delete `~/.scaly` and the PATH line the installer added to your shell profile.

A.2 scaly

The tool you will use most. Started without arguments it is the interactive session (Section 2.1.3); with a verb, it does one of these:

`scaly run [--gpu]` *`file`* `[`*`arguments`*`]`

> Compiles the program in memory and runs it at once, handing it the arguments. Nothing is left on disk. `--gpu` lets its loops go to the graphics card where the tool has one to give: in a web page (Chapter 8). Anywhere else the program runs as without the word; a card there is for a program that is built.

`scaly build` *`file`* `[-o` *`out`*`]`

> Builds an executable. `--release` optimises it, `-mcpu=native` generates code for the processor at hand, `--pgo-train` and `--pgo` build with a profile (Section A.2.1); `-g` adds debug information, `--gpu` lets a loop run on the graphics card where that pays (Chapter 8), `--target` builds for another system (`wasm32-wasip1`, for WebAssembly), `-L` and `-l` link C libraries, and `--export` keeps every definition visible, for a program that runs others in its own process.

`scaly test` *`file`* `[`*`filter`*`]`

> Runs the tests of the file: every function named `test` or `test_…` that takes nothing and answers an `int`, zero meaning passed; with a filter, only those whose name contains it.

`scaly new` *`name`* `[--lib]`

> Writes a start: a directory with a root file that holds a function, a test for it and a line that prints; with `--lib`, a package (Section 16.3.4).

`scaly install`, **`scaly uninstall`**

> Install the programs a package in a git repository offers, list them, remove one (Section A.2.2).

`scaly clean`

Empties the cache of compiled packages; they are compiled again when a program needs them (Section 16.3.2).

`scaly publish`, **`scaly next`**

For a package's author: record new versions as published, and begin a package's next version (Section 16.3.3).

A.2.1 Building for Production

A program meant to run in production is built in up to three steps, each optional; the toolchain's own programs are built with all three.

`--release` optimises the whole program as one piece: your code, the standard library and every package, linked and optimised together, with every definition the program does not export kept internal.

```
scaly build server.scaly --release -o server
```

`-mcpu=native` generates code for the processor of the machine that builds — the right choice where the program runs on the machine it was built on, or on identical ones (a container image built on its host). Without it the code runs on every processor of the architecture.

A profile makes the optimiser use what the program actually does. Build it instrumented, run it on a representative load, and build again with the profile it wrote:

```
scaly build server.scaly --pgo-train -o server-train
  ./server-train                ; under a typical load; writes default.profraw
  scaly build server.scaly --pgo default.profraw -mcpu=native -o server
```

The instrumented build needs the profile runtime of clang and **llvm-profdata** of LLVM 21 (Homebrew's `llvm@21` on macOS; `llvm-21` and `libclang-rt-21-dev` on Ubuntu). A profile belongs to the sources it was made from: make it again after they changed — a stale one costs speed, never correctness. The Scaly compiler built this way compiles about 30 percent faster than without its profile.

A.2.2 Installing a Program

A program somebody wrote in Scaly and keeps in a git repository is one command away. **dazzle**, for one — a DSSSL processor that turns a DocBook book into HTML or a typeset PDF, and the program this documentation is built with:

```
scaly install github.com/rschleitzer/dazzle dazzle 0.1.0
```

The three arguments are the repository, the package in it and the package's version. **scaly** fetches the package and the packages it uses, builds its programs optimised, and puts them into the `bin` directory of your Scaly installation, which the installer put on your PATH (if it is not there, **scaly install** says so). A fourth argument names one program where a package offers several. This needs **git**.

```
scaly install               ; what has been installed so
  scaly uninstall dazzle      ; remove one of them
```

Section 16.3.5 has the other side: how a package offers programs.

A.3 scalyc

The compiler by itself, with the command line of a C compiler: what **`scaly build`** does in one step, **scalyc** does in the steps a build system asks for.

`-o` *file*

The output: an executable, or with `-c` an object file, with `-S` LLVM IR.

`-c`, `-S`

Compile only, to an object file; emit LLVM IR as text.

`-O0` … `-O3`

The optimisation level; without one the code is not optimised.

`-g`

Debug information, for a source-level debugger (Section 2.2, on Windows Section 2.3).

`-L`*path*, `-l`*name*, *file*`.a`, *file*`.o`

Libraries and objects to link, as a C compiler takes them.

`--target` *triple*, `-mcpu=`*cpu*

The system the code is for, and the processor (`native` for the one at hand; the default runs on every processor of the architecture).

`--jit`, `--run` *`function`*

Compile in memory and run the whole program, or one function of it, in the compiler's own process.

`--no-prelude`

Compile without the standard library: nothing is visible that the file does not declare.

`--task-plan`, `--loop-census`

Reports for the curious: which `for` loops run on every core and why the others do not, which pairs of recursive calls may run on two (Chapter 4, Chapter 5), and which `while` loops could.

`scalyc --help` lists the rest: the stages of the compiler one by one, and the reports it writes about a program.

A.4 scalyls

The language server: an editor starts it and talks to it in the Language Server Protocol, over its standard input and output. It answers with the compiler's own reading of the program — the types inferred, the overload chosen, the region a value lives in — and offers:

- errors as you type, with the compiler's messages
- hover: the type of a name, its declaration, and where a value is allocated
- go to definition, find references, highlight the uses of a name, rename it
- completion, and the parameters of the call being written
- an outline of the file, folding, semantic highlighting, formatting

The Scaly extension for Visual Studio Code starts it (Section 2.2). Any other editor that speaks the protocol can use it: start **scalyls** for files ending in `.scaly`.

B. Standard Library and Packages

B.1 The Standard Library

The standard library is the package `scaly`. Every program sees all of it without a `use` — it is the prelude — unless it is compiled with `--no-prelude`. Its modules:

`scaly.memory`

The region runtime: pages, regions, the frame pages the compiler gives every routine that allocates, and the traps a program raises on a bounds violation, a null unwrap or a lack of memory.

`scaly.containers`

`String` and `StringBuilder`; `Slice[T]`, the borrowed view that never owns; `Vector[T]` of a fixed length, `Array[T]` that grows, `List[T]`; `HashMap[K, V]` and `HashSet[T]` with their builders; `Option[T]` (written `T?`) and `Result[T, E]`. Indexing with `[]` traps outside the bounds, `get(i)` answers an Option, and `put(i, v)` writes.

`scaly.fiber`

Fibers, the scheduler and the evented I/O beneath the operating-system module, the task pool a self-scaling `for` runs on, threads and channels.

`scaly.os`

`Console`, `File`, `Directory`, `Path` and `Tcp`, and the process environment.

`scaly.cluster`

Channels between processes over TCP: a region sent to another node moves with its whole object graph.

`scaly.tensor`

A tape for reverse-mode differentiation — matrix products, activations, attention, the optimizers — living in one region per tape.

`scaly.operation` and `scaly.logic`

The protocol that gives the operators their binding strength (Chapter 14), generated from the levels of the grammar, and `and` and `or` on `bool`, whose right operand is `lazy`.

B.2 Packages Beside It

These packages come with the toolchain. A program declares the ones it uses (`package json 0.2.0`, see Chapter 16), so a program that never speaks TLS does not link OpenSSL:

`http`, `https`, `h3`

An HTTP/1.1 and HTTP/2 server with a router, static files, WebSocket and gRPC; the same server over TLS 1.3; and HTTP/3 over QUIC (ngtcp2).

`tls`

TLS 1.3 for a TCP stream, bound to OpenSSL, and the hashes and key derivations a protocol above it needs.

`json`

RFC 8259 JSON: a pull reader, a value tree built on it, and a writer.

`compress`

gzip for HTTP bodies, over libdeflate.

`pg`, `redis`, `sqlite`, `tds`

Clients of PostgreSQL (protocol version 3, SCRAM authentication, prepared statements, a pool), Redis (RESP2), SQLite (its own library) and Microsoft SQL Server (TDS 7.4).

`scalygpu`

A tensor tape resident on the GPU (Metal).

C. Bootstrap History and the Seed

Scaly is self-hosted, and the committed *seed* — the compiler emitted as its own textual LLVM IR under `seed/` — is the sole bootstrap root. The normal way to build the compiler is from that seed — it is what the installer of Section 2.1.1 does on your machine; it needs only LLVM 21 and a C compiler. A single seed covers every 64-bit little-endian target, Windows included, and the release workflow builds the programs from it natively on macOS, Linux and Windows, on both x86-64 and AArch64.

The seed is refreshed by `tools/seed.sh`: it bootstraps a compiler from the committed seed, re-emits the compiler sources with it, and verifies the *fixed point* — the seed must reproduce itself byte-identically — before `tools/install-seed.sh` installs the new files. Textual IR is diffable and auditable, which is why Scaly ships it instead of opaque per-platform binaries. See `RELEASING.md` in the repository for the full seed-distribution checklist.

Historically, the self-hosted compiler was bootstrapped from a reference implementation written in C++. That C++ stage-0 compiler was retired in July 2026, once the seed became self-sufficient: its sources are frozen under `retired/scalyc0/` as a historical reference, it has no build, and

it cannot parse the current language. Today's build needs no C++ toolchain; C++ comes back only as a check on the seed.

A compiler that compiles itself could carry something its sources do not say and hand it on through every seed — Ken Thompson's *Reflections on Trusting Trust*. So the repository also holds the compiler as C++, under `trust/`, written by the compiler's second back end (`scalyc --emit-cpp`) from the same plan the LLVM back end reads, and written again whenever the seed is. `trust/check.sh` builds a compiler from that C++ with a C++ compiler of your choice, lets it emit the seed, and compares the result with `seed/` byte for byte; if they agree, the seed is what the sources say. This is David A. Wheeler's *diverse double-compiling*. The release workflow runs the check with GCC, in a container without clang, before it builds anything; `trust/README.md` says what the check proves and what stays trusted (the C++ compiler, libLLVM 21, the runtime's few C and assembly files, and the C++ itself).

Hosts with a different data model (32-bit or big-endian) are not covered by the seed; supporting one would mean generalizing the compiler's size and alignment emission off its fixed 64-bit constants and minting a seed for that data model.

D. On the Shoulders of Giants

Hardly anything in Scaly is new. Almost every spelling and almost every mechanism was invented by somebody else, often decades ago, and reached Scaly through a chain of languages that each kept what worked. This appendix follows those chains: for each detail it names the language or the work where the idea appeared first, as far as that can be told, and the languages that carried it on.

Two cautions. A first appearance is hard to prove — where an earlier source turns up, this appendix is wrong and wants correcting. And an entry says where an idea comes from, not that Scaly's author took it from there: many of these ideas arrived second or third hand, and some were found again before their history was looked up.

D.1 The Text of a Program

Comments that begin with `;`

> The semicolon opens a comment in most assembly languages since the 1960s, and in Lisp and its descendants. Scaly has it from Scheme: the compiler's own parser is generated by stylesheets written in DSSSL, which is a Scheme dialect.

Block comments `;* … *;`

> The bracketing pair with an asterisk is PL/I's `/* … */` (1964), which B and C made universal. Scaly keeps the shape and exchanges the slash for its own comment sign.

The line as a statement, the colon between two on one line

> That a line break ends a statement is as old as Fortran (1957). Putting a second statement on the same line after a colon is the habit of the BASIC dialects of the 1970s.

Blocks in braces

> Block structure — a group of statements that counts as one and has names of its own — is ALGOL 60's, written `begin` … `end`. CPL bracketed its blocks with the section sign, BCPL with `$(` and `$)`, and B (Ken Thompson, 1969) with the braces that C handed to everybody. Opening each brace on a line of its own is the style named after Eric Allman, whose BSD sources made it known; it keeps the vertical alignment of `begin` and `end`.

Operators are identifiers

> That an operator is an ordinary name made of operator characters, and its meaning an ordinary definition, is the way of ML and Haskell, and of Smalltalk's binary messages; Scala showed that it carries a whole language without a single operator built into the grammar. The words `and`, `or` and `not` are ALGOL 60's logical operators in the spelling ALGOL 68 and Pascal gave them.

= compares, <> is unequal

> The equals sign for equality is mathematics', and ALGOL 60 kept it that way by giving the assignment a sign of its own. <> is how Pascal, BASIC and SQL write ALGOL's ≠ on a keyboard that has none.

Precedence

> The order of the levels — multiplication before addition, then shifts, comparisons, equality, and the bitwise operators below those — is C's table, which C has from B. It is not a table in the compiler: the levels are written in the standard library as types whose values wait for what follows, and the planner reads an operation by them.

Hexadecimal literals with `0x`

> C.

Expressions embedded in a string

> Interpolation comes from the Unix shell, where `$name` inside double quotes is replaced by its value; Perl, Ruby and Swift extended it from names to expressions. The backtick as the sign of something to be evaluated in place is the shell's too.

D.2 Names and Values

The assignment `:=`

> ALGOL 58, the preliminary report that preceded ALGOL 60, and through ALGOL 60 the whole family: Simula, Pascal, Modula, Ada, Smalltalk in its ASCII form. The sign reads "becomes" and leaves = to mean what it means in mathematics. Before it, Zuse's Plankalkül and Rutishauser's notation wrote an arrow pointing at the target. Go uses the same sign for a declaration; Scaly does not — a name is introduced by `let` or `var`, and `:=` only ever changes one that exists.

`let` and `var`

> `let` for a name bound to a value is Peter Landin's ISWIM (1966), and from there ML and every functional language; `var` for something that varies is Pascal's (1970). The pair as two keywords of equal rank, one for the name that stays and one for the name that changes, is Swift's (2014) — Scala and Kotlin write `val` and `var`.

The name first, then a colon, then the type: `count: int`

> Parameters, fields, typed bindings and the arms of a union are all declared this way, and among programming languages it is Pascal's way (Niklaus Wirth, 1970). ALGOL 60 and Wirth's own ALGOL W before it put the type first, `integer n`, and C followed them. Older languages had the name in front — COBOL (1959) and PL/I (1964) write a name and then its attributes — but without the colon. The colon itself is older than programming: it is how mathematics states what a thing is, `f: A → B`, and from there it entered type theory (N. G. de Bruijn's Automath, 1967) and ML, independently of Pascal. Nearly every language designed since 2000 has gone back to this order, because a type that may be left out is easier to leave out at the end.

The value of a body is its last expression

> Lisp (1958), where everything is an expression; then ALGOL 68 and BLISS among the block-structured languages, and today Ruby, Scala and Rust. The conditional *expression* itself — an `if` that has a value — is John McCarthy's invention, taken into ALGOL 60.

A call without parentheses

> Application by juxtaposition, `print "x"`, is the notation of the lambda calculus and of ML and Haskell; Logo and Ruby brought it to imperative code. That the argument list in parentheses is a tuple — one value — is ML's view of a function of several arguments.

Integer and float types named by their width

`i32`, `u8`, `f64` are Rust's names, which follow LLVM's. `size_t` is C's. The vector types `f64x4`, `u8x16` are named as in Rust's portable SIMD and in WebAssembly.

D.3 Control Flow

`while`, `repeat`, `for … in`

`while` began as a clause of ALGOL 60's `for` statement and became a statement of its own in ALGOL W, BCPL and Pascal. `repeat` is Pascal's word; the unconditional loop it stands for in Scaly is BCPL's `REPEAT`. A `for` that walks over the elements of a collection instead of counting comes from CLU's iterators (1975); the spelling with `in` is the Bourne shell's and Python's.

`break`, `continue`, named loops

`break` is BCPL's and B's, `continue` C's. Leaving an outer loop by its name is Ada's (`exit Outer`), taken up by Java's labelled `break`. A loop that yields the value its `break` carries is Rust's `loop`.

`match` and `case`

The case statement was proposed by Tony Hoare and appeared in ALGOL W (1966); C's `switch` is its best-known child.

A self tail call is a loop

Scheme (Steele and Sussman, 1975) was the first language to require that a call in tail position take no stack, and so made recursion a legitimate way to write a loop.

D.4 Types and Definitions

A structure declared with its fields in parentheses, constructed by a call

Simula 67: a class is declared with parameters, the parameters are its attributes, and creating an object is handing over their values. Most languages after it separated the fields from the constructor that fills them and then spent decades removing the repetition again — Scala's class parameters (2004), Kotlin's primary constructor (2011), the records of C# and Java. Scaly's `define Point (x: int, y: int)` with `Point(1, 2)` is that form, and like Scala, Kotlin and Swift it needs no `new`.

`init` and `this`

`init` as the name of the routine that initializes an object is Objective-C's convention, made a keyword by Swift. `this` is Simula's, through C++. Writing the receiver as the first parameter of a method follows Python's `self` and Rust's.

`function` and `procedure`

The two words are ALGOL's and Pascal's, where they only say whether there is a result. That a function is *checked* to have no effect — no write, no global state, no input or output — goes back to Euclid (1977), a language designed for verifiable programs; Ada's functions with `in` parameters only, Fortran 95's `PURE` and Nim's `func` beside `proc` are the same idea.

`mutable` on a parameter that is written

Ada's parameter modes (`in`, `out`, `in out`, 1983) are the first to make a routine say in its signature what it changes; Swift's `inout` and Rust's `&mut` are the modern forms. A field that cannot change unless it says so is ML's default, and Rust's.

`returns` and `throws`

CLU (Barbara Liskov, 1975) wrote a procedure's result as `returns (int)` and its exceptions as `signals (…)` after the parameter list. `throws` is Java's word for the second.

What happens underneath is Rust's. A routine that can fail returns a `Result[Payload, Error]` — a union of the value and the error — and nothing is unwound: Rust's `Result` gave that type its name and made it the one way a systems language reports failure, after Haskell's `Either` had shown that an error can be an ordinary value. And `try` descends from Rust's `try!` macro, the predecessor of its `?` operator: look at the result, go on with the value, or return the error to the caller at once — which Scaly spells `else throw`. Swift (2015) put the vocabulary of exceptions, `throws` and `try`, on the same mechanism, and Scaly follows it in the words.

Unions and `choose`

A type that is one of several named alternatives, each with its own payload, taken apart by a construct that names the alternative and binds the payload: ALGOL 68's united modes and conformity clause are the first form, Hope (Edinburgh, 1980) and ML made it the centre of a language as algebraic data types with pattern matching, and Rust and Swift brought it to systems programming. The word `when` for an arm is Ada's.

`Option`, the `?`, and `null`

The null reference is Tony Hoare's, introduced into ALGOL W in 1965 and later called by him his billion-dollar mistake. The repair — a separate type for "a value or nothing" — is ML's `option`. The question mark after a type is C#'s (2005), and Swift and Kotlin made `T?` the only type that can hold `null`, as Scaly does. That an optional reference costs no extra space, the null address serving as the empty case, is known from Rust.

Generics in square brackets

CLU had parameterized types written `array[int]`; Eiffel and Scala kept the brackets where C++ and Java chose angle brackets and the parsing trouble that comes with them. Compiling one copy of the code per type argument is how Ada's generics and C++'s templates were always implemented. The names of those copies in the object file follow the Itanium C++ ABI, so that every existing debugger and linker tool can read them.

Lambdas and delegates

The anonymous function is Alonzo Church's lambda calculus (1930s) and Lisp's `LAMBDA`; the backslash as the nearest ASCII character to λ is Haskell's. `delegate` for the named type of a function value together with what it captured is C#'s.

`ref[T]` beside `pointer[T]`, `&` and `*`

The address-of and dereference operators are B's. The reference — an address that is used as if it were the thing itself and cannot do arithmetic — is C++'s (1985).

`is` and `as`

The type test written `v IS T` is Oberon's (Niklaus Wirth, 1987). The pair — `is` to ask for a type and `as` to convert to it, both written after the value — is Object Pascal's as Delphi had it (1995), and went with its designer, Anders Hejlsberg, into C# (2000). Swift, Kotlin and the Clinical Quality Language have the pair from there; Rust has `as`.

D.5 Memory

Scaly frees memory by region: every object belongs to a page that belongs to a call or to a longer-lived owner, and the whole region goes at once. Allocating by lifetime instead of by object is old systems practice — the arenas, zones and pools of compilers and servers, described by David Hanson in 1990. Making it a property of the *language*, with the compiler working out which region each value belongs to, is the work of Mads Tofte and Jean-Pierre Talpin (1994) and its implementation in the ML Kit. Cyclone (2002) brought regions to a C-like language and had the programmer name them; Rust's lifetimes descend from Cyclone.

Scaly stands closer to Tofte and Talpin than to Rust: the regions are inferred, ordinary code

names none, and what the programmer sees is a refusal when a value would outlive its region. That check is escape analysis — the analysis Java and Go compilers use to decide what may live on the stack — turned from an optimization into a rule.

The pattern `0xDD` that `SCALY_POISON` writes over released memory is the one Microsoft's debug heap writes over freed blocks.

D.6 Self-Scaling

That a plain loop runs in parallel when the compiler can prove it safe, with no syntax to ask for it, is the promise of the auto-parallelizing Fortran compilers of the 1980s; Fortress (Guy Steele, Sun, 2006) made the parallel loop the default of a language, and ParaSail (Tucker Taft, 2009) built a whole language around implicit parallelism without a global heap. What they needed and what Scaly has in its checked `function` is a way to know that a body has no effects.

The engine underneath is Cilk's (MIT, 1994): fork-join tasks scheduled by work stealing. When a potential fork becomes a real one is decided by heartbeat scheduling (Acar, Charguéraud, Guatto, Rainey and Sieczkowski, 2018), which bounds the cost of parallelism against the work done.

Fibers are coroutines (Melvin Conway, 1963); the routine that switches between them is modelled on Boost.Context. Channels come from Tony Hoare's Communicating Sequential Processes (1978) by way of occam and Go, and from Go comes the principle that a routine doing input and output looks like any other and needs no `async` mark.

D.7 Modules and Packages

`module` and `use`

> The module as the unit that hides its insides is Modula's (Niklaus Wirth, 1975), and `use` to make another unit's names visible is Ada's. The system as Scaly has it is Rust's: a file declares its child modules by name (`module parser`, Rust's `mod parser;`) and the compiler finds the file; the modules form a tree of paths; `use` shortens a path to its last name or, with a star, opens a whole module; and a package has one root file from which everything else is reached.

The interface beside the implementation

> Modula-2's definition modules and Ada's package specifications separated what a unit promises from how it keeps the promise, and were written by hand. Scaly's are generated from the sources, as Swift's module interfaces are.

The prelude

> A standard library that is visible without being asked for, under that name, is Haskell's.

A package is its sources

> Go: packages are distributed as source and compiled into a cache on the machine that uses them.

D.8 An Unexpected Ancestor

The Clinical Quality Language is a language of the health-care standards body HL7 for stating clinical rules and quality measures — a pure expression language over patient data, and no relative of any systems language. Scaly's author worked with it, and it left marks:

- `define` as the one word that introduces a named thing. A CQL library is a list of `define` statements, each a name and what it stands for.
- The logical operators as words, including `xor`; CQL writes `and`, `or`, `not`, `xor` and has no symbols for them.
- `is` and `as` after a value, to test its type and to convert it — which CQL in turn has from C#.
- A function's result type after the word `returns`.
- An identifier in quotes, which may then contain spaces or any other character.
- The version of a library named where the library is asked for, in the source text itself and not in a file beside it.
- = for equality in a language whose definitions have no need of it for anything else.

D.9 And What Is Scaly's Own?

Less than one would like to claim. The honest account of four candidates:

One `define` for everything

> A structure, a union, a namespace, a constant, a type alias and a delegate type are all a `define` followed by what the name stands for; a structure without fields *is* a namespace. The ambition is not new: BETA (Kristensen, Madsen, Møller-Pedersen and Nygaard, from 1976) folded class, procedure, function and type into a single construct, the pattern, and Zig binds structures, unions and namespaces alike with an ordinary constant declaration. Scaly's form — one keyword, and the shape of what follows decides — was arrived at on its own, and no other language is known to draw the line in the same place.

Precedence is not part of the grammar

> Not Scaly's invention. Prolog let a program declare its operators and their priorities in the 1970s, ML and Haskell have fixity declarations, Swift has precedence groups; all of them

parse an expression as a flat sequence first and give it structure afterwards, as Scaly does.

The operation, read strictly from left to right

Smalltalk reads its binary messages from left to right with no precedence at all, APL from right to left, and Pony refuses to choose and demands parentheses. What Scaly is building goes one step further: the *type* of the value so far says what the next thing means — an argument, a member, an operator that leaves a value waiting for its right side, as a section does in Haskell. That combination has no model that the author knows of. It is also not finished: the binding strength of the operators is such a protocol of waiting types already, but a step is still chosen by the shape of what follows as well as by the type.

No semicolons

The line end as a separator is Fortran's and BASIC's, and Python, Go, Swift, Kotlin and Scala all found ways to spare the programmer the semicolon. Scaly's part is small: the line end and the colon are the same token, so that the grammar needs no rule about where a statement may continue, and the semicolon, no longer wanted, became the comment sign.

One thing is a discipline more than an invention, and the author held to it throughout: *every sign means one thing.* Parentheses are always a tuple — a parameter list is a tuple, the fields of a structure are a tuple, the arguments of a call are a tuple, and because a tuple of one is that one, a single argument needs no parentheses at all. Braces are always a block. Brackets are always a list, of values or of types. And no arithmetic sign is borrowed for something that is not arithmetic: < and > compare and never enclose type arguments, = compares and never assigns, and the signs that are not arithmetic — `#`, `^`, `!`, `?`, the colon, the backslash — are kept out of operators altogether. Most languages descended from C give up each of these somewhere: parentheses that group, call and cast, braces that are blocks here and values there, angle brackets that are generics until they are a shift. The exceptions Scaly allows itself are inherited ones — the = inside ALGOL's `:=`, and B's prefix `*` and `&` for following and taking an address.

What does seem to be Scaly's own lies elsewhere. The lifetime suffixes on a construction — `#` for the caller's region, `^name` for a named one, `!` for a thrown value — are a notation found nowhere else, small as it is. So is the pairing that gives the language its name: regions the compiler infers, functions the compiler checks to be pure, and a plain `for` that runs in parallel because of both. Each part has a history told above; that one language rests all three on each other is new. And no other compiler is known whose grammar is an SGML document.

D.10 The Implementation

A compiler written in its own language

The Lisp compiler of Timothy Hart and Michael Levin (1962) is the first that compiled

itself. Shipping the compiler in a portable intermediate form, so that a new machine can build it without already having it, is the Pascal-P kit from Zurich (1970s); Scaly's seed is the same idea with LLVM's textual IR in place of P-code.

LLVM

Everything behind the emitter — optimization, code generation for every target, the in-process JIT — is LLVM, begun by Chris Lattner and Vikram Adve at the University of Illinois in 2000. Debug information is DWARF.

The grammar as a document

Scaly's grammar is an SGML document (Charles Goldfarb; ISO 8879, 1986), and parser, syntax tree and editor grammar are generated from it by DSSSL stylesheets (ISO/IEC 10179, 1996). The tools were James Clark's SP and Jade, and are now ports of them written in Scaly. This book is DocBook, produced by the same route.

Tests inside the prose

The chapters on expressions, definitions and control flow are generated from documents whose examples are executed as tests. That a program and its explanation are one text is Donald Knuth's literate programming (1984).

The REPL and the language server

The read-eval-print loop is Lisp's, in name and in substance. The language server speaks the protocol Microsoft published in 2016 for Visual Studio Code.